ROLVARN

TEMEL LİNUX PDF

HEDEF KİTLE :

Bu rehber, aşağıdaki gruplar ve ihtiyaçlar göz önüne alınarak titizlikle hazırlanmıştır:

Sıfırdan Başlayanlar: Linux'un ne olduğunu, Windows'tan farklarını ve temel çalışma mantığını (Kernel, Shell) teknik terimlerde boğulmadan öğrenmek isteyenler.

Siber Güvenlik Meraklıları: Kali Linux gibi dağıtımları kullanmayı hedefleyen; dosya izinleri, ağ analizleri (sniffing) ve sistem manipülasyonu konularında sağlam bir temel atmak isteyenler.

Sistem Yöneticisi (SysAdmin) Adayları: Kullanıcı yönetimi, süreç (process) takibi, servislerin kontrolü ve disk yönetimi gibi sunucu yönetiminin yapı taşlarını kavramak isteyenler.

Yazılım Geliştiriciler: Grafik arayüzden bağımsız olarak terminal üzerinde hız kazanmak, sunuculara SSH ile bağlanmak ve geliştirme ortamlarını daha verimli yönetmek isteyenler.

İÇİNDEKİLER

1. GİRİŞ VE TEMEL KAVRAMLAR

- Linux Nedir?
- Linux Felsefesi (Özgürlük, İşbirliği, Saydamlık)
- Linux vs Windows: Karşılaştırmalı Özellikler

2. LİNUX MİMARİSİ VE ÇALIŞMA MANTIĞI

- Linux Nasıl Çalışır?
- Kernel (Çekirdek) Nedir?
- Linux Mimarisi Katmanları (Donanım, Kernel, Shell, Uygulama)
- Linux Önyükleme (Boot) İşlemi ve Aşamaları

3. DOSYA SİSTEMİ VE HİYERARŞİ

- Linux Dosya Sistemi Standartları (FHS)
- Kök Dizin (/) ve Alt Dizinlerin Anlamları (/bin, /etc, /home, /var vb.)

4. LİNUX DAĞITIMLARI

- Dağıtım Nedir ve Bileşenleri Nelerdir?
- Kullanım Amacına Göre Dağıtım Seçimi
- Paket Yönetim Sistemleri Aileleri (Debian, Red Hat, Arch)

5. KABUK (SHELL) VE TERMİNAL

- Shell Nedir, Neden İhtiyaç Duyarız?
- CLI (Komut Satırı) vs GUI (Grafik Arayüz)
- Shell Tipleri (Bash, Zsh, Fish)
- Terminal Nedir?
- Temel Terminal Komutları (clear, history, type)
- Konfigürasyon Dosyaları (.bashrc, .zshrc) ve Ortam Değişkenleri ($PATH)
- Komut Zincirleme Operatörleri (;, &&, ||)

6. DOSYA VE DİZİN İŞLEMLERİ

- Dizin ve Dosya Yolu (Path) Kavramları
- Temel Gezinti Komutları: pwd, cd, ls
- Dosya ve Dizin Oluşturma: touch, mkdir
- Kopyalama İşlemleri: cp

- Taşıma ve Yeniden Adlandırma: mv
- Silme İşlemleri: rm, rmdir
- Dosya Bilgisi ve Inode: file, stat
- Hard Link ve Soft (Symbolic) Link Farkları
- Gelişmiş Kopyalama: dd
- Ağaç Gösterimi: tree

7. ARAMA İŞLEMLERİ

- Dosya ve Dizin Arama: find (Parametreler: -name, -type, -size, -user, -exec)
- Veritabanı Tabanlı Arama: locate
- Komut Konumu Bulma: which
- Dosya İçeriğinde Arama: grep ve Regex Mantığı

8. METİN DÜZENLEME VE İŞLEME

- Metin Editörleri: nano ve vim
- Filtreler ve Pipe (|) Mantığı
- Yönlendirme (Redirection): >, >>, 2>, <
- Standart Girdi, Çıktı ve Hata Kanalları (stdin, stdout, stderr)
- Metin İşleme Araçları:
    - cat, head, tail
    - sort, uniq, wc
    - cut, awk, sed, tee, diff, tr

9. PAKET YÖNETİMİ

- Paket ve Depo (Repository) Mantığı
- APT (Advanced Package Tool) Kullanımı (update, install, remove, purge)
- DPKG ile Manuel Kurulum
- Modern Paketleme: Snap ve Flatpak
- Diğer Paket Yöneticileri (RPM, Pacman)

10. KULLANICI VE GRUP YÖNETİMİ

- Kullanıcı Tipleri ve Yönetimi
- Kullanıcı Oluşturma (adduser, useradd) ve Silme (userdel)
- Kullanıcı Bilgisi Sorgulama (id, who, w, last)
- Kritik Dosyalar: /etc/passwd, /etc/shadow, /etc/skel
- Grup Yönetimi: groupadd, groupdel, groups
- Kullanıcı Yetkilendirme ve Gruba Ekleme (usermod, gpasswd)

11. İZİNLER (PERMISSIONS) VE GÜVENLİK

- Dosya İzinlerini Okumak (ls -l)
- Sahiplik ve Yetkiler (User, Group, Others)
- İzin Değiştirme: chmod (Sembolik ve Octal Yöntem)
- Sahiplik Değiştirme: chown
- Özel İzinler: SUID, SGID, Sticky Bit
- Varsayılan İzinler: umask
- Dosya Öznitelikleri: chattr ve lsattr

12. SÜREÇ (PROCESS) VE SERVİS YÖNETİMİ

- Process Nedir? Foreground ve Background İşlemler
- Süreç İzleme: ps, top
- Süreç Sonlandırma: kill, pkill, killall
- Öncelik Ayarları: nice, renice
- Servis Yönetimi: systemd ve systemctl

13. AĞ YÖNETİMİ (NETWORK MANAGEMENT)

- Temel Ağ Kavramları (IP, MAC, Gateway, Interface)
- Ağ Arayüzlerini Listeleme ve Yönetme (ip, ifconfig)
- Geçici IP Atama ve Promiscuous Mod
- MAC Adresi Değiştirme
- Port ve Bağlantı Kontrolü (netstat, ss)
- Yönlendirme Tablosu (Routing Table)
- Bağlantı Testleri: ping, nc, tracepath
- DNS Sorgulama: dig, nslookup
- Dosya Transferi: wget, curl, scp
- SSH (Secure Shell) ile Güvenli Bağlantı ve Anahtar Yönetimi

Linux Nedir?

Linux, Unix benzeri, açık kaynaklı ve ücretsiz işletim sistemidir. Açık kaynaklıdan kastımız, işletim sisteminin tüm dosyalarına tıpkı bir yazılım projesine ulaşır gibi ulaşabiliyorsunuz ve keyfinize göre özelleştirebiliyorsunuz. Linux Torvald tarafından 1991'de başlanan bu hobi projesi günümüzde dünyanın en büyük projelerinden birine dönüştü. Günümüzde "Linux" kelimesi (GNU/Linux) tabanlı işletim sisteminin kendisine deniyor fakat "Linux" kelimesi teknik olarak kerneli (çekirdek) belirtir.

Nerelerde Linux'a Rastlarız?

Linux tabanlı işletim sistemlerine genellikle günümüzde çoğu sunucuda rastlayabiliriz. Bunların yanı sıra süper bilgisayarlar, network (ağ) cihazları (router, switch, firewall) hatta uzay sistemlerinde bile. Stabilitesi, güvenliği ve modüler yapısı sayesinde çoğu kurumsal altyapının ve son kullanıcının vazgeçilmezi oluyor. (son kullanıcıdan kasıt biz oluyoruz.)

Linux'un Windows ile Karşılaştırmalı Bazı Özellikleri

Linux, Windows işletim sistemlerine oranla çok daha güvenli kabul edilir. Sebebi genel olarak hackerların hedef aldıkları kullanıcıda Windows olduğu düşünerek hareket etmesidir. Ayrıca Windowstaki .exe dosyaları Linux üzerinde direkt çalışmaz. Linux'ta ileriki konularda değineceğimiz gibi disk Windows'takinin aksine "C:\" gibi değil "/" şeklindedir ve bu sistem tektir. Yani Windows'ta birden fazla

C,D,E gibi disk isimleri verebiliyorken Linux tarafında bu tektir ve o da "/" şeklindedir ve hiyerarşik bir yapıdadır. (root dizinin altında /home klasörü onun altında kullanıcılar gibi). Ayrıca Linux Windows'un tek arayüzüne rağmen birden fazla arayüz sunabiliyor. Bunlar: GNOME (sade), KDE (özelleştirilebilir) gibi etkileşimli arayüzleri ve bunların yanı sıra komut satırı arayüzü (CLI) sunabiliyor.

Linux Felsefesi

Linux ruhu işbirliği ve özgürlüğü temsil eder. Gelişimi aşağıdaki ilkeleri göstermektedir.

Özgürlük: Linux, kullanıcılarına özgürlük, kontrol, özelleştirebilme ve kendi dağıtımlarını geliştirebilme fırsatı sunar. Bu da yenilikçi ve güvenli bir ortam sağlar.

İşbirliği: Dünyanın dört bir yanından binlerce insan Linux'un kullanıcı isteklerine doğru cevap verdiğinden ve güvenli olduğundan emin olması adına katkı sağlıyor.

Saydamlık: Linux'un açık kaynaklı yapısı tescillenmiş sistemlerin güvenilirliğini sağlıyor.

Her Şey Bir Dosya: Linux her şeye bir dosyaymış gibi davranır. Örneğin bir donanıma (klavye, yazıcı, flaş disk) dosyaymış gibi davranır.

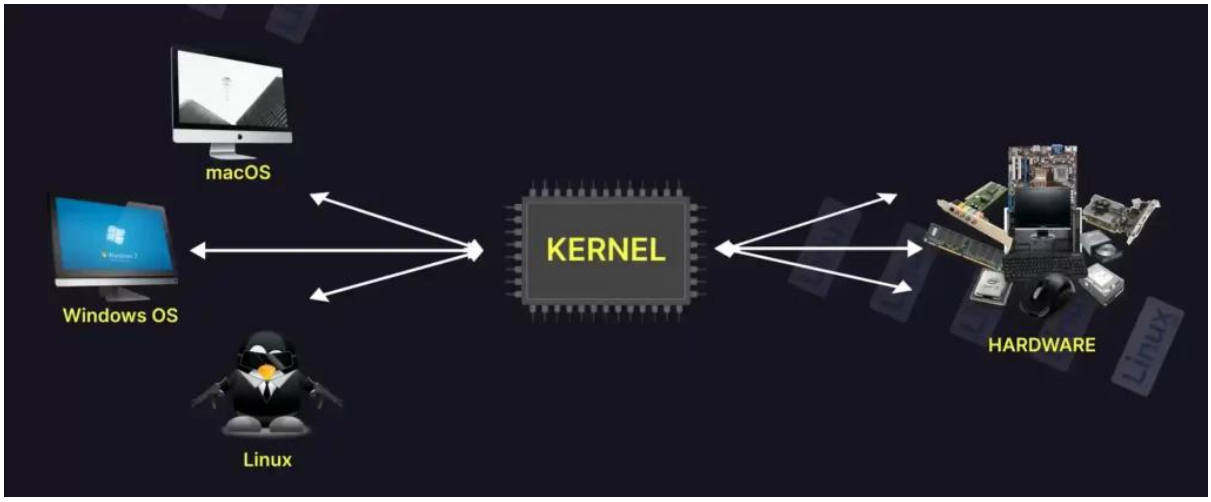
Küçük ve Özelleştirilmiş: "Bir şey yap ve en iyi şekilde yap." felsefesinin benimsendiği bir ilkedir. Toolların çoğu karmaşık işlemlerden ziyade bir işlemi çok iyi bir şekilde yerine getirebilmeleri için tasarlanmıştır. Karışık işlemler küçük işlemlerin bir araya gelmesi ile oluşturulur.

Linux Nasıl Çalışır?

Linux, Windows ve macOS gibi bir işletim sistemidir. Onlardan farkı açık kaynaklı ve ücretsiz olmasıdır. Linux'un kalbinde kernel vardır.

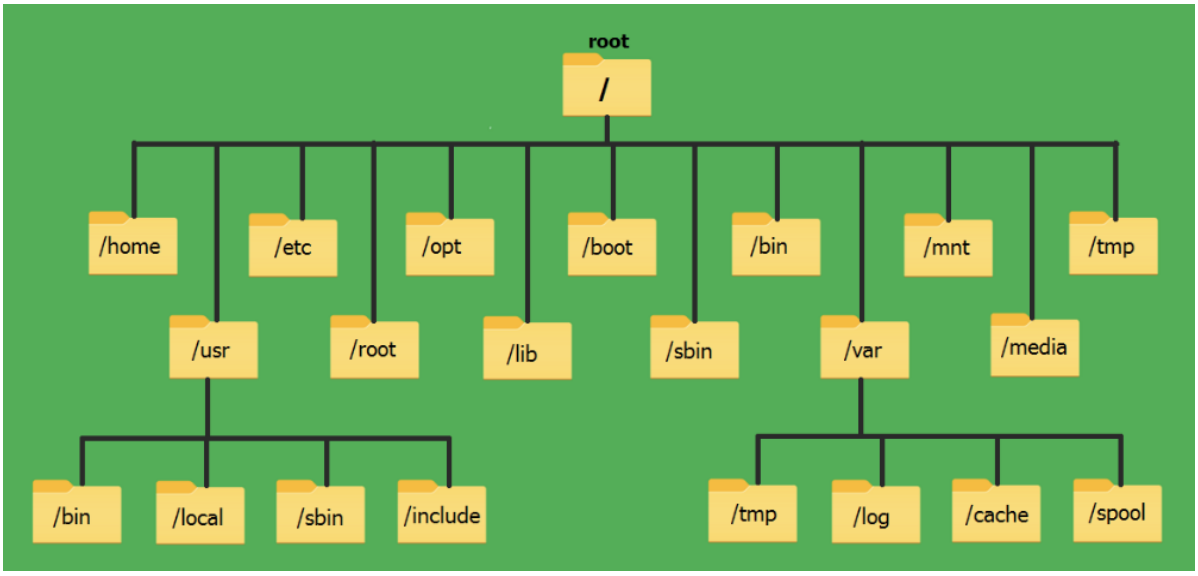
Kernel Nedir?: Kernel (çekirdek) bilgisayarın donanımını, işlemcisini (CPU), belleğini (RAM) ve diğer çevre birimlerinin yönetiminden ve bilgisayardaki tüm yazılım uygulamalarının donanımlar ile güvenli bir şekilde iletişim kurmasından sorumludur. Süreç (process) ve işlem parçacığı (thread) yönetimini sağlar, hangi işlem CPU'da ne kadar süre kalacak? Tüm zamanlama (scheduling) kernelin elindedir. RAM'in nasıl paylaşılacağı, sanal bellek (virtual memory), sayfalama (paging) ve bellek koruması gibi işlemlerin güvenle gerçekleşmesini sağlar. Kernel'in dosya sistemi yönetimi; disk bloklarını, güvenli, tutarlı, izin kontrollü bir dosya-katalog yapısına dönüştürme işidir. Uygulamaların dosya okuması, yazması, klasör işlemleri yapması tamamen kernel'in doğru çalışmasına bağlıdır. Kullanıcının kernele "şunu yap" diye seslenmesini sağlayan kapıdır, sistem çağrılarını (syscall) yönetir. İzinler, kullanıcı ayrımı, bellek koruması gibi olaylar da kernelin elindedir ve eğer bunları doğru yönetmezse sistem savunmasız hale gelebilir. (Bu paragrafta anlamadığınız yerler olabilir. Bu kısımlar işin biraz derinlikleri. Eğer merak ediyorsanız -etmeseniz de mantığını anlamak için- buradaki terimleri araştırmanız tavsiye edilir.)

Linux aynı anda birden fazla işlemi ve kullanıcıyı çalıştırabilen bir yapı sunar. Bu birden fazla kullanıcın aynı anda çalışmasını ve aynı anda işlemler gerçekleştirebileceği anlamına gelir. Bu sistem sunucularda (server) çok büyük bir kolaylık sağlar. Çünkü farklı kullanıcılar farklı işlemleri gerçekleştirmek için eş zamanlı olarak sistemde işlemler gerçekleştirebilir.



Linux Dosya Sistemi Hiyerarşisi Standartları

Bu isim sizi korkutmasın. Burada anlatacağımız konu Linux sistemlerdeki dosya sistemi yapısı. Yukarıda da bahsettiğimiz gibi Linux üzerinde Windows'un aksine C,D gibi diskler yerine "/" gibi tek bir klasörden başlar. Bunu bir ağaç gibi düşünün. Tek başına bir root (kök) dizininden başlıyor isminden de anlaşılabileceği üzere ağacın kökleri gibi bir dağılım gösteriyor.



Dizin (Klasör)	Açıklaması	İçeriği
/root	Sistem dosyalarının başladığı nokta.	Tüm sistem dosyaları.
/bin	Temel kullanıcı komutlarını içerir.	ls,cp,cat,bash

/boot	Sistemin çalışması için gereken dosyaları içerir.	Vmlinuz (Kernel), initrd, GRUB
/dev	Donanımların dosyalarını içerir.	/dev/sda (disk), /dev/tty (komut satırı)
/etc	Sistem konfigürasyon dosyalarının bulunduğu ana merkez.	/etc/passwd, /etc/sshd/ssh_config
/home	Kullanıcıların kişisel dosyalarının bulunduğu yer.	/home/rolvarn /home/user
/lib	Programlar için gerekli olan kütüphanelerin (kod paketleri) bulunduğu yer.	libc.so, Kernel modülleri
/mnt ve /media	Geçici olarak takılmış disklerin dosyalarının bulunduğu kısım. (USB)	/media/usb-disk
/opt	Manuel olarak indirilen uygulamaların bulunduğu dosya.	Tor, DuckDuckGO vb.
/proc	Sanal dosya sistemidir. Sistemin bilgisini tutar.	/proc/cpuinfo /proc/meminfo
/root	Sistemdeki en yetkili (administrator) kullanıcının dizinidir. (root)	Root kullanıcısının dosyaları.
/sbin	Sadece sistemdeki en yetkili kişiler tarafından (administrator) kullanılan komutları içerir.	iptables, fdisk, reboot
/tmp	Geçici dosyalar için.	Uygulama önbellekleri, gereksiz dosyalar.
/usr	İkinci dallanma kısmı. Kullanıcı araçları buradadır.	/usr/bin/python /usr/share
/var	Boyutu sürekli değişen dosyaların tutulduğu yerdir.	Loglar /var/log, websiteler (/var/www)

*Burada temel olarak değinilmiştir. Tüm dosyaların özeline girilmemiştir.

Linux Mimarisi

Linux, kullanıcı ile donanım arasında katmanlı bir yapı sunar. Bu yapıyı anlamak genel olarak Linux mimarisini anlama, sistem yönetimi ve sorun giderme açısından kritik bir öneme sahiptir.

1. Donanım Katmanı (Hardware Layer): Fiziksel bileşenler. CPU, RAM, Disk, Ağ kartı.
2. Çekirdek Katmanı (Kernel Layer): İşletim sisteminin kalbi.

    a. Kernel Kısmı: Donanımı yöneten, tüm kaynakları kontrol eden, ayrıcalıklı modda çalışan çekirdek alanı. Sürücüler ve düşük seviye bileşenler burada bulunur.
    b. Kullanıcı Kısmı: Uygulamaların çalıştığı izole alan. Donanıma erişmek yasaktır; tüm erişim syscall üzerinden kernel'e iletilir.
3. Sistem Kütüphaneleri (System Libraries): Kernel fonksiyonlarını uygulamaların anlayacağı şekilde çevirir.
4. Kabuk Katmanı (Shell Layer): Kullanıcıdan komutları alıp kernele ileten arayüz.
5. Uygulama Katmanı (Application Layer): Kullanıcının program çalıştırdığı (web browser, hesap makinesi, not defteri vb) kısımdır.

Linux Önyükleme (Boot) İşlemi

Linux'u daha iyi anlayabilmek için güç düğmesine bastıktan sonra neler gerçekleştiğini de bilmek önemlidir.

1. BIOS/UEFI: Donanımı kontrol eder ve boot eder.
2. MBR/GPT: Diskin haritasını tutar. Hangi bölümden boot olacak? Disk kaç bölümden oluşuyor? Bölümler nerede bitip nerede başlıyor? Bölümlerin türü ne?
3. Bootloader (Kernel): İşletim sistemi seçme ekranını gösterir ve RAM'in içine seçilen Linux kernelini yükler.
4. Kernel: Donanımları tanır, sürücüleri yükler ve root dizinini kullanılabilir hale getirir ve sisteme bağlar.
5. Init System (Systemd/Sysvinit): İlk işlem olarak çalışır. Arkaplan servislerini, ağ ve giriş ekranını yükler.

Linux Dağıtımları

Linux'un tek başına bir işletim sistemi olmadığını kernelin (çekireğin) adına denildiğini daha önce söylemiştik. Çeşitli amaçlar ve kullanım şekilleri için özelleştirilmiş linux dağıtımları bulunmaktadır. Burada da bunlardan bahsedeceğiz.

Linux Dağıtımları Nedir?

Linux dağıtımları tamamıyla linux kerneli etrafında şekillenmiş işletim sistemleridir. Bu işletim sistemleri linux kerneli, çeşitli yazılımlar, kütüphaneler ve gereksinime göre kullanıcı arayüzü (GUI) bulundururlar. Bu dağıtımlar çeşitli organizasyonlar, topluluklar hatta kişiler tarafından isteklerine ve ihtiyaçlarına göre özelleştirilmiş geliştirilmişlerdir.

Bir Dağıtım için Gerekli Olan Bileşenler

Linux Kernel: İşletim sisteminin kalbidir, donanımları yönetir.

Sistem Kütüphaneleri: Uygulamaları çalıştırmak için gereklidir. Kernel ile iletişimin standart yollarını sağlar.

Paket Yöneticisi: Bir yazılımı indirmek, silmek, güncellemek veya yönetmek için gerekli olan araç.

Önyükleyici: Güç düğmesine bastıktan sonra bilgisayarda hangi işlemlerin gerçekleşeceğini belirleyen yazılım.

Grafikli Kullanıcı Arayüzü (GUI): Her dağıtımda olmasına gerek yoktur fakat kolay kullanım için çoğu dağıtım kullanır. Örneğin: GNOME (modern, sade,güçlü donanım ister) KDE (aşırı özelleştirilebilir) ya da XFCE (eski arayüz fakat performans için en uygunlardan)

Aşağıdaki tabloda bununla ilgili fikir edinebilirsiniz.

Kullanıcı Tipi	Önerilen Dağıtım	Nedeni
Çaylaklar	Mint, Ubuntu	Windowsa çok yakın. Kurulumu çok basit.
Siber Güvenlik	Kali Linux, Parrot OS	Beraberinde 100'e yakın siber güvenlik aracı ile gelir.
Sunucu	Ubuntu Server, Debian	GUI yok. Çok stabil ve düşük sistem gereksinimi.
Yazılım Geliştiricileri	Fedora, Manjaro	En güncel programlama dillerini ve kütüphanelerini içerir.
Eski Bilgisayarlar	Lubuntu, Xubuntu	Çok düşük sistem gereksinimi. Eski donanımlarda bile çok hızlı çalışır.
Linux Uzmanları	Arch	Her şeyi sıfırdan kurarsınız. Tam kontrol sağlar.

Linux Ailelerinin Paket Yönetim Sistemleri

Linux dünyası paket formatına göre ana ailelere ayrılmıştır.

Aile	Paket Formatı	Paket Yöneticisi	Örnek Dağıtımlar	Özellik
Debian	.deb	apt, dpkg	Ubuntu, Kali, Mint, Debian	En yaygın aile. Geniş çaplı bir destek kitlesine sahiptir.
Red Hat	.rpm	dnf, yum, rpm	Fedora, CentOS, RHEL, Rocky	Kurumsal dünyada standarttır.
Arch	.pkg.tar.zst	pacman	Arch, Manjaro	Sürekli güncel.

Bazı popüler Linux dağıtımları:

Ubuntu:

Debian tabanlıdır.

Hedef kitle: Herkes (Masaüstü ve Sunucular)

Özellikler: En popüler dağıtımdır. Bu dağıtım ile ilgili sorunlarınızı internet üzerinde kolayca bulabilirsiniz.

Arayüz: Varsayılan olarak GNOME.

Kali Linux:

Debian tabanlıdır.

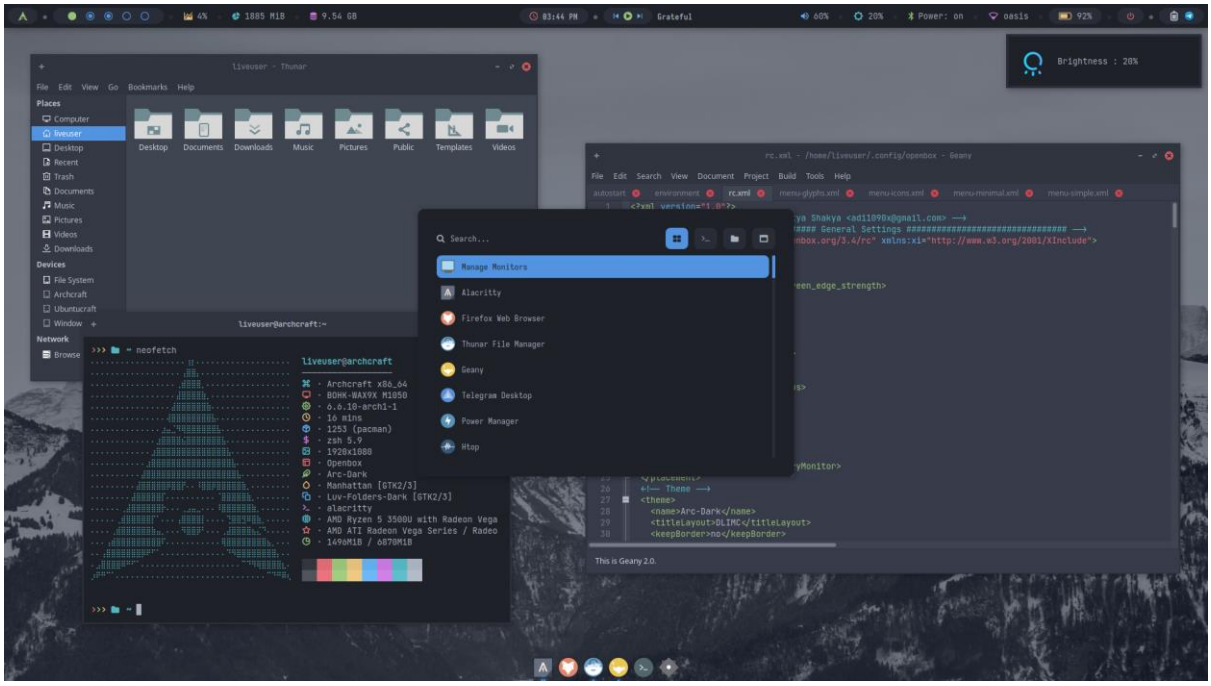
Hedef kitle: Sızma testi uzmanları.

Özellikler: Günlük kullanım için uygun değildir. Hazır araçlar ile gelir (WireShark, nmap, metasploit).

Arch Linux:

Hedef kitle: Sistemin kontrolünü tamamen elinde tutmak isteyen deneyimli kullanıcılar.

Özellikler: Grafik kurulum arayüzü yoktur, kurulum terminal üzerinden yapılır. “Rolling Release” modelini kullanır; ayrı sürümler bulunmaz, sistem sürekli güncellenir. Güncellemeler kullanıcıdan takip ve müdahale gerektirebilir.



Yayın Modelleri: Sabit Model ve Sürekli Güncellenen Model

Sabit Model (Fixed Release): Windows’un birkaç yıl aralıklı olarak yeni modeli çıkarması gibi (Windows 10 -> Windows 11)

Sürekli Güncellenen Model (Rolling Release): Windows 10 tarzı bir versiyon numarası yoktur. Sürekli güncelleme alır ve sürekli en güncel hali kullanılır.

Linux Kabuğu (Shell)

Shell Nedir, Neden İhtiyaç Duyarız?

Bilgisayar donanımları (CPU, RAM, Disk) tek başına işlevsizdir ve bu nedenden dolayı kernel kullanırız. Fakat kernel karışık bir yapıya sahiptir ve 0 ve 1leri anlar. İşte burası tam da shell’in ortaya girdiği yer.

Shell’in Görevi:

Shell insanların daha iyi anlayabileceği (0,1 gibi karışık ve anlaşılması zor olmayan) tarzda komutları kernelin anlayabileceği şekle çevirir ve kernelden gelen cevabı da anlayabileceğimiz şekilde ekrana geri yazdırır. Tıpkı bir çevirmen gibi. Linux’ta iki adet shell vardır: Komut satırı (CLI) ve Grafiksel Arayüz (GUI)

Komut Satırı - Command Line (CLI)

Komut satırı metin tabanlı bir iletişim yöntemine sahiptir. Terminal adı verilen ekranı açarsınız ve birtakım kodlar girip bilgisayarınızı yönetirsiniz.

```
[root@localhost ~]# ping -q fa.wikipedia.org
PING text.pmtpa.wikimedia.org (208.80.152.2) 56(84) bytes of data.
^C
--- text.pmtpa.wikimedia.org ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 540.528/540.528/540.528/0.000 ms
[root@localhost ~]# pwd
/root
[root@localhost ~]# cd /var
[root@localhost var]# ls -la
total 72
drwxr-xr-x. 18 root root 4096 Jul 30 22:43 .
drwxr-xr-x. 23 root root 4096 Sep 14 20:42 ..
drwxr-xr-x.  2 root root 4096 May 14 00:15 account
drwxr-xr-x. 11 root root 4096 Jul 31 22:26 cache
drwxr-xr-x.  3 root root 4096 May 18 16:03 db
drwxr-xr-x.  3 root root 4096 May 18 16:03 empty
drwxr-xr-x.  2 root root 4096 May 18 16:03 games
drwxrwx--T.  2 root gdm  4096 Jun  2 18:39 gdm
drwxr-xr-x. 38 root root 4096 May 18 16:03 lib
drwxr-xr-x.  2 root root 4096 May 18 16:03 local
lrwxrwxrwx.  1 root root   11 May 14 00:12 lock -> ../run/lock
drwxr-xr-x. 14 root root 4096 Sep 14 20:42 log
lrwxrwxrwx.  1 root root   10 Jul 30 22:43 mail -> spool/mail
drwxr-xr-x.  2 root root 4096 May 18 16:03 nis
drwxr-xr-x.  2 root root 4096 May 18 16:03 opt
drwxr-xr-x.  2 root root 4096 May 18 16:03 preserve
drwxr-xr-x.  2 root root 4096 Jul  1 22:11 report
lrwxrwxrwx.  1 root root    6 May 14 00:12 run -> ../run
drwxr-xr-x. 14 root root 4096 May 18 16:03 spool
drwxrwxrwt.  4 root root 4096 Sep 12 23:50 tmp
drwxr-xr-x.  2 root root 4096 May 18 16:03 yp
[root@localhost var]# yum search wiki
Loaded plugins: langpacks, presto, refresh-packagekit, remove-with-leaves
rpmfusion-free-updates                                                   | 2.7 kB     00:00
rpmfusion-free-updates/primary_db                                        | 206 kB     00:04
rpmfusion-nonfree-updates                                                | 2.7 kB     00:00
updates/metalink                                                         | 5.9 kB     00:00
updates                                                                  | 4.7 kB     00:00
updates/primary_db                        73% [=======================         ]  62 kB/s | 2.6 MB     00:15 ETA
```

Neden Komut Satırı Kullanayım?

Hız: Klavye ile yapılan dosya işlemleri ve program kurulumları, grafik arayüze göre daha hızlıdır.

Otomasyon: Tek bir komut veya script ile yüzlerce işlemi otomatik olarak yapabilirsin.

Uzaktan Erişim: Sunuculara SSH ile bağlanıldığında grafik arayüz yoktur; tüm işlemler komut satırından yapılır. (Sunucu veya SSH kullanıyorsanız)

Grafik Arayüz - Graphical Interface (GUI)

```
kali@kali: ~
File  Actions  Edit  View  Help
kali@kali:~$ #the command executed below is used to print a message
kali@kali:~$ echo "This is a terminal"
This is a terminal
kali@kali:~$ #the comamnd executed below is used to get the present working directory
kali@kali:~$ pwd
/home/kali
kali@kali:~$
```

Pencereler, ikonlar, mouse ve dahasını içeren bir arayüz. Windows ve macOS gibi.

Shell Tipleri (Bash, Zsh, Fish)

Linux üzerinde çeşitli sheller bulunur ve ihtiyacınıza uygun olanı seçip kullanabilirsiniz.

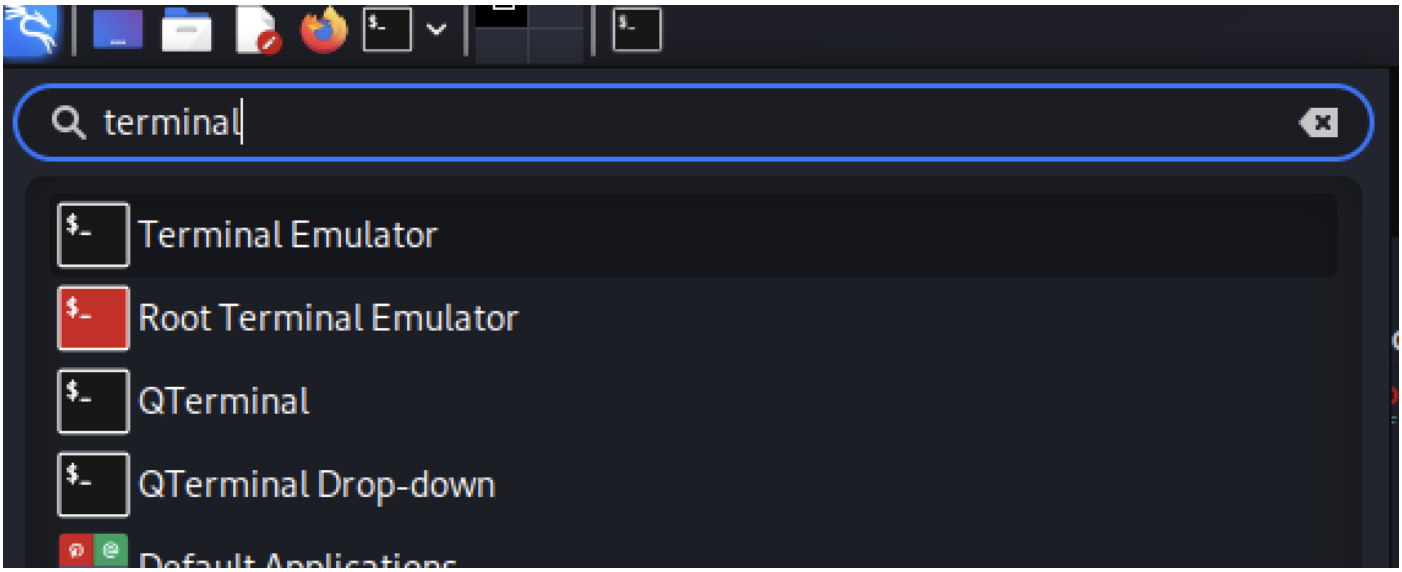
1. Bash (Bourne Again Shell): En yaygın ve standart kabuktur, çoğu Linux dağıtımında varsayılan olarak gelir. Script yazımı, otomasyon ve sunucu ortamları için idealdir; çünkü hemen her sistemde çalışır.
2. Zsh (Z Shell): Bash ile büyük ölçüde uyumludur ancak daha gelişmiş otomatik tamamlama ve tema desteği sunar (macOS'te varsayılandır). Terminali yoğun kullanan geliştiriciler için daha yüksek verim sağlar.
3. Fish: Kullanıcı dostu bir kabuktur; komutları yazarken renklendirir ve akıllı öneriler sunar. Öğrenmesi kolaydır ancak Bash uyumlu olmadığı için taşınabilir script'ler için uygun değildir. Yarın başka yerde çalıştırmak için bir script yazarsanız bash ile uyumlu olmadığı için farklı bir Shell'de çalışmayabilir.

Kullandığınız shell tipini görmek için komut satırına echo $SHELL yazabilirsiniz.

```
┌──(root㉿kali)-[~]
└─# echo $SHELL
/usr/bin/zsh
```

Terminal Nedir?

Terminal uygulaması kullanıcıların shell ile iletişim kurmasını sağlar. Siz bir komut girersiniz ve çıktısını size gösterir.



Bazı terminal komutları:

- *clear* (ekranı temizler): Terminal ekranı çok karıştığında ekranı komple temizlemek için kullanılır.
- *history* (eski komutlar): Eski komutlara ulaşmak için yazılır. Dikkat ederseniz hepsi sıralı gidiyor. Bunun nedeni burada direkt olarak eski bir kodu kolayca çalıştırabilmek. Örnek kullanım: !1 yazarsanız "nano /etc/apt/sources.list" komutu terminal ekranına gelir.

```
┌──(root㉿kali)-[~]
└─# history
    1  nano /etc/apt/sources.list
    2  cat /etc/apt/sources.list
    3  nano /etc/apt/sources.list
    4  cat /etc/apt/sources.list
    5  apt-get update
    6  netdiscover
```

- *type* (komut tipi): Bir komut yazıldığında shell onu çalıştırmak için belirli bir sıralama izler. Önce, komutun bir **alias** olup olmadığına bakar; alias, bir komutun başka bir komutla veya ek parametrelerle eşleştirilmiş kısa adıdır (örneğin ls komutunun otomatik olarak ls --color=auto çalıştırması gibi). Eğer alias yoksa, shell komutun kendi içinde yer alan bir **builtin** olup olmadığını kontrol eder; builtin'ler cd veya exit gibi, disk üzerinde ayrı bir program dosyası olmayan ve doğrudan shell tarafından çalıştırılan komutlardır. Komut ne alias ne de builtin ise, shell bu kez **PATH** adlı ortam değişkeninde tanımlı dizinleri sırayla dolaşarak aynı isimde bir program arar ve bulduğu ilk dosyayı çalıştırır (örneğin /usr/bin/grep). **type komutu**, bu sürecin sonunda yazılan bir komutun gerçekte alias mı, shell builtin mi yoksa PATH üzerinden bulunan gerçek bir program mı olduğunu gösterir ve shell'in hangi komutu çalıştıracağını açıkça belirtir.

```
┌──(root㉿kali)-[~]
└─# type cd
cd is a shell builtin

┌──(root㉿kali)-[~]
└─# type ls
ls is an alias for ls --color=auto

┌──(root㉿kali)-[~]
└─#
```

Shell Konfigürasyon Dosyaları (.bashrc, .zshrc)

Shell yapılandırma dosyaları (örneğin **.bashrc** ve **.zshrc**), terminal her açıldığında shell tarafından otomatik olarak okunan ve kullanıcıya özel ayarların yüklendiği gizli dosyalardır; bu dosyalar genellikle kullanıcının home dizininde (/home/kullanici veya ~) bulunur. Bash kullanıyorsanız ayarlar **.bashrc**, Zsh kullanıyorsanız **.zshrc** dosyasına yazılır ve buraya eklenen alias'lar, ortam değişkenleri veya fonksiyonlar her yeni terminal oturumunda otomatik olarak geçerli olur. Örneğin clear komutunu her seferinde uzun uzun yazmamak için bu dosyaya alias c='clear' satırı eklenebilir; dosya kaydedildikten sonra source ~/.bashrc (veya Zsh için source ~/.zshrc) komutu çalıştırılarak ayarlar terminali kapatmadan hemen yüklenir.

Ortam Değişkenleri

Environment variables (ortam değişkenleri), işletim sistemi ve çalışan programların nasıl davranacağını belirleyen dinamik değerlerdir ve shell oturumu boyunca geçerlidir. Örneğin $HOME kullanıcının ana dizinini, $USER aktif kullanıcı adını, $PWD bulunulan dizini, $SHELL kullanılan shell programını, $PATH ise komutların aranacağı dizinlerin sırasını tutar. Bu değişkenlerin değerleri echo $DEĞİŞKEN_ADI komutu ile ekrana yazdırılabilir; örneğin echo $HOME kullanıcının home dizinini gösterir. $PATH değişkeni özellikle kritiktir çünkü bir komut yazıldığında Linux bu değişkende listelenen dizinleri soldan sağa doğru kontrol eder ve komutu bulduğu ilk yerden çalıştırır; dizinler iki nokta (:) ile ayrılır. Bu yüzden PATH'in içeriği, hangi programın gerçekten çalıştırılacağını doğrudan etkiler.

Değişken	Açıklama
$HOME	Kullanıcının home dizini
$USER	Geçerli kullanıcı adı
$PWD	O anki bulunulan dizin
$SHELL	Kullanılan shell'i gösterir
$PATH	Komutu hangi dizinlerde ve hangi sırayla arayacağını belirleyen listeyi gösterir

Command Chaining Operators (Komut Zincirleme Operatörleri)

Bu operatörler, birden fazla komutu tek satırda yazmanı sağlar ve ikinci komutun çalışıp çalışmayacağına birincinin sonucuna göre karar verir.

1) Sequential Execution (;)

"Ne olursa olsun devam et" anlamına gelir. Shell komutların sonuçlarını umursamadan tek tek çalıştırır. Örneğin:

- echo "Command" çalışır → ekrana yazı basılır.
- Shell hata var mı yok mu bakmaz.
- ls komutu hemen ardından çalışır.

2) Continue If Successfull (&&)

- mkdir test_directory çalışır.
- Dizin başarıyla oluşturulduğu için komut başarılı sayılır.
- cd test_directory çalıştırılır.

3) Run if Error (||)

“Hata varsa çalıştır” anlamına gelir. İlk komutun başarısız olması durumunda diğer komut çalışır. Örneğin:

Komut: cd nonexistent_directory || echo "No such directory!"

- cd nonexistent_directory çalışır.
- Dizin bulunamadığı için hata oluşur.
- || devreye girer.
- echo "No such directory!" çalışır.

Script Başlatma (Shebang #!)

Bir dosyanın shell script olarak çalıştırılabilmesi için en üst satırına **shebang (#!)** eklenir; bu satır işletim sistemine dosyanın hangi programla çalıştırılması gerektiğini söyler. #!/bin/bash ifadesi, “bu dosyadaki komutları **/bin/bash** kullanarak çalıştır” anlamına gelir ve dosya doğrudan çalıştırıldığında (./script.sh gibi) sistem bu satıra bakarak Bash’i başlatır. Shebang olmadan dosya yalnızca metin olarak görülür ve hangi shell ile çalıştırılacağı belirsiz kalır, bu yüzden script’lerin doğru ve tutarlı şekilde çalışması için shebang satırı kritiktir.

Örneğin ilk satırı #!/bin/bash olan bir dosya çalıştırıldığında, sistem dosyanın içindeki komutları Bash ile çalıştırır; ilk satırı #!/usr/bin/env python3 olan bir dosya ise Python ile çalıştırılır. Böylece aynı dosya adı ve çalışma şekli korunurken, hangi programın kullanılacağı shebang satırıyla açıkça belirlenmiş olur.

Dosya Yönlendirme

Dizin (Directory) ve Dosya Yolu (Path) Nedir?

Linux’te her şey bir dosya olarak kabul edilir ve bu dosyalar dizinler (directories) içinde hiyerarşik bir yapı halinde organize edilir. Tüm sistemin başlangıç noktası root dizini (/)'dir ve Windows’taki C:\ benzeri bir rol oynar; ancak Linux’te tek bir kök dizin vardır. Path (yol), bir dosya veya dizinin sistem içindeki adresini ifade eder. Yol tanımı kök dizinden başlayarak yapılırsa absolute path – tam yol - (örneğin /home/john/pictures/holiday.jpg), mevcut bulunulan dizine göre yapılırsa relative path - yakın yol - olarak adlandırılır (örneğin pictures/holiday.jpg ---> Anlık olarak bulunduğunuz dizin ile birlikte yazar.).

Bazı Dosya İşlemleri Komutları

Pwd (Print Working Directory - Çalışılan Dizini Yazdır)

pwd (Print Working Directory) komutu, dosya sistemi içinde **o anda hangi dizinde bulunduğunuzu** gösterir. Terminalde çalıştığınız her komut bulunduğunuz dizine göre anlam kazandığı için bu bilgi temel öneme sahiptir. Örneğin pwd çıktısı /home/user ise, şu anda kullanıcıya ait home dizininin içindesiniz demektir ve dosya işlemleri bu dizine göre yapılır.

```
┌──(root㉿kali)-[~]
└─# pwd
/root
```

realpath, basename ve dirname (Tam yol, dosya ismi, dizin ismi)

Dosya yollarını anlamak ve parçalamak için bazı yardımcı komutlar kullanılır. realpath, bir dosyanın veya dizinin tam (absolute) yolunu gösterir; örneğin realpath notes.txt çıktısı /home/user/notes.txt ise, dosyanın sistemdeki kesin konumu budur. basename, bir yolun yalnızca dosya adını döndürür; /home/user/notes.txt için sonuç notes.txt olur. dirname ise yolun yalnızca dizin kısmını verir; aynı örnekte çıktı /home/user olur. Bu komutlar özellikle script yazarken ve yol manipülasyonu yaparken kullanılır.

```
┌──(root㉿kali)-[~/Desktop]
└─# realpath test.txt
/root/Desktop/test.txt

┌──(root㉿kali)-[~/Desktop]
└─# basename test.txt
test.txt

┌──(root㉿kali)-[~/Desktop]
└─# dirname test.txt
.
```

cd (Change Directory – Dizin Değiştir)

cd (Change Directory) komutu, dosya sistemi içinde dizinler arasında geçiş yapmanızı sağlar. cd dizin_adi yazıldığında belirtilen dizine girilir; eğer tam bir yol (/var/log gibi) verilirse, bu absolute path kullanılarak doğrudan o konuma gidilir. cd .. komutu, bulunduğunuz dizinin bir üst dizinine çıkmanızı sağlar; örneğin /home/user/Downloads içindeyken cd .. yazarsanız /home/user dizinine geçersiniz.

Bazı Özel Cd Komutu Kullanımları

cd ~ komutu, nerede olursanız olun sizi doğrudan **kendi home dizininize** (/home/username) götürür;
cd - komutu ise shell'in hafızasını kullanır ve sizi **bir önce bulunduğunuz dizine geri döndürür**. Örneğin önce /home/user dizininden /etc dizinine giderseniz, ardından cd - yazdığınızda shell sizi otomatik olarak

tekrar /home/user dizinine götürür. Bu komut özellikle iki dizin arasında hızlıca gidip gelmek gerektiğinde kullanılır ve çoğu kullanıcı tarafından geç fark edilir.

Listeleme (Listing)

ls (List) komutu, bir dizinin içeriğini listelemek için kullanılır. Tek başına çalıştırıldığında yalnızca dosya ve dizin adlarını gösterir; bu yüzden pratikte neredeyse her zaman **parametrelerle (flags)** birlikte kullanılır. Parametreler, dosyaların ayrıntılarını görmeyi, gizli dosyaları listelemeyi veya belirli bir kritere göre sıralamayı sağlar.

```
┌──(root㉿kali)-[~/Desktop]
└─# ls
test.txt
```

Sık Kullanılan ls Parametreleri:

-l parametresi = dosyaları detaylı liste halinde gösterir; izinler, sahip, grup, boyut ve tarih gibi bilgileri içerir.
-a parametresi = adı nokta (.) ile başlayan gizli dosyaları da listeye ekler.
-h parametresi = dosya boyutlarını byte yerine insan okunabilir formatta (KB, MB gibi) gösterir.
-t parametresi = dosyaları son değiştirilme tarihine göre sıralar.
-r parametresi = mevcut sıralamayı tersine çevirir.
-R parametresi = yalnızca bulunduğunuz dizini değil, alt dizinlerin içeriğini de listeleyerek tüm dizin ağacını gösterir.
-S parametresi = dosyaları boyutlarına göre sıralar (varsayılan olarak büyükten küçüğe).

Bu parametreler birlikte de kullanılabilir (örneğin ls -lah).

Detaylı Listeleme (ls –l)

ls -l çıktısı, her dosya veya dizin için sütunlar halinde bilgi verir. İlk sütun dosya türünü ve izinleri gösterir; d ile başlayanlar dizin, - ile başlayanlar normal dosyadır. Ardından gelen sayı, dosyanın hard link sayısını belirtir. Sonraki iki sütun dosyanın sahibi olan kullanıcıyı ve grubu gösterir. Bunu dosya boyutu (byte cinsinden), son değiştirilme tarihi ve en sonda dosya veya dizin adı takip eder. Bu çıktı sayesinde bir dizindeki dosyaların türünü, kim tarafından sahiplenildiğini ve çalıştırılabilir olup olmadığını tek bakışta anlayabilirsiniz.

Hard Link Sayısı Nedir?

Linux dosya sisteminde bir dosyanın asıl kimliği inode adı verilen veri yapısıdır; inode, dosyanın içeriğinin disk üzerinde nerede durduğunu, boyutunu, sahibini, izinlerini, zaman bilgilerini ve hard link sayısını tutar, ancak dosya adını tutmaz. Dosya adları, dizinler içinde inode numaralarına işaret eden etiketlerdir. Bir dosya oluşturulduğunda, sistem önce bir inode yaratır, ardından bu inode'a bir isim bağlar ve bu yüzden hard link sayısı 1 olur. Aynı inode'a ln komutu ile ikinci bir isim bağlandığında, dosya kopyalanmaz; yalnızca aynı inode'a işaret eden ek bir ad oluşturulur ve bu durum ls -l çıktısında hard link sayısının artmasıyla görülür. Dosyanın içeriği inode üzerinden yönetildiği için, isimlerden biri üzerinden yapılan değişiklik diğer isimden de aynı şekilde görülür; inode'a bağlı isimler tek tek silinse bile, inode'a bağlı son isim kaldırılmadıkça dosya diskten gerçekten silinmez.

Gizli Dosyaları Görmek (ls- la)

ls -la komutu, detaylı listelemeye ek olarak gizli dosyaları da gösterir. Nokta (.) ile başlayan dosyalar genellikle **konfigürasyon dosyalarıdır** ve normalde görünmez. Bu çıktıda ayrıca . mevcut dizini, .. ise bir üst dizini temsil eder. Özellikle kullanıcı ayarları ve shell yapılandırmaları bu komutla görünür hale gelir.

```
┌──(root㉿kali)-[~/Desktop]
└─# ls -la
total 12
drwxr-xr-x  2 root root 4096 Dec 22 11:13 .
drwx------ 20 root root 4096 Dec 22 09:15 ..
-rw-r--r--  1 root root    8 Dec 22 11:14 test.txt
```

Zamana Göre Sıralama (ls -lt)

ls -lt komutu, dosyaları en son değiştirilen en üste gelecek şekilde sıralar. Bu, "en son hangi dosya üzerinde çalıştım?" sorusuna hızlı cevap almak için kullanılır. Çıktının en üstündeki dosya, en güncel olandır.

Alt Dizinlerle Birlikte Listeleme (ls -R)

ls -R komutu, yalnızca mevcut dizini değil, içindeki tüm alt dizinleri de rekürsif olarak listeler. Çıktıda önce ana dizin, ardından her alt dizin kendi başlığı altında gösterilir. Bu komut, bir dizin yapısının tamamını tek seferde görmek için kullanılır ancak büyük dizinlerde çok fazla çıktı üretebilir.

Boyuta Göre Sıralama (ls -lS)

ls -lS komutu, dosyaları boyutlarına göre sıralayarak listeler; varsayılan olarak en büyük dosya en üstte yer alır. Disk alanını neyin doldurduğunu hızlıca görmek için en pratik yöntemlerden biridir.

NOT:

ls, yalnızca “dosya isimlerini gösteren basit bir komut” değildir; doğru parametrelerle kullanıldığında bir dizinin yapısını, güvenliğini, sahipliğini ve kullanım durumunu analiz etmeni sağlar. ls -l çıktısını okuyamayan biri, Linux dosya sistemiyle gerçekten çalışıyor sayılmaz.

Directory Stack (pushd ve popd)

Directory Stack Nedir?

Dizin yığını (directory stack), shell’in ziyaret edilen dizinleri belirli bir sırayla bellekte tutan geçici bir liste yapısıdır. Bu yapı, dizinlerin girilme sırasına göre saklanmasını sağlar ve en son girilen dizin en üstte yer alır. Amaç, kullanıcı birden fazla dizin arasında dolaşırken önceki konumları kaybetmeden geri dönebilmeyi mümkün kılmaktır; yani dizin yığını, “nerelerden geçildiğini hatırlayan” bir mekanizma olarak çalışır.

pushd ve popd Komutları

pushd ve popd komutları, dizinler arasında geçici olarak gidip gelmeyi kolaylaştırmak için kullanılan dizin yığını (directory stack) mantığıyla çalışır. pushd komutu, sizi belirtilen dizine götürürken bulunduğunuz mevcut dizini bellekte tutulan bir yığına ekler; örneğin pushd /var/log komutu çalıştırıldığında shell hem /var/log dizinine geçer hem de önceki dizini (örneğin ~/project) hafızaya kaydeder. Ardından popd komutu çalıştırıldığında, shell bu yığının en üstündeki dizini alır ve sizi tam olarak bıraktığınız yere geri götürür. Bu yöntem, özellikle bir dizinde çalışırken başka bir konumu kısa süreli kontrol edip hiçbir yolu tekrar yazmadan geri dönmek gerektiğinde kullanılır ve cd ile yapılan manuel geri dönüşlere göre daha kontrollüdür.

Ağaç Gösterimi (Tree View) tree

tree komutu, bir dizinin içindeki dosya ve alt dizinleri hiyerarşik bir ağaç yapısı şeklinde görselleştirerek gösterir. Dizinler dallar gibi, alt dizinler ise bu dalların devamı olarak listelenir; bu sayede klasörlerin birbiriyle olan ilişkisi tek bakışta anlaşılır. Örneğin tree -L 2 komutu, bulunduğunuz dizinden başlayarak yalnızca 2 seviye derinliğe kadar olan yapıyı gösterir; -L parametresi derinlik sınırını belirler ve büyük dizinlerde çıktının kontrolsüz büyümesini engeller. tree, özellikle karmaşık proje dizinlerinde veya bir sistemin genel klasör yapısını hızlıca kavramak için kullanılır.

```
┌──(root㉿kali)-[/home]
└─# tree
.
`-- kali
    |-- Desktop
    |   `-- test.txt
    |-- Documents
    |-- Downloads
    |   `-- ,.png
    |-- Music
    |-- Pictures
    |-- Public
    |-- Templates
    `-- Videos

10 directories, 2 files
```

Dosya ve Dizin İşlemleri

Linux'te "Everything is a file" felsefesi geçerlidir; yani resimleriniz, metin dosyalarınız ve hatta klavye, disk gibi donanımlar bile sistem tarafından dosya olarak temsil edilir. Bir dosya oluşturulduğunda disk üzerinde aslında iki ayrı şey meydana gelir: data ve inode. Data, dosyanın gerçek içeriğini yani yazı, resim veya bilgiyi barındıran kısımdır. Inode (Index Node) ise dosyanın kimlik kartı gibidir; dosyanın sahibini, izinlerini, boyutunu, zaman bilgilerini ve içeriğinin disk üzerinde nerede bulunduğunu tutan bir metadata kaydıdır. Dosya adı ise inode'un parçası değildir; yalnızca dizin içinde bu inode'a işaret eden bir etikettir. Bu nedenle Linux'te dosyanın gerçek varlığı inode'a bağlıdır, isim ise değiştirilebilir veya birden fazla olabilir.

Dosya Oluşturma (touch)

touch komutu, verilen isimde bir dosya varsa onun zaman damgasını (timestamp) günceller, dosya yoksa boş bir dosya oluşturur. Örneğin touch notes.txt komutu çalıştırıldığında, disk üzerinde boyutu 0 byte olan bir dosya oluşur; bu durum ls -l çıktısında açıkça görülür. touch, genellikle hızlıca dosya oluşturmak, test yapmak veya bir dosyanın son değiştirilme zamanını güncellemek için kullanılır. Ayrıca süslü parantez ({}) kullanımıyla touch file{alpha,beta,gamma}.txt gibi bir komutla aynı anda birden fazla dosya oluşturulabilir; shell bu ifadeyi genişleterek her dosyayı tek tek yaratır.

```
┌──(root㉿kali)-[~/Desktop]
└─# touch rolvarn.txt

┌──(root㉿kali)-[~/Desktop]
└─# ls
rolvarn.txt  test.txt
```

Dizin Oluşturma (mkdir)

mkdir (make directory) komutu, yeni dizinler oluşturmak için kullanılır. Tek başına kullanıldığında yalnızca belirtilen dizini oluşturur; ancak dizin yolu iç içe ise ve üst dizinler yoksa hata verir. -p (parents) parametresi bu noktada devreye girer ve yol üzerindeki eksik tüm dizinleri sırayla oluşturur. Örneğin mkdir -p projects/python/app komutu, projects ve python dizinleri mevcut olmasa bile hata vermeden tüm yolu

oluşturur. Bu özellik, derin dizin yapıları kurarken manuel olarak tek tek dizin açma ihtiyacını ortadan kaldırır.

Kopyalama (cp) – Genel Mantık

cp komutu, dosya ve dizinlerin bir kopyasını oluşturur. Varsayılan olarak yalnızca dosyalar kopyalanabilir; bir dizini kopyalamak istendiğinde bunun açıkça belirtilmesi gerekir. cp mevcut dosyaların üzerine yazabilir, bu yüzden parametreler kullanılmadan yapılan kopyalamalar veri kaybına yol açabilir.

Dizin Kopyalama (-r)

-r (recursive) parametresi, bir dizinin içindeki tüm dosya ve alt dizinlerle birlikte kopyalanmasını sağlar. Örneğin cp -r projects backup_projects komutu, projects dizinini tüm içeriğiyle birlikte backup_projects adlı yeni bir dizin olarak kopyalar. -r kullanılmazsa cp dizin kopyalamayı reddeder ve hata verir, çünkü dizinlerin tek başına değil, içerikleriyle birlikte ele alınması gerekir.

Etkileşimli Kopyalama (-i)

-i (interactive) parametresi, hedef konumda aynı isimde bir dosya varsa üzerine yazmadan önce kullanıcıdan onay ister. Örneğin cp -i notes.txt /tmp/ komutu çalıştırıldığında /tmp dizininde notes.txt zaten varsa shell “overwrite?” sorusunu sorar ve yalnızca kullanıcı y derse dosya değiştirilir. Bu seçenek, özellikle önemli dosyalarla çalışırken yanlışlıkla üzerine yazmayı engelleyen bir güvenlik katmanıdır.

Yedekli Kopyalama (--backup)

--backup parametresi, hedefte aynı isimde bir dosya bulunduğunda eski dosyayı silmek yerine yedekler. Eski dosyanın adına otomatik olarak ~ (tilde) eklenir; örneğin /tmp/notes.txt zaten varsa, bu dosya /tmp/notes.txt~ olarak yeniden adlandırılır ve yeni dosya normal adıyla kopyalanır. Bu sayede hem yeni dosya yazılır hem de önceki sürüm korunmuş olur. ~ işareti burada “eski sürüm” anlamına gelir ve birçok Linux aracında yedek dosyaları göstermek için kullanılır.

Örnek Kullanım : cp -ri --backup projects /tmp/

Taşıma ve Yeniden Adlandırma (mv)

Linux’ta ayrı bir “rename” komutu yoktur; taşıma ve yeniden adlandırma işlemlerinin ikisini de mv yapar. Bunun sebebi basittir: Linux’ta bir dosyanın adı da konumu da inode’a işaret eden bir etikettir. Bu etiketi değiştirdiğinde ya dosyanın adı değişir ya da bulunduğu dizin.

mv old.txt new.txt komutu, dosyanın içeriğine dokunmadan sadece adını değiştirir. Yani bu işlem bir “kopyala–sil” değildir; aynı dosya aynı inode ile kalır, sadece ismi güncellenir.

mv file.txt /tmp/ ise dosyayı bulunduğu dizinden alır ve /tmp dizinine taşır. Taşıma tamamlandığında dosya eski yerinde artık yoktur, çünkü inode artık yeni dizindeki isimle ilişkilendirilmiştir.

Bu yüzden mv, hızlıdır ve büyük dosyalarda bile neredeyse anında çalışır.

Linux’ta mv komutunun ne yaptığı, dosyanın aynı dosya sistemi içinde mi yoksa farklı bir dosya sistemine mi taşındığına bağlıdır. Eğer bir dosyanın sadece adı değiştiriliyorsa ya da aynı disk üzerindeki başka bir dizine taşınıyorsa, dosyanın inode’u değişmez. Bu durumda olan şey şudur: eski isim ya da eski dizindeki bağlantı silinir, yeni isim veya yeni dizin altında aynı inode’a işaret eden yeni bir bağlantı oluşturulur. Dosyanın içeriği kopyalanmaz, dosya yeniden yaratılmaz; yalnızca dosyaya giden yol (isim ve dizin bilgisi) değişmiş olur. Bu yüzden mv bu senaryolarda çok hızlı çalışır.

Ancak dosya, farklı bir dosya sistemine taşınıyorsa (örneğin ana diskten USB belleğe), mv artık sadece bir isim değişikliği yapamaz. Bu durumda dosya önce hedef dosya sistemine kopyalanır, orada yeni bir inode oluşturulur ve ardından eski dosya silinir. Yani işlem mantıksal olarak cp ve ardından rm yapılmış gibidir. Bu yüzden inode değişir ve özellikle büyük dosyalarda işlem süresi fark edilir şekilde uzar. Özetle, mv aynı dosya sistemi içinde çalışıyorsa inode korunur; farklı bir dosya sistemine geçiliyorsa yeni inode oluşturulur. Burada etiket mantığını iyi oturtmak lazım.

Silme İşlemleri (rm)

```
┌──(root㉿kali)-[~/Desktop]
└─# rm rolvarn.txt

┌──(root㉿kali)-[~/Desktop]
└─# ls
test.txt
```

Terminalde silme işlemi geri dönüşsüzdür. Grafik arayüzlerdeki çöp kutusu mantığı burada yoktur. rm dediğin anda dosya inode bağlantısını kaybeder ve çoğu durumda kurtarılamaz.

rm file.txt dosyayı doğrudan siler.

rm -i file.txt silmeden önce onay ister; bu seçenek, refleksle yanlış dosya silmeni engelleyen bir emniyet kemeridir. Terminalde alışkanlık haline getirilmesi gereken bir parametredir.

Dizin silmek için rm -r directory/ kullanılır. Bunun nedeni dizinlerin tek başına değil, içindeki tüm dosyalarla birlikte silinmesi gerekmesidir.

rm -rf directory/ ise en tehlikeli kombinasyondur: ne sorar, ne durur, ne hata affeder. Yanlış dizinde çalıştırırsan sistemini saniyeler içinde yok edebilirsin. Bu komut, “ne yaptığını %100 biliyorsan” içindir.

rmdir directory/ sadece boş dizinleri siler. İçinde tek bir dosya bile varsa çalışmaz. Bu yüzden en güvenli dizin silme yöntemidir.

Dosya Bilgisi ve Inode

file Komutu

file komutu, dosyanın gerçek türünü belirler. Uzantıya bakmaz; dosyanın içeriğini analiz eder. Örneğin .txt uzantılı ama aslında binary olan bir dosyayı yakalar.

Bu özellikle siber güvenlikte kritiktir: zararlı bir dosya kendini masum bir metin ya da resim gibi gösterebilir ama file bunu ele verir.

stat Komutu ve Inode

```
┌──(root㉿kali)-[~/Desktop]
└─# stat test.txt
  File: test.txt
  Size: 8               Blocks: 8          IO Block: 4096   regular file
Device: 254,3   Inode: 664217      Links: 1
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2025-12-22 11:14:08.710086565 -0900
Modify: 2025-12-22 11:14:05.373769368 -0900
Change: 2025-12-22 11:14:05.373769368 -0900
 Birth: 2025-12-22 11:13:54.848767394 -0900
```

stat komutu, dosyanın kimliğini açığa çıkarır. Burada gördüğün inode numarası, dosyanın dosya sistemindeki gerçek varlığıdır. Dosya adı sadece bu inode’a bağlanan bir etikettir.

Size, dosyanın boyutunu;

Links, bu inode’a kaç farklı isimle erişildiğini;

Access, Modify, Change zamanları ise dosyanın ne zaman okunduğunu, içeriğinin ne zaman değiştiğini ve metadata’sının ne zaman güncellendiğini gösterir.

Buradaki kritik nokta şudur:

Bir dosyanın adı silinse bile inode'a bağlı başka bir link varsa dosya hala vardır. Bu yüzden hard link, ad, inode kavramlarını birlikte düşünülür. Yani dosyanın tam silinmesi için bunların hepsinin silinmiş olması gerekir.

File Links (Hard Link & Soft Link)

Linux'ta bir dosyaya birden fazla isim verilebilir. Bu isimler, dosyanın disk üzerindeki aynı verisine erişim sağlar veya yalnızca o veriye giden yolu işaret eder. Bu fark, hard link ve soft link ayrımını oluşturur.

Hard link, bir dosyanın disk üzerindeki verisine ikinci (veya üçüncü) bir isim vermektir. Hard link oluşturulduğunda, her iki dosya adı da aynı veriyi gösterir. Bu yüzden dosyalardan biri silinse bile veri kaybolmaz; diğer isim üzerinden erişim devam eder. Ancak hard link'ler yalnızca aynı disk bölümü içinde çalışır ve dizinler için kullanılmaz. ls -li çıktısında inode numaralarının aynı olması, iki ismin aslında tek bir dosyaya ait olduğunu gösterir.

Soft link (symbolic link) ise dosyanın kendisini değil, dosyanın konumunu işaret eder. Yani bir adres tabelası gibidir. Soft link'in kendine ait ayrı bir kimliği vardır ve hedef dosyadan bağımsızdır. Hedef dosya silinir veya yeri değişirse soft link boşa düşer. Buna karşılık soft link'ler farklı diskler arasında çalışabilir ve dizinlere de verilebilir. ls -li çıktısında inode numaralarının farklı olması, soft link'in ayrı bir varlık olduğunu gösterir.

Gelişmiş Kopyalama (dd)

dd komutu, Linux'ta dosya sistemi seviyesinde değil, doğrudan ham veri seviyesinde çalışan bir kopyalama aracıdır. Bu nedenle normal kopyalama komutları gibi "dosya", "klasör" ya da "içerik" kavramlarını dikkate almaz; yalnızca verilen kaynaktan veriyi okur ve hedefe birebir yazar. Kaynak (if) ve hedef (of) olarak belirtilen şeyler bir dosya olabileceği gibi, bir disk ya da bölüm de olabilir. Örneğin bir ISO dosyasını USB belleğe yazarken dd, ISO içeriğini olduğu gibi USB'nin üzerine yazar ve USB'de daha önce bulunan tüm verileri tamamen siler. Bu yüzden dd disk imajı alma, USB'ye işletim sistemi yazma ve disk klonlama gibi işlemlerde kullanılır.

dd veriyi tek parça halinde değil, bloklar halinde kopyalar. Blok boyutu (bs), her seferinde kaç byte'lık veri okunup yazılacağını belirler. Bu değer dosyanın boyutunu veya içeriğini değiştirmez; sadece kopyalama işleminin nasıl yapılacağını etkiler. Küçük blok boyutları çok fazla okuma–yazma işlemi gerektirdiği için yavaştır, bu yüzden pratikte genellikle bs=4M gibi daha büyük değerler kullanılır. Böylece dd daha az işlem yaparak daha hızlı çalışır. Varsayılan blok boyutu küçük olduğu için bs parametresi verilmeden kullanılan dd genellikle gereksiz yere yavaş olur.

dd komutunun tehlikeli olmasının nedeni, hiçbir güvenlik mekanizması sunmamasıdır. Yanlış hedef belirtildiğinde uyarı vermez, onay istemez ve işlemi durdurmaz. Örneğin hedef olarak sistem diskini gösterirseniz, disk üzerindeki tüm veriler saniyeler içinde geri dönüşü olmayacak şekilde silinir. Bu yüzden işlem sırasında ilerlemeyi görmek için genellikle status=progress parametresi eklenir; bu parametre kopyalama hızını ve aktarılan veri miktarını gösterir. Özetle dd, ne yaptığını tam olarak bilenler için son derece güçlü, ama yanlış kullanıldığında ciddi veri kaybına yol açabilecek düşük seviyeli bir araçtır.

cp vs dd

cp ve dd her ikisi de kopyalama yapabilir gibi görünse de, tamamen farklı seviyelerde çalışan araçlardır. cp, dosya sistemi seviyesinde çalışır; dosyanın ne olduğunu bilir, izinlerini, sahipliğini ve zaman bilgilerini mantıklı şekilde kopyalar ve gerekirse kullanıcıyı uyarır. Bu yüzden günlük dosya kopyalama işlemleri için güvenlidir ve hata yapma ihtimali düşüktür. dd ise dosya sistemiyle ilgilenmez, yalnızca ham veriyi okur ve yazar. Dosya mı disk mi olduğunu umursamaz; verdiğin kaynağı byte byte hedefe aktarır ve yanlış hedef seçildiğinde hiçbir uyarı vermeden veriyi siler. Bu nedenle cp insan odaklı ve güvenli bir kopyalama aracıyken, dd düşük seviyeli, güçlü ama tehlikeli bir araçtır ve disk imajları, ISO yazma ve klonlama gibi özel durumlar için kullanılır.

Linux’ta Dosya ve Dizin Arama

Linux’ta dosya ve dizin arama işlemleri, büyük ve karmaşık dosya sistemleri içinde hızlı ve kontrollü şekilde gezinmeyi sağlar. Bu amaçla kullanılan find, locate ve which komutları, farklı ihtiyaçlara göre farklı çözümler sunar. find gerçek zamanlı ve detaylı arama yaparken, locate önceden oluşturulmuş bir veritabanı üzerinden çok hızlı sonuç verir; which ise bir komutun sistemde hangi dosyadan çalıştırıldığını gösterir. Bu üç komut birlikte kullanıldığında, hem dosyaların yerini bulmak hem de sistemin nasıl çalıştığını anlamak mümkün olur.

Find

Name (İsme Göre Arama / -name)

-name kriteri, dosya veya dizinleri tam ismine göre aramak için kullanılır ve büyük–küçük harf duyarlıdır. Yani notes.txt ile Notes.txt Linux açısından farklı isimlerdir. Bu kriter genellikle dosya adını tam olarak bildiğinizde veya belirli bir uzantıyı ararken kullanılır. Örneğin find / -name "notes.txt" komutu, kök dizinden başlayarak adı birebir notes.txt olan tüm dosyaları arar. Joker karakter (*) ile birlikte kullanıldığında ise belirli bir desenle eşleşen dosyalar bulunabilir; find . -name ".txt" komutu, bulunduğunuz dizinden başlayarak tüm alt dizinlerde .txt uzantılı dosyaları listeler.

Name (Case Insensitive / -iname)

-iname kriteri, -name ile aynı işi yapar ancak büyük–küçük harf ayrımını tamamen ortadan kaldırır. Dosya adının tam olarak nasıl yazıldığından emin olmadığınız durumlarda tercih edilir. Örneğin find . -iname "PiC.Jpg" komutu, pic.jpg, PIC.JPG veya Pic.Jpg gibi tüm varyasyonları bulur. Özellikle farklı işletim sistemlerinden kopyalanmış dosyalarla çalışırken bu kriter büyük kolaylık sağlar.

Type (Türe Göre Arama / -type)

-type kriteri, aramayı yalnızca belirli türdeki nesnelerle sınırlar. En yaygın kullanılan türler d (directory – dizin) ve f (file – normal dosya) şeklindedir. Bu filtre kullanılmadığında, find aynı isme sahip dosya, dizin veya linkleri birlikte getirebilir. Örneğin find /home -type d komutu, /home dizini altında bulunan tüm dizinleri listeler. Bu kriter özellikle dizinler üzerinde işlem yapılacağı zaman hatalı sonuçları önlemek için kritik öneme sahiptir.

Size (Boyuta Göre Arama / -size)

-size kriteri, dosyaları boyutlarına göre filtrelemek için kullanılır. + işareti belirtilen değerden büyük, - işareti ise küçük dosyaları ifade eder. Boyut birimleri K (KB), M (MB) ve G (GB) olarak belirtilir. Örneğin find / -size +100M komutu, sistem genelinde 100 MB'tan büyük dosyaları arar. Bu kriter en sık, disk doluluk sorunlarının kaynağını bulmak amacıyla kullanılır.

User (Sahibine Göre Arama – -user)

-user kriteri, dosyaları sahip olan kullanıcıya göre arar. Linux'ta her dosya bir kullanıcıya aittir ve bu bilgi dosyanın metadata'sında tutulur. Örneğin find /var -user www-data komutu, /var dizini altında www-data kullanıcısına ait tüm dosyaları listeler. Bu arama, servislerin oluşturduğu dosyaları tespit etmek, yetki problemlerini analiz etmek veya güvenlik denetimleri yapmak için sıklıkla kullanılır.

/dev/null (Hata ve Çıktı Yönlendirmede Kullanımı)

/dev/null, Linux sistemlerinde bulunan özel bir aygıt dosyasıdır ve işlevi oldukça nettir: kendisine gönderilen her şeyi sessizce yok eder. Bu nedenle genellikle “kara delik” olarak adlandırılır. Bir komut çalıştırıldığında üretilen çıktılar normalde ekrana yazdırılır; ancak bazen bu çıktılar, özellikle hata mesajları, gereksiz gürültü oluşturur. İşte bu noktada /dev/null devreye girer.

Linux’ta standart hata çıktısı 2 numaralı kanal (stderr) üzerinden gönderilir. find komutu geniş dizinlerde arama yaparken, yetkinizin olmadığı dizinlerde Permission denied gibi hatalar üretir ve bu mesajlar aramanın sonucunu değil, yalnızca erişim problemlerini ifade eder. 2>/dev/null kullanımı, bu hata mesajlarını alıp /dev/nulla yönlendirir; yani ekrana basılmalarını engeller. Örneğin find / -name "*.log" 2>/dev/null komutu, tüm sistemi tarar ancak yetki hatalarını göstermez, yalnızca gerçekten bulunan dosyaları listeler.

Önemli bir ayrım şudur: 2>/dev/null yalnızca hata çıktısını gizler. Eğer >/dev/null kullanılırsa, bu sefer normal çıktı (stdout) da çöpe atılır ve ekranda hiçbir sonuç görünmez. Bu yüzden /dev/null kullanımı bilinçli yapılmalıdır; amacı hata gürültüsünü susturmak, gerçek sonuçları değil.

```
┌──(root㉿kali)-[~]
└─# find -name "test.txt" 2>/dev/null
./Desktop/test.txt
```

Standart Girdi, Çıktı ve Hata Kanalları (stdin, stdout, stderr)

Linux’ta çalışan her komut, sistemle iletişimini üç temel kanal üzerinden kurar ve bu kanallar numaralarla temsil edilir. 0 numaralı kanal standart girdi (stdin) olarak adlandırılır ve komutun dışarıdan aldığı veriyi ifade eder; klavyeden yazılan bir değer ya da başka bir komuttan gelen çıktı bu kanala girer. 1 numaralı kanal standart çıktı (stdout)tır ve komutun normal, başarılı sonuçlarını ekrana yazdırdığı kanaldır. 2 numaralı kanal ise standart hata (stderr) kanalıdır ve hatalar, uyarılar veya beklenmeyen durumlar bu kanaldan gönderilir.

Bu ayrımın yapılmasının nedeni, Linux’un başarılı çıktılar ile hata mesajlarını bilinçli olarak ayırmasıdır. Örneğin find komutu dosyaları bulduğunda sonuçları stdout (1) üzerinden verirken, yetkinizin olmadığı dizinlere girdiğinde oluşan Permission denied mesajlarını stderr (2) üzerinden üretir. Bu sayede kullanıcı isterse yalnızca hataları gizleyebilir (2>/dev/null), isterse yalnızca başarılı çıktıları yönlendirebilir (1>dosya.txt). Kanal numaraları sayesinde Linux’ta çıktılar ayrı ayrı kontrol edilebilir ve bu mekanizma pipe (|), yönlendirme (>, >>) ve hata yönetiminin temelini oluşturur.

Logical Operators (AND, OR, NOT)

find komutunda mantıksal operatörler, birden fazla arama kriterini aynı komut içinde kontrollü şekilde birleştirmeyi sağlar. Varsayılan davranış AND mantığıdır; yani birden fazla kriter yazıldığında, bulunan dosya bu şartların tamamını sağlamak zorundadır. Örneğin find . -name ".txt" -size +1M komutu, yalnızca .txt uzantılı ve 1 MB’tan büyük dosyaları listeler; şartlardan biri sağlanmıyorsa dosya sonuçlara girmez. OR (-o)

operatörü, kriterlerden herhangi birinin sağlanmasını yeterli görür; find . -name ".jpg" -o -name ".png" komutu, JPG ya da PNG uzantılı tüm dosyaları bulur. Bu tür aramalarda, daha karmaşık filtreler için parantezler (( )) kullanılabilir. NOT (-not) operatörü ise belirli bir özelliğe sahip dosyaları hariç tutar; örneğin find . -not -user root -type f komutu, root kullanıcısına ait olmayan normal dosyaları listeler. Bu operatörler birlikte kullanılarak çok daha hassas aramalar yapılabilir; örneğin .log uzantılı ve root'a ait olmayan dosyaları bulmak için find . -name ".log" -not -user root şeklinde bir arama yapılabilir.

Bulunan Dosyalar Üzerinde Komut Çalıştırmak (-exec ve xargs)

find komutunun en güçlü özelliklerinden biri, bulduğu dosyalar üzerinde doğrudan işlem yapabilmesidir. -exec parametresi, findin bulduğu her dosya için belirtilen komutu çalıştırmasını sağlar. Yani find yalnızca sonuç üretmez, aynı zamanda bu sonuçları başka bir komuta tek tek aktarır. Bu sayede silme, taşıma, yetki değiştirme gibi işlemler otomatik hale gelir.

-exec kullanımında görülen {} ifadesi, find tarafından bulunan dosyanın kendisini temsil eden bir yer tutucudur (placeholder). find her eşleşme bulduğunda {} yerine o dosyanın tam yolunu koyar ve komutu o dosya için çalıştırır. Komutun sonunda yer alan \; ise -exec ifadesinin burada bittiğini finde söyleyen işarettir. Noktalı virgül (;) normalde shell tarafından yorumlandığı için, finde aynen iletilebilmesi adına ters eğik çizgi (\) ile kaçırılır. Yani \;, teknik olarak "bu komut tamamlandı" anlamına gelir.

xargs ise farklı bir yaklaşım kullanır. findin ürettiği çıktılar, pipe (|) operatörü ile xargsa aktarılır. Pipe, bir komutun çıktısını alıp başka bir komutun girdisi haline getirir; yani find yalnızca dosya listesini üretir, xargs ise bu listeyi alıp başka bir komuta argüman olarak verir. find . -name "*.log" | xargs rm örneğinde, find .log dosyalarının listesini üretir, xargs bu listeyi alır ve rm komutunu toplu şekilde çalıştırır. Bu yöntem, her dosya için ayrı komut çalıştıran -exece göre çok daha performanslıdır.

Ancak burada önemli bir risk vardır: xargs, varsayılan olarak boşluk ve satır sonlarını ayırıcı kabul eder. Eğer dosya adında boşluk, sekme veya özel karakter varsa, xargs bu dosya adını parçalara ayırabilir ve hatalı işlemler yapılmasına neden olabilir. İşte bu problemi çözmek için -print0 ve -0 seçenekleri kullanılır. finddeki -print0, bulunan dosya adlarını satır sonu yerine NULL karakteri ile ayırarak çıktılar. xargs -0 ise bu NULL karakterleri doğru şekilde okuyarak dosya adlarını asla bölmeden işler.

Güvenli kullanım şu şekildedir:

find . -name "*.log" -print0 | xargs -0 rm

Bu yapı, hem xargsın hız avantajını korur hem de boşluk içeren dosya adlarında bile güvenli çalışır. Özetle, -exec daha kontrollü ve risksiz, xargs ise doğru seçeneklerle kullanıldığında çok hızlı ve ölçeklenebilir bir çözümdür.

locate Komutu ile Arama

locate komutu, dosyaları disk üzerinde tek tek gezerek değil, sistemde önceden oluşturulmuş bir indeks (veritabanı) üzerinden arar. Bu yüzden find'a kıyasla inanılmaz hızlıdır, ama bu hızın bir bedeli vardır. locate, gerçek zamanlı çalışmaz; aramayı updatedb adlı bir veritabanı dosyası üzerinde yapar. Bu veritabanı, sistemdeki dosya ve dizin yollarının bir "fotoğrafı" gibidir. locate notes.txt dediğinde aslında şunu yaparsın: "Bu isim, en son alınmış sistem fotoğrafında var mı?" Eğer varsa, saniyenin altında sonuç döner. Hızının sebebi budur.

Buradaki kritik nokta şudur: locate, yeni oluşturulmuş dosyaları her zaman bulamaz. Çünkü veritabanı sürekli güncellenmez; genelde sistem tarafından belli aralıklarla (örneğin günde bir kez) güncellenir. Sen az önce notes.txt oluşturduysan ve veritabanı henüz güncellenmediyse, dosya gerçekten var olsa bile locate hiçbir şey bulamayabilir. Bu, komutun bozuk olduğu anlamına gelmez; verinin eski olduğu anlamına gelir. Bu durumda çözüm nettir: sudo updatedb komutunu çalıştırırsın ve veritabanını manuel olarak güncellersin. Bundan sonra locate aynı dosyayı bulacaktır.

which Komutu ile Arama

which komutu, bir komutu yazdığında sistemin gerçekte hangi dosyayı çalıştırdığını gösterir. Linux'ta bir komut yazdığında sihirli bir şey olmaz; kabuk (shell), PATH değişkeni içinde tanımlı dizinleri soldan sağa sırayla dolaşır ve ilk bulduğu çalıştırılabilir dosyayı kullanır. which python dediğinde aslında şu soruyu sorarsın: "Ben python yazınca, PATH içinde ilk olarak hangi python dosyası bulunuyor ve çalıştırılıyor?" Dönen /usr/bin/python çıktısı, aktif olarak kullanılan Python yorumlayıcısının tam yoludur.

Bu komut özellikle birden fazla sürüm yüklü olduğunda kritik hale gelir. Örneğin sistemde /usr/bin/python, /usr/local/bin/python veya bir virtualenv altında başka bir python olabilir. Sen python yazdığını sanırsın ama aslında bambaşka bir sürüm çalışıyordur. which, bu kafa karışıklığını bitirir ve "gerçek fail bu" diye net şekilde gösterir. Bu yüzden which, "komut var mı?" sorusundan çok, "hangi komut çalışıyor?" sorusunun cevabıdır.

Dosyaların İçinde Arama: grep

grep komutu, dosya adlarını değil dosyaların içeriğini arar. Yani "bu dosya var mı?" sorusuna değil, "bu dosyanın içinde şu metin geçiyor mu?" sorusuna cevap verir. Bu fark kritik bir farktır çünkü find dosya isimleriyle, grep ise dosyanın içindeki satırlarla ilgilenir. Özellikle log dosyaları, konfigürasyon dosyaları ve büyük metin dosyalarında belirli hataları veya anahtar kelimeleri bulmak için kullanılır.

grep "Error" server.log komutunu çalıştırdığında, server.log dosyası satır satır okunur ve içinde "Error" kelimesi geçen yalnızca o satırlar ekrana yazdırılır. Dosyanın tamamı gösterilmez, eşleşmeyen satırlar

tamamen yok sayılır. Bu yüzden grep, büyük dosyalarda çalışırken hem hızlıdır hem de gereksiz bilgi kirliliği oluşturmaz. Çıktı her zaman bağlamlıdır; yani hatanın geçtiği satırı aynen görürsün.

Regex (Regular Expression) Mantığı

grep'in gerçek gücü regex, yani düzenli ifadelerden gelir. Regex sayesinde "bu kelime geçiyor mu?" demek yerine "şu kurala uyan satırları bul" dersin. Örneğin ^ karakteri satırın başını temsil eder. grep "^Error" dersen, yalnızca satırın en başında "Error" ile başlayan satırlar eşleşir; satırın ortasında geçenler dikkate alınmaz. $ karakteri ise satırın sonunu ifade eder. grep "ok$" komutu sadece "ok" ile biten satırları bulur.

. karakteri herhangi tek bir karakteri, * ise kendisinden önce gelen karakterin sıfır veya daha fazla tekrarını temsil eder. Bu yüzden c.t ifadesi cat, cut, cot gibi kelimeleri eşleştirirken; ab* ifadesi a, ab, abb, abbb gibi varyasyonları kapsar. Bu yapı sayesinde grep, basit bir arama aracı olmaktan çıkar ve desen tanıma aracına dönüşür.

Dizin İçinde Arama (-r)

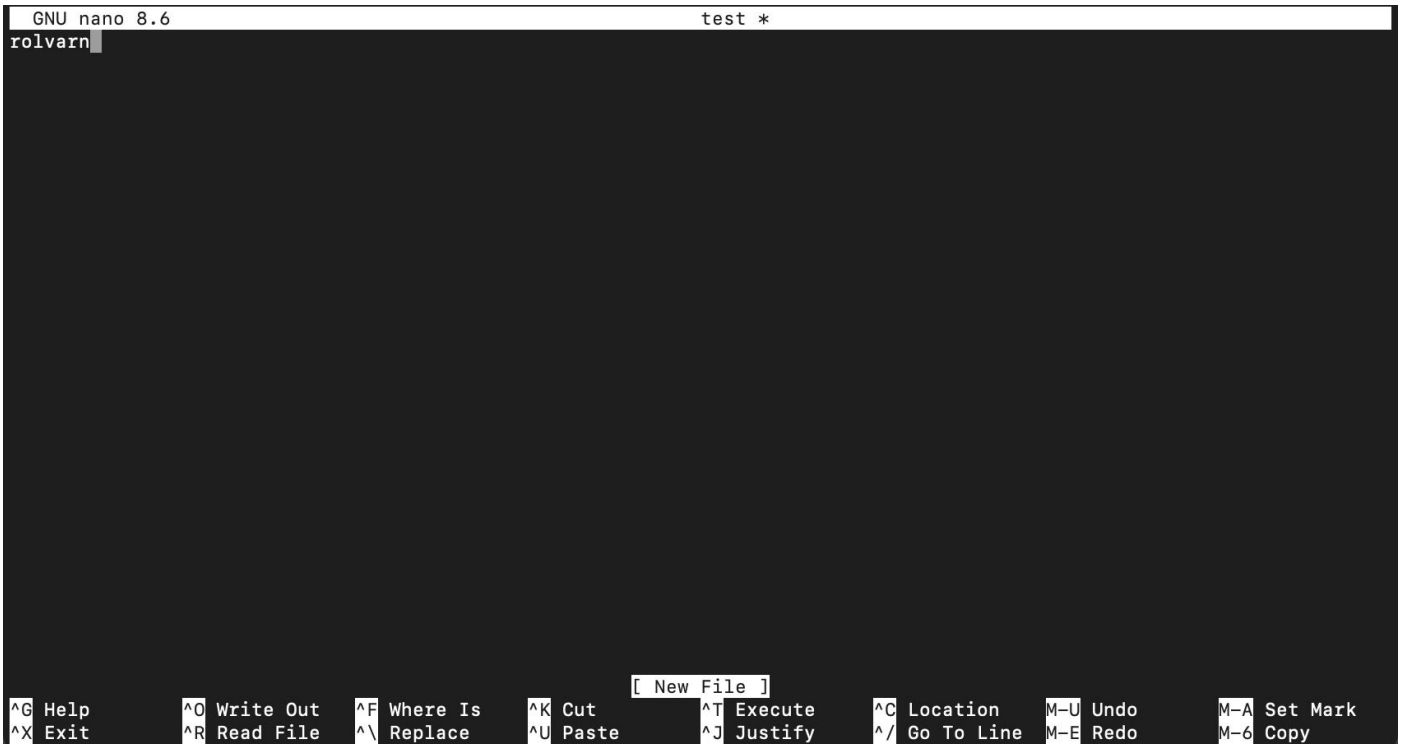
grep -r -i "error" /var/log/ komutu, tek bir dosyayla sınırlı kalmaz; verilen dizinin altındaki tüm dosyaları tarar. -r parametresi recursive yani "aşağı doğru in" anlamına gelir. -i ise büyük–küçük harf duyarlılığını kapatır. Böylece Error, ERROR ve error ifadelerinin tamamı eşleşir. Bu kullanım özellikle sistem loglarında hata ayıklarken standarttır.

Bu noktada şunu net söylemek gerekir: grep -r, mantık olarak find ile dosyaları bulup her birinin içinde arama yapmakla aynıdır; sadece bunu tek komutta ve daha pratik şekilde yapar. Büyük dizinlerde çalıştırıldığında yavaşlaması veya yetki hataları vermesi normaldir.

Metin Düzenleme (Text Editing)

Linux, metin dosyalarını oluşturmak, düzenlemek ve incelemek için komut satırı üzerinden erişilebilen güçlü araçlar sunar. Bu araçlar, basit not dosyalarından kritik sistem yapılandırmalarına kadar her seviyede kullanılır. Bu bölümde iki temel yaklaşımı temsil eden editörlere odaklanılır: yeni başlayanlar ve hızlı düzenlemeler için nano, daha ileri seviye kullanıcılar ve karmaşık işler için ise vim. Nano, öğrenmesi kolay ve sezgisel bir editördür; Vim ise yüksek verimlilik, hız ve tam kontrol sağlar.

Nano Metin Düzenleyici (Nano Text Editor)



Nano, Linux sistemlerde en yaygın kullanılan metin düzenleyicilerden biridir çünkü basit, hafif ve öğrenme eşiği neredeyse yoktur. Çoğu Linux dağıtımında varsayılan olarak yüklü gelir ve nano'yu kullanmak için önceden herhangi bir editör bilgisine sahip olman gerekmez. Nano'nun en önemli farkı şudur: Vim gibi editörlerde olduğu gibi "komut modları" yoktur. Dosya üzerinde yaptığın tüm işlemler (kaydetme, çıkma, arama, kesme vb.) ekranın en altında sana açık açık gösterilir. Yani ne yapacağını tahmin etmen gerekmez, görürsün.

Nano'yu bir dosyayı düzenlemek için çalıştırdığında, örneğin nano note.txt komutunu verdiğinde, eğer dosya varsa açılır, yoksa boş bir dosya oluşturulur ve doğrudan yazmaya başlayabilirsin. İmleci ok tuşlarıyla hareket ettirir, normal şekilde metin girersin. Nano'da yazı yazmak için özel bir moda geçmezsin; açıldığı anda düzenleme modundasındır. Bu sadelik, özellikle sistem dosyalarında hızlı bir değişiklik yaparken nano'yu çok cazip kılar.

Nano'nun alt kısmında gördüğün ^ işareti CTRL tuşunu temsil eder. Yani ekranda ^O Write Out yazıyorsa, bu "CTRL + O tuşlarına bas, dosyayı kaydet" demektir. CTRL + O ile dosyayı kaydettiğinde nano senden dosya adını onaylamanı ister; Enter'a bastığında kayıt tamamlanır. CTRL + X ise editörden çıkmak içindir. Eğer kaydedilmemiş değişiklikler varsa nano seni uyarır, yani sessizce veriyi kaybetmez.

Dosya içinde arama yapmak için CTRL + W kullanılır. Bu komuta bastığında editörün alt kısmında bir arama satırı açılır ve aramak istediğin kelimeyi yazarsın. Enter'a bastığında imleç, metin içinde bulunan ilk eşleşmeye gider. Bu, özellikle uzun konfigürasyon dosyalarında belirli bir ayarı hızlıca bulmak için çok kullanılır. Satır kesmek için CTRL + K, kestiğin satırı geri yapıştırmak için ise CTRL + U kullanılır. Nano'da kesme işlemi satır bazlıdır; yani CTRL + K bastığında imlecin bulunduğu satır tamamen kesilir.

Değişikliklerini yaptıktan sonra tipik akış şudur: Önce CTRL + O ile dosyayı kaydedersin, Enter ile onaylarsın, ardından CTRL + X ile çıkarsın. Editörden çıktıktan sonra, yaptığın değişikliklerin gerçekten dosyaya

yazıldığını görmek için cat note.txt gibi bir komutla dosya içeriğini ekrana bastırırsın. Nano'nun mantığı budur: hızlı aç, hızlı düzenle, güvenli şekilde kaydet ve çık. Bu yüzden nano, Linux'ta "ilk öğrenilen ama yıllarca bırakılmayan" editörlerden biridir.

Vim (Vi IMproved)

Vim (ve onun atası olan vi), Linux dünyasındaki en güçlü ve en hızlı metin editörlerinden biridir, fakat bunu bir bedelle sunar: öğrenme eğrisi diktir. Vim ilk açıldığında kullanıcıyı zorlar çünkü alıştığın "yaz ve sil" mantığıyla çalışmaz. Ancak mantığını kavradığında, fareye hiç dokunmadan, elini klavyeden kaldırmadan metin düzenleyebildiğin için hızın dramatik şekilde artar. Bu yüzden Vim özellikle sistem yöneticileri, yazılımcılar ve siber güvenlik uzmanları tarafından tercih edilir. Büyük dosyalarda, uzak sunucularda (SSH üzerinden) ve kritik sistem dosyalarında Vim rakipsizdir.

Vim ile bir dosya açmak için vim script.sh gibi bir komut kullanılır. Dosya açıldığında Vim seni doğrudan Normal Mode ile karşılar. Bu nokta yeni başlayanların en çok kafasının karıştığı yerdir çünkü bu modda yazı yazamazsın. Normal Mode'un amacı yazmak değil, dosya üzerinde komutlar vermektir. İmleci hareket ettirmek, satır silmek, kopyalamak, arama yapmak gibi tüm işlemler bu modda yapılır. Vim'de ne olursa olsun ESC tuşuna basmak seni her zaman Normal Mode'a geri getirir; bu, Vim'in en temel refleksidir.

Metin yazmak için Normal Mode'dan Insert Mode'a geçmen gerekir. Bunu yapmak için i tuşuna basarsın. Insert Mode'a geçtiğinde ekranın sol alt köşesinde -- INSERT -- ibaresini görürsün; bu, artık normal bir editör gibi yazı yazabileceğin anlamına gelir. Yazını yazdıktan sonra tekrar komut verebilmek için ESC ile Normal Mode'a dönersin. Vim'in gücü tam olarak burada yatar: yazma ve komut verme net şekilde ayrılmıştır, bu da kazara bir şeyleri bozmadan çok hızlı çalışmanı sağlar.

Vim'de metin seçmek için Visual Mode kullanılır. Normal Mode'dayken v tuşuna bastığında seçim moduna geçersin ve imleci hareket ettirdikçe metin bloklarını seçersin. Bu özellikle birden fazla satırı silmek, kopyalamak ya da başka bir yere taşımak istediğinde kullanılır. Yani Visual Mode, "işaretle ve işlem yap" mantığının Vim'deki karşılığıdır.

Temel senaryolarla düşünürsek: Bir dosyada yazı yazmak için önce Vim'i açarsın, i ile Insert Mode'a geçersin, metni yazarsın ve ardından ESC ile Normal Mode'a dönersin. Bir satırı silmek istediğinde imleci o satıra getirip dd yazman yeterlidir; bu komut satırı tamamen keser. Bir satırı kopyalamak için yy, yapıştırmak için p kullanılır. Yapılan bir hatayı geri almak için u tuşu kullanılır; bu Vim'deki "undo" işlemidir.

Dosyayı kaydetmek ve çıkmak da yine Normal Mode'da yapılır. :w dosyayı kaydeder, :q editörden çıkar. En sık kullanılan kombinasyon :wq'dur; yani "kaydet ve çık". Eğer değişiklik yaptın ama kaydetmeden çıkmak istiyorsan :q! kullanılır; bu komut Vim'e "ne yaptıysam umurumda değil, çık" demektir. Arama yapmak için Normal Mode'da /kelime yazılır; Enter'a bastığında Vim ilk eşleşmeye gider, n tuşu ile bir sonraki eşleşmeye geçilir.

Özetle Vim, başta zorlar çünkü sezgisel değildir; mantıklıdır. Ne yaptığını bilmeni ister. Ama bu mantık oturunca, büyük dosyalar içinde saniyeler içinde gezebilir, sistem dosyalarını güvenle düzenleyebilir ve uzaktan bağlı olduğun makinelerde bile tam kontrol sağlanabilir.

Ayrıca view komutu vim'i sadece okunabilir modda açar ve bu sayede dosyayı kazara değiştirme gibi risklerden korur.

Filters ve Pipe (|) Mantığı

Linux'ta filter (filtre) olarak adlandırılan programlar, düz metnin standart girdi (stdin) üzerinden alınması, bu metin üzerinde belirli bir işlemin uygulanması ve sonucun standart çıktı (stdout) üzerinden geri verilmesi prensibiyle çalıştırılır. Bu tür programlar dosyanın kendisiyle değil, yalnızca kendilerine verilen metinle ilgilenir. grep, sort, uniq, cut, tr ve wc gibi komutlar Linux'ta filtre olarak kullanılan temel araçlardır. Tek başlarına çalıştırıldıklarında sınırlı işlemler yapılabilse de, asıl güçleri başka komutların çıktısını işleyebilmelerinden kaynaklanır.

Bu noktada pipe (|) mekanizması devreye sokulur. Pipe, bir komut tarafından üretilen çıktının, herhangi bir dosyaya yazılmadan doğrudan başka bir komutun girdisi haline getirilmesini sağlar. Yani Linux'ta verinin önce ekrana yazılması, ardından tekrar okunması gibi verimsiz bir süreç kullanılmaz. Örneğin cat log.txt | grep "Error" komutu çalıştırıldığında, dosya içeriği cat tarafından okunur, ortaya çıkan metin pipe aracılığıyla grep komutuna aktarılır ve yalnızca "Error" kelimesini içeren satırlar süzülerek ekrana yazdırılır. Bu işlem sırasında ara dosya oluşturulmaz; veri akış halinde işlenir.

Bu yapı, bir üretim bandı mantığıyla açıklanabilir. Ham madde olarak dosya içindeki metin alınır. İlk makine (örneğin cat) bu metni okur ve üretir. Konveyör bant görevini üstlenen pipe (|), bu çıktıyı gecikme olmadan bir sonraki makineye taşır. İkinci makine (örneğin grep) gelen veriyi filtreler. Sonuçta elde edilen ürün, terminal ekranında görülen çıktıdır. İstenirse bu bant uzatılabilir; örneğin cat | grep | sort | uniq | wc şeklinde bir zincir kurularak veri adım adım işlenebilir.

Bu mekanizma sayesinde Linux'ta tek bir işi yapan küçük programlar, birlikte kullanıldıklarında son derece karmaşık işlemleri gerçekleştirebilir hale getirilir. Tek başına sınırlı görünen bir komut, pipe ile başka bir komuta bağlandığında güçlü bir analiz aracına dönüştürülür. Linux felsefesinin temelinde de bu yaklaşım yer alır: her araca tek bir sorumluluk verilmesi ve bu araçların zincirlenerek büyük işlerin yapılması.

Redirection (Yönlendirme)

Linux'ta çalışan her programın üç temel iletişim kanalı bulunduğu kabul edilir. Bunlar sırasıyla standart girdi (STDIN – 0), standart çıktı (STDOUT – 1) ve standart hata (STDERR – 2) olarak adlandırılır. Varsayılan

durumda standart girdi klavyeden alınır, standart çıktı ekrana yazdırılır ve hata mesajları da yine ekranda gösterilir. Ancak Linux'un en güçlü özelliklerinden biri, bu kanalların istenilen yere yönlendirilebilmesidir (redirection). Böylece bir programın ürettiği çıktı ekrana basılmak yerine bir dosyaya yazdırılabilir ya da bir dosya, klavye yerine girdi olarak kullanılabilir.

> operatörü, bir komutun standart çıktısının bir dosyaya yazılmasını sağlar. Bu işlem sırasında dosya zaten varsa içeriği tamamen silinir ve yeni çıktı yazılır. Örneğin ls > list.txt komutu çalıştırıldığında, dizin listesi ekranda gösterilmez; doğrudan list.txt dosyasına yazılır. >> operatörü ise benzer şekilde çalışır fakat mevcut içeriği silmez, çıktıyı dosyanın sonuna ekler. Bu nedenle günlük (log) tutma gibi senaryolarda sıkça tercih edilir. Örneğin echo "Last line" >> note.txt komutu, metni dosyanın en altına ekler.

2> operatörü, yalnızca hata çıktılarının (STDERR) yönlendirilmesini sağlar. Normal çıktı ekrana basılmaya devam ederken, hata mesajları belirtilen dosyaya yazılır. Örneğin ls /nonexistent_directory 2> error.txt komutunda, "No such file or directory" hatası ekranda görülmez; bu mesaj error.txt dosyasına kaydedilir. Bu ayrım, özellikle betiklerde hata kayıtlarını ayrı tutmak için kritik öneme sahiptir.

< operatörü ise yönlendirmeyi tersine çevirir; yani bir dosya, komutun girdisi (STDIN) olarak verilir. Örneğin wc -l < list.txt komutunda, wc komutu klavyeden veri beklemek yerine list.txt dosyasının içeriğini okur ve satır sayısını hesaplar. Bu yöntem, dosya içeriğinin bir programa manuel olarak yazılmasına gerek kalmadan işlenmesini sağlar.

Pipe (|) operatörü ise bir komutun standart çıktısının, doğrudan başka bir komutun standart girdisi haline getirilmesini sağlar. Örneğin cat file | grep "search" ifadesinde, dosya içeriği cat tarafından üretilir, pipe aracılığıyla grep komutuna aktarılır ve yalnızca aranan kelimeyi içeren satırlar ekrana yazdırılır. Bu işlem sırasında herhangi bir ara dosya oluşturulmaz; veri akış halinde işlenir.

Pratik bir senaryo olarak find komutu ele alındığında, yetkisiz dizinlerde sıkça "Permission denied" hata mesajlarının üretildiği görülür. Bu mesajlar çoğu zaman istenmeyen gürültü oluşturur. Daha temiz bir çıktı elde edebilmek için hata kanalı /dev/null dosyasına yönlendirilir. /dev/null, kendisine gönderilen her şeyi yok eden özel bir dosya olarak çalışır. Örneğin find / -name "secret_file" 2> /dev/null komutu çalıştırıldığında, hata mesajları tamamen yok sayılır ve ekranda yalnızca gerçekten bulunan dosya yolu gösterilir. Bu yöntem, özellikle sistem genelinde arama yapılırken çıktının okunabilirliğini ciddi şekilde artırır.

Text Processing Tools (Metin İşleme Araçları)

Linux'ta metin işleme araçları, düz metin dosyalarının hızlıca incelenmesi, analiz edilmesi ve diğer komutlarla birlikte işlenmesi amacıyla kullanılır. Bu araçlar genellikle dosya içeriğini standart çıktı (stdout – komutu yazınca aldığınız çıktı) üzerinden üretir ve pipe (|) ile başka komutlara aktarılabilecek şekilde çalışır.

cat (Concatenate)

```
┌──(root㉿kali)-[~]
└─# cat test
rolvarn
```

cat komutu, bir veya birden fazla metin dosyasının içeriğinin tamamının terminal ekranına yazdırılması amacıyla kullanılır. Komutun temel işlevi dosyayı okumak ve olduğu gibi çıktı vermektir; herhangi bir filtreleme, sıralama veya kısaltma işlemi yapmaz. Bu nedenle küçük dosyalar veya konfigürasyon dosyaları hızlıca incelenmek istendiğinde tercih edilir. Örneğin cat names.txt komutu çalıştırıldığında, dosya içindeki tüm satırlar sırayla ekrana basılır. Aynı şekilde cat /etc/ssh/sshd_config kullanıldığında, SSH servisinin yapılandırma dosyasının içeriği doğrudan görüntülenir. Ancak dosya çok büyükse cat kullanımı pratik değildir, çünkü tüm içerik kontrolsüz şekilde ekrana akar. Bu nedenle cat, daha çok hızlı kontrol ve pipe ile başka araçlara veri beslemek için kullanılır.

head

```
┌──(root㉿kali)-[~]
└─# cat test
rolvarn
cyber
security

┌──(root㉿kali)-[~]
└─# head -n 1 test
rolvarn
```

head komutu, bir dosyanın yalnızca başlangıç kısmının görüntülenmesini sağlar. Varsayılan olarak dosyanın ilk 10 satırı ekrana yazdırılır; ancak -n parametresi ile bu sayı istenildiği gibi değiştirilebilir. Örneğin head -n 3 access.log komutu, ilgili dosyanın yalnızca ilk 3 satırını gösterir. Bu özellik, özellikle büyük log dosyalarının yapısını hızlıca anlamak, dosyanın doğru formatta olup olmadığını kontrol etmek veya ilk kayıtları incelemek için kullanılır. head, dosyanın tamamını okumaya gerek kalmadan hızlı bir ön izleme sunar ve bu yönüyle performans açısından da avantajlıdır.

tail

tail komutu, bir dosyanın son kısmında yer alan satırların görüntülenmesini sağlar. Varsayılan olarak dosyanın son 10 satırı ekrana yazdırılır; ancak -n parametresi kullanılarak bu sayı istenildiği gibi değiştirilebilir. Örneğin tail -n 3 /var/log/auth.log komutu, kimlik doğrulama (authentication) olaylarının kaydedildiği auth.log dosyasının en son eklenen 3 satırını gösterir. Bu özellik özellikle log dosyaları için kritiktir; çünkü log dosyaları zamanla büyür ve genellikle en değerli bilgi en sonda bulunur. tail komutunun en güçlü kullanım biçimlerinden biri -f (follow) parametresidir. tail -f ile dosya yalnızca okunmaz, aynı

zamanda dosyaya yeni satırlar eklendikçe bu satırlar anlık olarak ekrana yansıtılır. Bu yöntem “canlı log izleme” olarak adlandırılır ve sunucuya gelen istekleri, hataları veya sistem olaylarını gerçek zamanlı takip etmek için kullanılır. CTRL + C tuş kombinasyonu ile bu izleme sonlandırılır.

sort

sort komutu ise bir dosyanın veya standart girdinin (STDIN) satırlarını alfabetik veya sayısal sıraya göre düzenlemek için kullanılır. Varsayılan davranış alfabetik sıralamadır. Örneğin sort names.txt komutu, dosya içindeki isimleri A’dan Z’ye doğru sıralar. Bu komut yalnızca görsel düzenleme yapmaz; çıktıyı başka komutlara aktarmak için de kritik bir rol oynar. Büyük veri kümelerinde anlamlı karşılaştırma yapabilmek, tekrar eden satırları tespit edebilmek veya raporlanabilir çıktılar üretmek için verinin önce sıralanması gerekir. sort, bu nedenle çoğu zaman tek başına değil, başka filtrelerle birlikte kullanılır. Kalıcı değildir sadece terminal ekranında düzenler. Kalıcı olması için “sort dosya.txt -o dosya.txt” gibi bir yapı kullanılmalıdır.

```
┌──(root㉿kali)-[~]
└─# cat test
rolvarn
cyber
security

┌──(root㉿kali)-[~]
└─# sort test
cyber
rolvarn
security
```

uniq

uniq komutu, ardışık (consecutive) olarak tekrar eden satırları filtreleyerek tekilleştirir. Buradaki kritik nokta şudur: uniq komutu dosyanın tamamına bakarak tekrarları bulmaz, yalnızca yan yana duran aynı satırları dikkate alır. Bu yüzden tek başına kullanıldığında beklenen sonucu vermeyebilir. Örneğin aynı satır dosyada farklı yerlerde geçiyorsa, uniq bunları ayrı kabul eder. Bu sorunu çözmek için uniq neredeyse her zaman sort komutuyla birlikte kullanılır. sort names.txt | uniq ifadesi önce veriyi sıralar, ardından tekrar eden satırları temizler. uniq -c parametresi ise her satırın kaç kez tekrarlandığını sayar ve bu sayıyı satırın başına ekler. Örneğin sort names.txt | uniq -c çıktısı, hangi ismin kaç kez geçtiğini açıkça gösterir. Bu kombinasyon, log analizinde, kullanıcı listelerinde, veri temizleme işlemlerinde ve raporlama süreçlerinde temel bir yapı taşı olarak kullanılır.

grep

grep, dosyalar içinde belirli bir metni, deseni veya ifadeyi aramak ve yalnızca bu ifadeyi içeren satırları filtreleyerek ekrana yazdırmak için kullanılır; dosya adıyla değil, doğrudan dosya içeriğiyle ilgilenir. Bu nedenle özellikle log dosyaları, konfigürasyon dosyaları ve büyük metin çıktıları üzerinde vazgeçilmezdir.

Örneğin Apache erişim loglarında belirli bir IP adresini aramak için grep '192.168.1.1' access.log komutu çalıştırıldığında, yalnızca bu IP adresinin geçtiği satırlar ekrana yazdırılır. -i parametresi aramayı büyük–küçük harf duyarsız hâle getirir; yani “Error”, “error” ve “ERROR” ifadelerinin tamamı eşleşme olarak kabul edilir. -v parametresi ise mantığı tersine çevirir ve eşleşenleri değil, eşleşmeyen satırları gösterir; örneğin grep -v '127.0.0.1' access.log komutu, localhost kaynaklı tüm kayıtları hariç tutarak diğer tüm satırları listeler. -r parametresi recursive (özyinelemeli) arama yapar; yani yalnızca tek bir dosyada değil, belirtilen dizinin altındaki tüm dosyalarda arama yapılmasını sağlar. Örneğin grep -r 'PasswordAuthentication yes' /etc/ komutu, /etc dizini altındaki tüm konfigürasyon dosyalarını tek tek dolaşır ve bu ifadeyi içeren satırları bulur. Bu özellik, özellikle büyük dizin yapılarında hangi dosyada neyin tanımlı olduğunu hızlıca tespit etmek için kritik öneme sahiptir. Bu parametreler sayesinde grep, sadece bir arama aracı olmaktan çıkar ve güçlü bir içerik filtreleme mekanizmasına dönüşür.

Burada önemli bir nokta da şu: -r kullanıldığında grep yüzlerce hatta binlerce dosyayı tarıyormuş gibi görünür ama gerçekte yaptığı şey, dizin ağacını sistematik biçimde aşağı doğru gezerek her dosyanın içeriğini satır satır kontrol etmektir.

wc (Word Count)

```
┌──(root㉿kali)-[~]
└─# wc test
 3  3 23 test
```

wc (Word Count) komutu, bir dosyanın ya da bir komut çıktısının kaç satır, kaç kelime ve kaç karakter (ya da byte) içerdiğini hızlıca öğrenmek için kullanılır. Varsayılan olarak wc çalıştırıldığında üç temel ölçüm aynı anda üretilir ve bu ölçümler belirli bir sırayla ekrana yazdırılır: ilk sütun satır sayısını (line count), ikinci sütun kelime sayısını (word count), üçüncü sütun ise karakter veya byte sayısını (character count) gösterir; en sağda ise ölçümü yapılan dosyanın adı yer alır. Örneğin wc /etc/passwd çıktısında görülen 46 67 2544 /etc/passwd ifadesi, bu dosyanın 46 satırdan oluştuğunu, toplam 67 kelime içerdiğini ve dosya boyutunun 2544 karakter (byte) olduğunu anlatır. Bu sıralama ezber değildir; Linux’ta wc çıktısı her zaman satır–kelime–karakter mantığıyla okunur. İhtiyaca göre bu değerler tek tek de alınabilir: -l yalnızca satır sayısını, -w yalnızca kelime sayısını, -c ise karakter/byte sayısını verir. wc, özellikle pipe (|) ile birlikte kullanıldığında daha da anlamlı hâle gelir; örneğin grep -r "error" /var/log | wc -l komutu, kaç satırda “error” geçtiğini sayarak loglardaki hata yoğunluğunu nicel olarak ölçmeyi sağlar. Bu yönüyle wc, içeriği okumaktan çok çıktının büyüklüğünü ve etkisini ölçmek için kullanılan analitik bir araçtır.

cut

cut komutunu doğru anlamak için önce iki temel kavramın netleşmesi gerekir: ayraç (delimiter) ve sütun / alan (field). Linux’ta birçok metin dosyası, tek parça yazı gibi görünse de aslında her satırı belirli parçalara bölünmüş yapıdadır. Bu parçaları birbirinden ayıran karaktere ayraç denir; bu genellikle :, ,, ; veya boşluk

olabilir. Ayraçla ayrılmış her parça ise bir sütun (field) olarak adlandırılır. Örneğin /etc/passwd dosyasındaki bir satır root:x:0:0:root:/root:/bin/bash şeklindedir; burada : ayraçtır ve root birinci sütun, x ikinci sütun, 0 üçüncü sütun olacak şekilde satır parçalara ayrılır. İşte cut komutu, bu sütunlu yapıdan yalnızca ihtiyacın olan parçayı çekip almak için kullanılır.

Bu mantık üzerine kurulan cut komutu, satırları belirtilen ayraç üzerinden böler ve sadece seçilen sütunları ekrana yazdırır. -d parametresi hangi karakterin ayraç olduğunu söylerken, -f parametresi hangi sütunun alınacağını belirtir. Örneğin cut -d ":" -f 1 /etc/passwd komutu, her satırı : karakterine göre böler ve yalnızca birinci sütunu, yani kullanıcı adlarını listeler. Eğer -f 1,3 şeklinde bir kullanım yapılırsa birinci ve üçüncü sütunlar birlikte alınabilir; -f 1-4 ise ilk dört sütunu kapsar. Bu sayede büyük ve karmaşık görünen dosyalar, anlamlı ve sade çıktılara dönüştürülebilir. cut komutu hızlıdır, basittir ve yalnızca ayraçlı metinlerle çalışır; bu yüzden basit sütun çekme işlemlerinde awk gibi daha karmaşık araçlara gerek kalmadan etkili bir çözüm sunar

awk

awk'in sütunlara ayırma mantığı, metin işleme gücünün temelini oluşturur ve oldukça otomatik bir şekilde çalışır. awk bir dosyayı okuduğunda, içeriği satır satır ele alır ve her satırı belirli bir ayraç (delimiter) kullanarak parçalara böler. Varsayılan durumda bu ayraç boşluk ve tab karakterleridir; yani bir satırdaki kelimeler arasında bir veya daha fazla boşluk varsa, awk bunları doğal sınırlar olarak kabul eder. Ortaya çıkan her parça bir "field" yani sütun olarak adlandırılır. awk bu sütunları numaralandırır: birinci sütun $1, ikinci sütun $2, üçüncü sütun $3 şeklinde ilerler. Ayrıca $0 özel bir alandır ve satırın tamamını, hiçbir bölme yapılmadan temsil eder.

Örneğin içinde "Ali Veli 23" gibi boşluklarla ayrılmış veriler bulunan bir satır ele alındığında, awk bu satırı otomatik olarak üç sütuna ayırır: "Ali" birinci sütun ($1), "Veli" ikinci sütun ($2) ve "23" üçüncü sütun ($3) olur. Bu nedenle awk '{print $1}' komutu yalnızca isimleri, awk '{print $3}' komutu ise yalnızca yaş bilgilerini ekrana yazdırır. Aynı satırdan birden fazla sütun almak da mümkündür; awk '{print $1, $3}' gibi bir ifade kullanıldığında, her satırdan birinci ve üçüncü alanlar birlikte yazdırılır. Bu işlem sırasında awk, sütunları oluşturmak için ekstra bir tanımlamaya ihtiyaç duymaz; boşlukları görmesi yeterlidir.

Eğer satırdaki veriler boşlukla değil de başka bir karakterle ayrılmışsa, awk'ye bu ayırıcının ne olduğu açıkça söylenir. Örneğin /etc/passwd dosyasında alanlar ":" karakteri ile ayrılır. Bu durumda -F parametresi kullanılarak ayraç tanımlanır ve awk -F ':' '{print $1}' gibi bir komutla yalnızca ilk sütun, yani kullanıcı adları alınabilir. Sonuç olarak awk, satırları otomatik olarak sütunlara böler, bu sütunları numaralandırır ve kullanıcının istediği alanları çok az komutla ayıklamasını sağlar; bu da onu kolon bazlı veri işlemede son derece güçlü ve esnek bir araç hâline getirir.

sed

Sed (stream editor) komutu, metin akışı üzerinde satır satır çalışarak düzenleme, dönüştürme ve filtreleme işlemleri yapan güçlü bir araçtır. “Stream” ifadesi, sed’in dosyayı bir bütün olarak değil; satır satır okuyup anında işlediğini anlatır. Bu nedenle sed, büyük dosyalar üzerinde bile hızlı çalışır ve çoğunlukla pipe (|) ile başka komutların çıktısı üzerinde kullanılır. Sed’in en yaygın kullanım alanı metin değiştirme (substitution) işlemidir. Örneğin sed 's/Alice/George/' names.txt komutu, names.txt dosyasındaki her satırı okur ve satır içinde geçen ilk “Alice” ifadesini “George” ile değiştirerek sonucu ekrana yazdırır. Buradaki s harfi substitute (değiştir) anlamına gelir; ilk ifade aranan metni, ikinci ifade ise yerine konacak metni temsil eder.

Sed’in önemli bir özelliği, varsayılan olarak dosyanın kendisini değiştirmemesidir; yani yaptığı tüm düzenlemeler yalnızca çıktıda görülür. Bu davranış, sed’i güvenli kılar çünkü yanlış bir komut dosyayı bozamaz. Eğer değişikliğin kalıcı olması istenirse -i (in-place) parametresi kullanılır; örneğin sed -i 's/Alice/George/' names.txt komutu, dosyanın içeriğini gerçekten değiştirir. Sed yalnızca kelime değiştirmekle sınırlı değildir; satır silme (/pattern/d), belirli satırları seçme, satır ekleme veya düzenli ifadeler (regex) ile karmaşık metin dönüşümleri yapma gibi işlemleri de destekler.

tee

Tee komutu, standart çıktıyı (STDOUT) iki farklı hedefe aynı anda göndermek için kullanılır: ekran ve bir dosya. Normalde > veya >> yönlendirmeleri kullanıldığında çıktı yalnızca dosyaya yazılır ve ekranda görünmez; tee ise bu davranışı değiştirerek çıktının akışını bölmeden kopyalar. Bu yüzden tee, özellikle bir komutun ne ürettiğini anlık olarak izlerken aynı çıktıyı arşivlemek istendiğinde tercih edilir. Örneğin ls -l | tee files.txt komutu, dizin listesini terminalde gösterirken aynı bilgiyi files.txt dosyasına da kaydeder. -a parametresi kullanıldığında (tee -a log.txt), dosyanın içeriği silinmez ve yeni çıktı dosyanın sonuna eklenir. Bu sayede tee, hem izleme hem kayıt alma ihtiyacını tek adımda çözen bir “akış kopyalayıcı” gibi çalışır.

diff

Diff komutu, iki dosyayı satır satır karşılaştırarak aralarındaki farkları gösterir. Çıktıda < işareti birinci dosyada olup ikinci dosyada olmayan satırları, > işareti ise ikinci dosyada olup birincide olmayan satırları temsil eder. Bu yaklaşım, dosyaların hangi noktalarda ayrıştığını net bir şekilde görmeyi sağlar. Örneğin diff config_old.conf config_new.conf komutu çalıştırıldığında, yapılan ayar değişiklikleri doğrudan gözlemlenebilir. diff yalnızca “fark var mı” sorusuna cevap vermez, farkın tam olarak nerede ve nasıl olduğunu da gösterir. Bu nedenle sistem yapılandırmaları, yedek dosyalar veya kod dosyaları karşılaştırılırken vazgeçilmez bir araçtır ve sürüm kontrol sistemlerinin temel mantığını anlamak için de iyi bir zihinsel model sunar.

tr

tr komutu, metin üzerinde karakter bazlı dönüşüm yapar; satır, kelime veya sütun kavramlarıyla ilgilenmez. Kendisine verilen karakter kümesini başka bir karakter kümesiyle birebir eşleyerek dönüştürür. Bu yüzden genellikle pipe (|) ile birlikte kullanılır. Örneğin echo "Hello World" | tr "A-Z" "a-z" komutu, tüm büyük harfleri küçük harfe çevirerek hello world çıktısını üretir. Benzer şekilde tr -d ' ' kullanılarak boşluk karakterleri tamamen silinebilir veya belirli karakterler ayıklanabilir. tr, veriyi normalize etmek yani standart bir forma sokmak için kullanılır; özellikle log analizi veya ham veri temizleme aşamalarında basit ama çok etkili bir filtredir.

Package Management (Paket Yönetimi)

Linux işletim sistemlerinin günlük kullanımında ve sistem yönetiminde yazılımların kurulması, güncellenmesi ve kaldırılması temel bir ihtiyaç olarak ortaya çıkar. Bu işlemler, paket yönetim sistemleri aracılığıyla merkezi ve kontrollü bir şekilde gerçekleştirilir. Bir paket yöneticisi, yalnızca yazılımı kurmakla kalmaz; aynı zamanda o yazılımın çalışması için gerekli olan bağımlılıkları (dependencies) otomatik olarak tespit eder, indirir ve yönetir. Bu sayede kullanıcı, tek tek kütüphane aramak veya uyumsuzluklarla uğraşmak zorunda kalmaz ve sistem bütünlüğü korunmuş olur.

Her Linux dağıtımı, kendi paket formatını ve buna uygun paket yönetim araçlarını kullanır. Örneğin Debian tabanlı dağıtımlarda (.deb paketleri) farklı araçlar tercih edilirken, başka dağıtımlarda farklı sistemler kullanılabilir. Bu bölümde, özellikle Debian tabanlı Linux dağıtımlarında yaygın olarak kullanılan paket yönetim araçları ele alınacak ve yazılım kurma, güncelleme ve kaldırma gibi temel işlemlerin nasıl gerçekleştirildiği açıklanacaktır.

Paket ve Depo (Repository) Nedir?

Windows’ta bir program kurmak için genellikle internetten bir .exe dosyası indirilir ve “Next > Next” şeklinde ilerlenir; Linux’ta ise bu süreç daha güvenli, düzenli ve merkezi bir yapıyla yürütülür. Linux’ta paket, bir yazılımın çalışması için gerekli olan tüm bileşenleri (program kodları, kütüphaneler, yapılandırma dosyaları ve yardımcı dosyalar) tek bir sıkıştırılmış dosya içinde barındıran bir formattır; Debian tabanlı sistemlerde bu dosyalar genellikle .deb uzantılıdır. Bu paketler rastgele sitelerden indirilmez; bunun yerine repository (depo) adı verilen, dağıtıma ait resmi ve güvenilir sunucularda saklanır. Repository’ler, binlerce yazılımın düzenli biçimde tutulduğu ve sürekli güncellendiği merkezi yazılım havuzları olarak düşünülebilir; mantık olarak App Store veya Google Play’e benzer.

Bu yapının merkezinde ise paket yöneticisi bulunur. Paket yöneticisi (örneğin apt), repository’lere bağlanarak istenen paketi bulur, indirir, sisteme kurar ve gerektiğinde günceller. En kritik nokta, bir yazılımın ihtiyaç duyduğu ek kütüphanelerin (bağımlılıkların) da otomatik olarak çözülmesidir; yani kullanıcı “şu da eksik, bu da eksik” diye manuel işlem yapmak zorunda kalmaz. Tüm yazılımlar resmi depolardan indirildiği için zararlı yazılım riski ciddi şekilde azalır ve sistem genelinde tutarlılık sağlanmış olur.

Paket Yöneticilerinin Temel Özellikleri

Linux paket yöneticileri, yazılım yönetimini merkezi, tutarlı ve güvenli hale getiren bir dizi kritik özelliğe sahiptir. Öncelikle paket indirme işlevi sayesinde, yazılımlar ve onların ihtiyaç duyduğu ek bileşenler (bağımlılıklar) kullanıcı adına otomatik olarak internet üzerindeki resmi repository'lerden indirilir; bu süreçte manuel dosya arama veya uyumluluk kontrolü yapılmasına gerek kalmaz. Bununla doğrudan bağlantılı olan bağımlılık çözümleme (dependency resolution) özelliği, kurulmak istenen bir paketin çalışabilmesi için hangi kütüphanelere ve yardımcı paketlere ihtiyaç duyduğunu analiz eder ve eksik olanları otomatik olarak sisteme ekler.

Paket yöneticileri aynı zamanda kurulum ve kaldırma işlemlerini kontrollü bir şekilde gerçekleştirir; bir yazılım kurulduğunda dosyalar sistemin doğru dizinlerine yerleştirilir, kaldırıldığında ise geride gereksiz dosya ve yapılandırmalar bırakmadan temiz bir şekilde silinir. Güncelleme ve yükseltme (update/upgrade) özelliği sayesinde sistemde kurulu tüm yazılımlar tek bir merkezden kontrol edilir; yeni sürümler çıktığında güvenlik yamaları ve iyileştirmeler kolayca uygulanabilir. Son olarak, paket yöneticileri yapılandırma dosyaları ve dizin standartlarını koruyarak her yazılımın belirli kurallara göre konumlanmasını sağlar; bu da sistem yöneticilerinin dosya yapısını tahmin edilebilir, düzenli ve yönetilebilir bulmasına olanak tanır.

APT (.deb – Debian Package)

APT, Debian tabanlı Linux dağıtımlarında kullanılan paket yönetim sistemidir. Debian, Ubuntu ve Kali Linux gibi sistemlerde yazılım kurma, güncelleme ve kaldırma işlemleri bu yapı üzerinden gerçekleştirilir. APT tarafından kullanılan .deb uzantılı dosyalar, bir yazılımın çalışması için gerekli tüm bileşenleri içeren paketlerdir; bu paketlerin içinde program dosyaları, ihtiyaç duyulan kütüphaneler ve yapılandırma (konfigürasyon) dosyaları yer alır. Kullanıcı, tek bir komut vererek bu paketin sisteme doğru şekilde yerleştirilmesini sağlar.

APT, yüksek seviyeli (high-level) bir paket yöneticisi olarak çalışır. Yüksek seviyeli ifadesiyle kastedilen, yalnızca paketin kurulmasının değil; paketin çalışabilmesi için gerekli olan tüm bağımlılıkların (dependencies) de otomatik olarak tespit edilip indirilmesi ve kurulmasıdır. Yani kullanıcıdan "önce şu paketi, sonra bunu kur" gibi manuel bir işlem beklenmez. Bu süreç, sistemde tanımlı olan depolar (repository) üzerinden yürütülür; depolar, yazılımların güvenli ve doğrulanmış şekilde saklandığı resmi sunuculardır.

Bu işlemlerin arka planında dpkg aracı kullanılır. dpkg, Debian paket sisteminin düşük seviyeli (low-level) bileşenidir ve görevi yalnızca .deb paketinin içindeki dosyaları alıp dosya sistemine fiziksel olarak yerleştirmektir. dpkg kendi başına internetten paket indiremez ve bağımlılık çözme yeteneğine sahip değildir. Bu nedenle dpkg, tek başına kullanıldığında eksik bağımlılık hataları üretir ve manuel müdahale gerektirir. APT ise dpkg'yi bir altyapı olarak kullanır; paketleri indirir, gerekli bağımlılıkları hesaplar ve tüm

kurulumu dpkg aracılığıyla sisteme uygulatır. Bu sebeple günlük kullanımda doğrudan dpkg yerine APT tercih edilir.

Örneğin apt install git komutu çalıştırıldığında, yalnızca git paketi değil, git'in ihtiyaç duyduğu tüm ek kütüphaneler ve yardımcı paketler de resmi depolardan otomatik olarak indirilir ve doğru sırayla sisteme kurulmuş olur. Kullanıcı bu sürecin hiçbir aşamasında bağımlılıkları tek tek düşünmek veya yönetmek zorunda kalmaz. Bu yapı, Debian tabanlı sistemlerde yazılım yönetimini güvenli, tutarlı ve kullanıcı açısından hataya kapalı hale getirir.

RPM (.rpm – Red Hat Package Manager)

RPM, Red Hat tabanlı Linux dağıtımlarında kullanılan paketleme formatıdır. Fedora, Red Hat Enterprise Linux (RHEL) ve CentOS gibi sistemlerde yazılım paketleri .rpm uzantısı ile dağıtılır. RPM, hem paket formatının adıdır hem de bu paketlerin nasıl yapılandırıldığını tanımlayan sistemdir.

RPM tabanlı sistemlerde genellikle dnf veya daha eski sistemlerde yum paket yöneticileri kullanılır. dnf, yüksek seviyeli bir paket yöneticisi olarak çalışır ve apt ile benzer bir role sahiptir. Yani paket indirme, bağımlılık çözme, güncelleme ve kaldırma işlemleri otomatik olarak yönetilir. yum ise dnf'in önceki sürümü olarak kabul edilir ve aynı mantıkla çalışır.

dnf install git komutu çalıştırıldığında, git paketi ve ihtiyaç duyduğu tüm bağımlılıklar resmi Fedora veya RHEL depolarından indirilerek sisteme kurulmuş olur.

Pacman (.pkg.tar.zst)

Pacman, Arch Linux ve Arch tabanlı Manjaro gibi dağıtımlarda kullanılan paket yöneticisidir. Pacman tarafından kullanılan paket formatı genellikle .pkg.tar.zst şeklindedir. Bu format, yazılım dosyalarının sıkıştırılmış ve sistem kurallarına uygun şekilde paketlenmiş halini ifade eder.

Pacman, hem düşük hem de yüksek seviyeli işlemleri tek bir araç içinde birleştirir. Bu sayede ayrı bir araç (apt–dpkg gibi) ayrımına gerek kalmaz. Bağımlılık çözme, paket indirme ve kurulum işlemleri tek bir komutla gerçekleştirilir.

pacman -S git komutu kullanıldığında, git paketi Arch depolarından çekilir ve sistemde gerekli dizinlere yerleştirilir. Buradaki -S parametresi, "senkronizasyon" anlamına gelir ve paketin depodan indirilerek kurulacağını belirtir.

Snap ve Flatpak (Container Tabanlı Paketler)

Snap ve Flatpak, klasik paket yönetim sistemlerinden farklı olarak container (izole edilmiş ortam) mantığıyla çalışan paketleme çözümleridir. Container kavramı, bir uygulamanın kendi bağımlılıklarıyla birlikte, sistemden büyük ölçüde bağımsız bir ortamda çalıştırılması anlamına gelir. Bu yaklaşım sayesinde uygulama, farklı Linux dağıtımlarında aynı şekilde çalışabilir.

Snap paketleri snap aracı ile, Flatpak paketleri ise flatpak aracı ile yönetilir. Bu sistemler dağıtımdan bağımsızdır; yani Debian, Fedora veya Arch fark etmeksizin aynı paket çalıştırılabilir. Uygulamalar genellikle kendi kütüphaneleriyle birlikte geldiği için bağımlılık çakışmaları büyük ölçüde önlenir.

Örnek olarak snap install spotify komutu çalıştırıldığında, Spotify uygulaması kendi çalışma ortamıyla birlikte sisteme kurulmuş olur. Bu uygulama, sistemdeki mevcut kütüphanelerden minimum düzeyde etkilenir.

Bu yaklaşım özellikle masaüstü uygulamalar için avantaj sağlarken, disk kullanımı ve sistem entegrasyonu açısından klasik paket yöneticilerine göre daha maliyetlidir.

APT Kullanmak (Advanced Package Tool)

APT (Advanced Package Tool), Debian ve türevi Linux dağıtımlarında (Ubuntu, Debian, Kali Linux gibi) yazılım paketlerinin yönetilmesi için kullanılan merkezi ve yüksek seviyeli bir paket yönetim aracıdır. Bu araç sayesinde sisteme yeni yazılımlar kurulabilir, mevcut yazılımlar güncellenebilir, artık ihtiyaç duyulmayan yazılımlar kaldırılabilir ve tüm bu işlemler sırasında yazılımlar arasındaki bağımlılıklar otomatik olarak çözümlenir. Kullanıcıdan yalnızca basit komutlar beklenir; arka plandaki karmaşık işlemler APT tarafından yönetilir. Bu yapı, sistemin hem güncel hem de tutarlı kalmasını sağlar. APT, Ubuntu ve Debian tabanlı sistemlerde varsayılan olarak kullanılan paket yöneticisidir.

1. Update (Paket Listesini Güncelleme)

APT ile herhangi bir kurulum veya güncelleme işlemine başlamadan önce paket listesinin güncellenmesi gerekir. sudo apt update komutu çalıştırıldığında, sistemde tanımlı olan tüm yazılım depolarına bağlanılır ve bu depolardaki en güncel paket bilgileri indirilir. Bu işlem sırasında herhangi bir yazılım kurulmaz veya güncellenmez; yalnızca “hangi paketin hangi sürümü mevcut” bilgisi yenilenir. Ekranda görülen Reading package lists ifadesi, sistemin bu paket listelerini okuduğunu gösterir. Bu adım atlanırsa, sistem eski bilgilerle çalışacağından güncel olmayan paketler kurulabilir.

```
┌──(root㉿kali)-[~]
└─# apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/contrib Sources [79.8 kB]
Get:3 http://kali.download/kali kali-rolling/non-free-firmware Sources [10.2 kB]
Get:4 http://kali.download/kali kali-rolling/non-free Sources [117 kB]
Get:5 http://kali.download/kali kali-rolling/main Sources [17.4 MB]
19% [5 Sources 5514 kB/17.4 MB 32%]                                    570 kB/s 2min 25s
```

2. Search ve Bilgi Alma (Search & Show)

Kurulmak istenen paketin adı tam olarak bilinmiyorsa arama yapılabilir. apt search komutu, hem paket adlarında hem de paket açıklamalarında arama yapar. Örneğin apt search htop komutu, adı veya açıklaması içinde “htop” geçen tüm paketleri listeler. Eğer yalnızca paket adları içinde arama yapılmak istenirse --names-only parametresi kullanılır. Bu durumda gereksiz sonuçlar elenmiş olur. Ayrıca regex (düzenli ifade) desteği sayesinde ^htop gibi ifadelerle yalnızca belirli bir desenle başlayan paketler bulunabilir.

Bir paket hakkında sürüm, boyut, bağımlılıklar gibi detaylı bilgi almak için apt show paket_adi komutu kullanılır. Kurulu sürüm ile depolardaki aday sürümün karşılaştırılması ise apt-cache policy paket_adi komutu ile yapılır. Bu bilgiler, özellikle sürüm uyumsuzluklarını anlamak için önemlidir.

3. Installation (Paket Kurulumu)

Bir paketi sisteme kurmak için sudo apt install paket_adi komutu kullanılır. Bu komut çalıştırıldığında APT, kurulacak paketi ve bu paketin çalışması için gerekli olan tüm bağımlılıkları otomatik olarak tespit eder, resmi depolardan indirir ve sisteme kurar. Kurulum sırasında ekranda görülen bilgiler; indirilecek veri miktarını, disk üzerinde kaplanacak alanı ve hangi paketlerin kurulacağını gösterir. Bu süreçte kullanıcıdan onay alınır ve ardından paketler sisteme yerleştirilir.

Sistemde kurulu tüm paketleri güncellemek için sudo apt upgrade komutu kullanılır. Bu komut, mevcut paketleri daha yeni sürümlerine yükseltir ancak yeni bağımlılıklar eklemez veya mevcut paketleri kaldırmaz. Eğer sistem genelinde daha kapsamlı bir güncelleme yapılmak istenirse sudo apt dist-upgrade komutu tercih edilir; bu komut gerekirse yeni paketler ekleyebilir veya eski paketleri kaldırabilir.

4. Removal (Paket Kaldırma)

Kurulu bir yazılımı sistemden kaldırmak için iki temel yöntem bulunur. sudo apt remove paket_adi komutu, programı kaldırır ancak yapılandırma dosyalarını (ayar dosyalarını) sistemde bırakır. Bu yöntem, ileride aynı yazılım yeniden kurulmak istendiğinde eski ayarların korunmasını sağlar. Eğer yazılımın tüm izleriyle birlikte tamamen silinmesi istenirse sudo apt purge paket_adi komutu kullanılır. Bu komut, programın kendisini ve tüm yapılandırma dosyalarını sistemden temizler.

5. Clean ve Autoremove (Temizlik)

Zamanla sistemde, kaldırılmış programlardan kalan ve artık hiçbir paket tarafından kullanılmayan bağımlılıklar (kütüphaneler) birikebilir. sudo apt autoremove komutu, bu gereksiz bağımlılıkları tespit eder ve sistemden siler. Bu işlem disk alanı kazanmak ve sistemi temiz tutmak için düzenli olarak uygulanır

DPKG (Debian Package)

DPKG, Debian tabanlı Linux sistemlerinde .deb uzantılı paketlerin sisteme doğrudan kurulmasını sağlayan düşük seviyeli bir paket yönetim aracıdır. İnternetten manuel olarak indirilen paketler (örneğin Discord veya Google Chrome'un resmi sitesinden indirilen .deb dosyaları) APT tarafından tek başına kurulamaz; bu noktada DPKG devreye girer. sudo dpkg -i paket.deb komutu çalıştırıldığında, paket dosyasının içeriği sistemde ilgili dizinlere açılır ve program kurulmaya çalışılır. Ancak DPKG'nin kritik bir sınırlaması vardır: bağımlılık çözme yeteneği yoktur. Yani kurulan programın ihtiyaç duyduğu ek kütüphaneler sistemde yoksa kurulum yarım kalır ve hata mesajı gösterilir. Bu durum bir bozukluk değil, DPKG'nin tasarım gereğidir.

Bu tür bir hata alındığında çözüm olarak sudo apt install -f komutu kullanılır. Bu komut, sistemde yarım kalmış veya eksik bağımlılıklara sahip paketleri tespit eder ve gerekli bağımlılıkları resmi depolardan indirerek kurulumu tamamlar. Burada dikkat edilmesi gereken nokta şudur: DPKG paketi yerleştirir, APT ise eksik parçaları tamamlar. Bu nedenle günlük kullanımda APT tercih edilirken, manuel .deb kurulumlarında DPKG kaçınılmaz hale gelir.

Modern Paketler: Snap

Snap, geleneksel paket yönetim sistemlerine alternatif olarak geliştirilmiş, daha modern bir paketleme yaklaşımıdır. Snap paketleri, bir yazılımın çalışması için gerekli olan tüm bağımlılıkları kendi içinde barındırır. Bu sayede sistemde hangi kütüphanenin hangi sürümünün kurulu olduğu önemini büyük ölçüde kaybeder. Snap paketleri izole (sandbox) bir yapıda çalıştırılır; yani sistemin geri kalanından belirli ölçüde ayrılmıştır. Bu yapı hem güvenliği artırır hem de "bağımlılık cehennemi" olarak bilinen sorunları ortadan kaldırır. sudo snap install spotify gibi tek bir komutla, sistemde hiçbir ek ayar yapılmadan uygulama kurulabilir. Bu yaklaşım özellikle masaüstü uygulamaları ve farklı dağıtımlar arasında tutarlılık sağlamak amacıyla tercih edilir.

Özetle; DPKG manuel ve düşük seviyeli kurulumlar için kullanılırken, Snap modern, bağımsız ve dağıtımdan bağımsız bir paketleme çözümü sunar. Bu araçların her biri farklı bir ihtiyaca cevap vermek üzere tasarlanmıştır.

Kullanıcı Yönetimi (User Management)

Linux işletim sistemlerinde kullanıcı yönetimi, sistem güvenliğinin sağlanması ve kaynakların kontrollü biçimde paylaşılabilmesi açısından kritik bir rol üstlenir. Linux'un çok kullanıcılı (multi-user) yapısı sayesinde aynı sistem üzerinde birden fazla kullanıcı eşzamanlı olarak çalışabilir ve bu kullanıcılar birbirlerinden

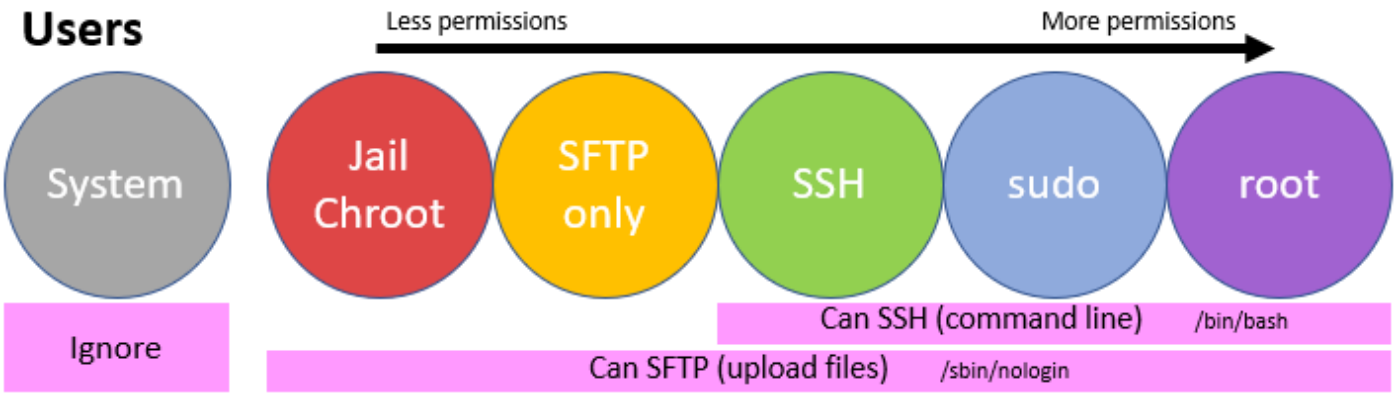
mantıksal olarak ayrılmış şekilde yönetilir. Bu bölümde kullanıcı kavramının ne olduğu ve neden temel bir yapı taşı olduğu açıklanmaktadır.

Linux'ta her kullanıcı bir kimlik ile tanımlanır. Bu kimlik, kullanıcı adı (username) ve sistem içinde benzersiz olan sayısal bir değer olan UID (User ID) üzerinden oluşturulur. Sistem, aslında kullanıcı adından ziyade bu UID değerini esas alarak işlem yapar. Her kullanıcıya özel bir home dizini atanır (/home/kullanici_adi) ve bu dizin, kullanıcının kişisel dosyalarını ve ayarlarını barındırır. Varsayılan yapılandırmada bir kullanıcı, başka bir kullanıcının dosyalarına erişemez; bu durum izolasyon (isolation) olarak adlandırılır ve güvenliğin temelini oluşturur.

Kullanıcıların sistem üzerindeki yetkileri sınırlıdır. Dosya silme, sistem ayarlarını değiştirme veya başka kullanıcıları yönetme gibi kritik işlemler yalnızca root (super user) tarafından gerçekleştirilebilir. Root kullanıcı, sistem üzerindeki tüm yetkilere sahiptir ve herhangi bir kısıtlamaya tabi değildir. Bu yetki ayrımı sayesinde, bir kullanıcının hata yapması veya zararlı bir yazılım çalıştırması durumunda, sistemin tamamı etkilenmez. Ancak bu tür işlemler root yetkileriyle yapılırsa, tüm sistem risk altına girebilir.

Sonuç olarak kullanıcı yönetimi; güvenlik, kararlılık ve düzenli kaynak kullanımı için vazgeçilmez bir mekanizma olarak tasarlanmıştır. Linux'un çok kullanıcılı mimarisi, bireysel hataların sistem geneline yayılmasını engelleyen güçlü bir koruma katmanı sunar.

Types of Users (Kullanıcı Tipleri)



Linux sistemlerinde kullanıcılar genel olarak iki gruba ayrılır: sistem kullanıcıları ve normal (regular) kullanıcılar. Sistem kullanıcıları, işletim sistemi kurulumu sırasında otomatik olarak oluşturulur ve genellikle arka planda çalışan servisleri ve uygulamaları çalıştırmak için kullanılır. Bu kullanıcıların sisteme giriş yapması amaçlanmaz; daha çok web sunucusu, veritabanı veya ağ servisleri gibi bileşenlerin güvenli şekilde çalıştırılması için vardırlar.

Normal kullanıcılar ise sistem yöneticisi (administrator) tarafından oluşturulan ve bilgisayarı doğrudan kullanan gerçek kullanıcılardır. Bu kullanıcılar oturum açabilir, kendi dosyalarını oluşturabilir ve kendilerine verilen izinler çerçevesinde sistem kaynaklarına erişebilir. Yetkileri sınırlıdır ve kritik sistem işlemleri

yalnızca gerekli izinler verilmişse gerçekleştirilebilir. Bu ayrım, sistem servisleri ile gerçek kullanıcıların birbirinden izole edilmesini sağlayarak güvenliği artırır.

Creating a User (Kullanıcı Oluşturma)

Linux'te kullanıcı oluşturmak için iki temel komut bulunur: adduser ve useradd. Bu iki komut aynı amaca hizmet etse de kullanım seviyeleri ve sundukları kolaylıklar farklıdır. adduser, kullanıcı oluşturma sürecini otomatikleştiren ve yönlendirici sorular sorarak ilerleyen yüksek seviyeli bir araçtır. Komut çalıştırıldığında kullanıcı adı oluşturulur, aynı isimde bir grup tanımlanır, ev dizini (/home/kullanıcı_adı) otomatik olarak oluşturulur, varsayılan yapılandırma dosyaları /etc/skel dizininden kopyalanır ve parola belirlenmesi istenir. Ayrıca kullanıcının adı gibi isteğe bağlı bilgiler de bu aşamada girilebilir. Bu nedenle günlük kullanımda ve yeni başlayanlar için önerilen yöntem adduser'dır.

useradd ise daha düşük seviyeli ve daha "ham" bir araçtır. Varsayılan haliyle çalıştırıldığında kullanıcıyı oluşturur ancak ev dizini oluşturmaz, parola sormaz ve ek ayarlar yapmaz. Bu nedenle genellikle parametrelerle birlikte kullanılır. Örneğin -m parametresi ev dizininin oluşturulmasını, -s /bin/bash parametresi ise kullanıcının hangi kabuğu (shell) kullanacağını belirtir. useradd, daha çok otomasyon senaryolarında veya sistem yöneticilerinin kullanıcıyı tamamen kendi istediği şekilde tanımlamak istediği durumlarda tercih edilir. Özetle adduser "kullanıcı dostu ve güvenli", useradd ise "kontrolün tamamen yöneticide olduğu" bir yöntem olarak düşünülmelidir.

Kullanıcı Bilgilerini Görüntüleme (id, who, w, last)

```
┌──(root㉿kali)-[~]
└─# id
uid=0(root) gid=0(root) groups=0(root)

┌──(root㉿kali)-[~]
└─# who
root     sshd pts/1   Dec 22 09:15 (192.168.64.1)
root     seat0        Dec 22 09:14 (:0)

┌──(root㉿kali)-[~]
└─# w
 11:47:00 up  2:12,  2 users,  load average: 0.27, 0.11, 0.02
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU  WHAT
root     pts/1    192.168.64.1     09:15    4.00s  7.05s   ?    w
root              -                09:14           0.00s   ?    lightdm --session-child 13 24

┌──(root㉿kali)-[~]
└─# last
root     pts/1        192.168.64.1     Mon Dec 22 09:15 - still logged in
root     pts/2        192.168.64.1     Mon Dec 22 00:43 - 01:47  (01:03)
kali     pts/1        192.168.64.1     Sun Dec 21 21:12 - 21:20  (00:07)

wtmpdb begins Sun Dec 21 21:12:37 2025
```

Linux'te kullanıcı yönetimi yalnızca kullanıcı oluşturmakla sınırlı değildir; hangi kullanıcının kim olduğu, hangi yetkilere sahip olduğu ve sistemde o anda ne yaptığı da net şekilde izlenebilmelidir. Bu amaçla kullanılan temel komutlar id, who, w ve last'tir.

id komutu, bir kullanıcının kimliğini teknik olarak gösterir. Çıktıda yer alan uid (User ID) kullanıcının sistemdeki benzersiz numarasını, gid (Group ID) kullanıcının birincil grubunu ve groups kısmı ise üye olduğu tüm grupları ifade eder. Bu bilgiler, kullanıcının hangi dosyalara erişebileceğini ve hangi işlemleri yapabileceğini anlamak için kritik öneme sahiptir. Bir kullanıcının yetki problemi yaşaması durumunda ilk bakılması gereken yer genellikle id çıktısıdır.

```
┌──(root㉿ kali)-[~]
└─# id
uid=0(root) gid=0(root) groups=0(root)
```

who komutu, o anda sisteme bağlı olan kullanıcıları gösterir. Hangi kullanıcının hangi terminalden (TTY/PTS) ve hangi IP adresinden bağlandığı bu komutla görülebilir. Bu, özellikle çok kullanıcılı sistemlerde veya uzaktan bağlantıların denetlenmesinde kullanılır. who yalnızca “kim bağlı” sorusuna cevap verir; kullanıcının ne yaptığıyla ilgilenmez.

```
┌──(root㉿ kali)-[~]
└─# who
root     sshd pts/1   Dec 22 09:15 (192.168.64.1)
root     seat0        Dec 22 09:14 (:0)
```

w komutu, who’ya göre daha ayrıntılı bilgi sunar. Sistemin ne kadar süredir açık olduğu, kullanıcı sayısı ve sistem yükü gibi genel bilgilerle başlar. Ardından her kullanıcının ne zamandır bağlı olduğu, ne kadar süredir işlem yapmadığı (IDLE) ve şu anda hangi komutu çalıştırdığı (WHAT) gösterilir. Bu nedenle w, “kim bağlı ve şu anda ne yapıyor?” sorusunun cevabını verir. Sistem performansını veya olağandışı aktiviteleri analiz ederken oldukça değerlidir.

IDLE, bir kullanıcının klavye veya terminal üzerinden ne kadar süredir hiçbir işlem yapmadığını gösteren süredir. Yani kullanıcının oturumu açık olsa bile, en son ne zaman bir komut çalıştırdığı veya giriş yaptığı esas alınır ve o andan itibaren geçen süre IDLE olarak yazdırılır.

```
┌──(root㉿ kali)-[~]
└─# w
 11:47:00 up  2:12,  2 users,  load average: 0.27, 0.11, 0.02
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU  WHAT
root     pts/1    192.168.64.1     09:15    4.00s  7.05s   ?    w
root              -                09:14           0.00s   ?    lightdm --session-child 13 24
```

last komutu ise geçmişe dönük giriş kayıtlarını gösterir. Hangi kullanıcının ne zaman, nereden bağlandığı ve oturumunun bitip bitmediği bu komutla incelenir. Bu bilgiler sistemdeki wtmp kayıtlarından okunur ve özellikle güvenlik incelemelerinde, yetkisiz erişim şüphesi olduğunda veya kullanıcı aktivitelerinin zaman çizelgesini çıkarmak gerektiğinde kullanılır.

```
┌──(root㉿ kali)-[~]
└─# last
root     pts/1        192.168.64.1     Mon Dec 22 09:15 - still logged in
root     pts/2        192.168.64.1     Mon Dec 22 00:43 - 01:47  (01:03)
kali     pts/1        192.168.64.1     Sun Dec 21 21:12 - 21:20  (00:07)

wtmpdb begins Sun Dec 21 21:12:37 2025
```

Kritik Sistem Dosyaları

/etc/passwd dosyası, sistemdeki tüm kullanıcıların temel kimlik bilgilerinin saklandığı dosyadır ve herkes tarafından okunabilir durumdadır. Her satır bir kullanıcıyı temsil eder ve satır içindeki bilgiler : karakteri ile ayrılmış yedi alandan oluşur. Bu alanlar sırasıyla kullanıcı adı (sisteme girişte kullanılan isim), parola alanı (artık gerçek parola burada tutulmaz, güvenlik nedeniyle sadece x yazar), UID yani kullanıcı numarası (root her zaman 0'dır), GID yani kullanıcının birincil grup numarası, açıklama alanı (GECOS olarak adlandırılır; ad-soyad gibi ek bilgiler burada yer alır), kullanıcının giriş yaptıktan sonra yönlendirildiği home dizini ve kullanıcının varsayılan kabuğu (shell) bilgisidir. Bu dosya, bir kullanıcının sistemde var olup olmadığını ve hangi temel özelliklerle tanımlandığını gösterir. Güvenlik nedeniyle parola bilgisi burada tutulmaz; bu bilinçli bir tasarımdır ve yetkisiz kişilerin parolalara erişmesini engeller.

```
┌──(root㉿kali)-[~]
└─# cat /etc/passwd
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

/etc/shadow dosyası ise kullanıcı doğrulamasının yapıldığı asıl güvenlik dosyasıdır. Gerçek parolalar burada düz metin olarak değil, geri döndürülemez şekilde hash'lenmiş hâlde saklanır ve bu dosyaya yalnızca root kullanıcısı erişebilir. Her kullanıcı için parolanın hash değeri, parolanın en son ne zaman değiştirildiği, minimum ve maksimum geçerlilik süresi ile parola süresi dolmadan önce kaç gün uyarı verileceği gibi bilgiler tutulur. Parola alanının 6 ile başlaması, parolanın SHA-512 algoritması ile şifrelenmiş olduğunu gösterir. Parola alanında ! veya * bulunması ise hesabın kilitli olduğunu ifade eder. Kilitli kullanıcı, sistemde var olmaya devam eden ancak giriş yapması bilinçli olarak engellenmiş kullanıcıdır. Bu durum genellikle güvenlik amacıyla oluşturulan sistem kullanıcılarında, geçici olarak erişimi kapatılan hesaplarda veya yanlış parola denemeleri sonrası otomatik kilitlenen kullanıcılarda görülür. Yani kilitli kullanıcı silinmiş değildir; sadece sisteme giriş yetkisi pasif hâle getirilmiştir.

```
┌──(root㉿kali)-[~]
└─# cat /etc/shadow
root:$y$j9T$HBvvjJ9nR4t/0bbUricLn1$9v2kozTgVW.S2xubat2H9BwACNrKHiD6hvOM.SlaQo8:20396:0:99999:7:::
daemon:*:20390:0:99999:7:::
bin:*:20390:0:99999:7:::
sys:*:20390:0:99999:7:::
sync:*:20390:0:99999:7:::
games:*:20390:0:99999:7:::
man:*:20390:0:99999:7:::
lp:*:20390:0:99999:7:::
mail:*:20390:0:99999:7:::
```

/etc/skel dizini, yeni oluşturulacak kullanıcılar için hazırlanan varsayılan başlangıç şablonunu tanımlar. Yeni bir kullanıcı oluşturulduğunda (örneğin adduser veya useradd -m ile), bu dizinin içindeki tüm dosyalar kopyalanarak kullanıcının kendi home dizinine aktarılır. Bu noktada kritik detay şudur: .bashrc dosyası

merkezi bir dosya değildir; her kullanıcının home dizini altında kendine ait bir .bashrc dosyası bulunur. Ancak bu dosyanın ilk hâli, /etc/skel içindeki .bashrc dosyasından türetilir. Bu yapı sayesinde kullanıcı sisteme ilk kez giriş yaptığında, komut satırı davranışı (.bashrc ile), oturum başlangıç ayarları (.profile ile) ve temel ortam değişkenleri önceden tanımlanmış olur. Böylece tüm kullanıcılar, sistem yöneticisi tarafından belirlenmiş ortak bir başlangıç yapılandırmasıyla işe başlar; sonrasında ise kendi .bashrc dosyalarını bağımsız olarak düzenleyebilirler. Bu yaklaşım, çok kullanıcılı sistemlerde hem standart bir başlangıç ortamı oluşturulmasını hem de kullanıcı bazlı özelleştirmeye izin verilmesini sağlar.

Usermod Parametreleri ve Kullanım Mantığı

usermod komutu, mevcut bir kullanıcının sistemdeki özelliklerini değiştirmek için kullanılır ve özellikle yetki yönetimi açısından kritik bir role sahiptir. Bu komutla yapılan değişiklikler, kullanıcının kimliğini silmeden onun sisteme nasıl ve ne ölçüde erişebileceğini belirler.

Gruba Ekleme (-aG sudo)

Linux'ta yönetici yetkileri doğrudan kullanıcıya değil, belirli gruplara verilir. Bu gruplardan en önemlisi sudo grubudur. sudo grubu, üyelerine sudo komutu aracılığıyla root (tam yetkili) işlemler yapma hakkı tanır. sudo usermod -aG sudo john komutunda -G parametresi, kullanıcının eklenmek istendiği ek grubu belirtir; burada bu grup sudo grubudur. -a (append) parametresi ise mevcut grup üyeliklerinin korunmasını sağlar. Eğer -a kullanılmazsa, kullanıcı yalnızca belirtilen gruba atanır ve daha önce üyesi olduğu tüm gruplardan çıkarılır. Bu nedenle -aG birlikte kullanılması gereken, güvenlik açısından kritik bir kombinasyondur. Bu işlemden sonra kullanıcı oturumu kapatıp tekrar açtığında, sudo yetkisi aktif hâle gelir.

Hesap Kilitleme (-L) ve Açma (-U)

Bir kullanıcının sisteme giriş yapmasını geçici olarak engellemek gerektiğinde usermod -L parametresi kullanılır. sudo usermod -L john komutu çalıştırıldığında, kullanıcının parolasına ! benzeri bir kilit işareti eklenir ve parola doğrulaması başarısız hâle gelir. Bu işlem, kullanıcıyı silmeden erişimi kapatır; dosyalar, ayarlar ve kullanıcı kimliği korunur. Genellikle izin süresi dolan, geçici olarak askıya alınan veya güvenlik incelemesi yapılan hesaplar için tercih edilir. Kilitli bir hesabın tekrar kullanılabilir hâle getirilmesi için sudo usermod -U john komutu kullanılır ve parola alanındaki kilit kaldırılır.

Parola Süresi Yönetimi (chage)

chage komutu, bir kullanıcının parolasının yaşam döngüsünü kontrol etmek için kullanılır. Bu araç sayesinde parolanın ne zaman değiştirildiği, ne kadar süre geçerli olduğu ve süresi dolmadan önce kullanıcıya kaç gün önceden uyarı verileceği belirlenir. sudo chage -l john komutu, kullanıcıya ait tüm parola zamanlama bilgilerini okunabilir şekilde listeler. Örneğin sudo chage -M 90 john komutu, parolanın en fazla 90 gün geçerli olacağını belirtir; bu sürenin sonunda kullanıcı bir sonraki girişte parolasını değiştirmek zorunda kalır. Bu mekanizma, özellikle kurumsal ortamlarda parola güvenliğini zorunlu hâle getirmek için kullanılır.

Kullanıcı Silme (userdel)

Bir kullanıcının sistemden tamamen kaldırılması gerektiğinde userdel komutu kullanılır. sudo userdel john komutu yalnızca kullanıcı hesabını siler, ancak kullanıcının home dizini ve dosyaları sistemde kalır. Bu durum, ileride inceleme yapılması veya verilerin yedeklenmesi isteniyorsa tercih edilebilir. Eğer kullanıcının tüm izleriyle birlikte kaldırılması isteniyorsa sudo userdel -r john komutu kullanılır; -r parametresi, kullanıcının home dizinini ve içindeki tüm dosyaları da siler. Bu işlem geri alınamaz olduğu için, özellikle üretim sistemlerinde dikkatli uygulanması gerekir.

Group Management

Linux işletim sistemlerinde grup yönetimi, kullanıcı yönetimi kadar kritik bir konudur. Gruplar, birden fazla kullanıcıya aynı anda erişim yetkileri ve izinler atanmasını sağlayan yapılardır. Bu sayede her kullanıcıya tek tek yetki vermek yerine, belirli bir grup tanımlanır ve bu gruba üye olan tüm kullanıcılar aynı kaynaklara erişebilir hâle getirilir. Bu yaklaşım, hem güvenliği artırır hem de sistem yönetimini sadeleştirir.

Gruplar özellikle dosya ve dizin izinlerinde, servis erişimlerinde ve yönetici yetkilerinin dağıtılmasında kullanılır. Örneğin bir proje klasörüne yalnızca belirli kullanıcıların erişmesi isteniyorsa, bu kullanıcılar aynı gruba eklenir ve klasör izinleri bu grup üzerinden tanımlanır. Böylece kullanıcı ekleme veya çıkarma işlemleri, izinleri yeniden düzenlemeye gerek kalmadan kolayca yapılabilir.

Bu bölümde grup oluşturma, mevcut grupları görüntüleme, kullanıcılara grup atama ve grupları silme gibi temel işlemler ele alınır. Anlatılan yapı sayesinde Linux sistemlerinde yetki kontrolünün neden kullanıcılar üzerinden değil, gruplar üzerinden tasarlandığı net bir şekilde anlaşılır.

Temel Komutlar

groupadd

groupadd komutu, sistemde yeni bir grup oluşturmak için kullanılır. Bu komut çalıştırıldığında sistem, gruba benzersiz bir GID (Group ID) atar ve grup bilgisi grup veritabanına (genellikle /etc/group) eklenir. Oluşturulan grup başlangıçta boştur; yani henüz hiçbir kullanıcı bu gruba dahil değildir. Gruplar, dosya ve dizin izinlerini tek tek kullanıcılar yerine toplu olarak yönetmek için kullanılır.

Örnek senaryo:

Bir şirkette “muhasebe” departmanına ait dosyalara yalnızca bu ekip erişmelidir.

sudo groupadd muhasebe

Bu komutla muhasebe adında bir grup oluşturulur. Daha sonra ilgili kullanıcılar bu gruba eklenir ve dosya izinleri bu grup üzerinden verilir. Böylece tek tek kullanıcı yetkilendirmek yerine grup bazlı kontrol sağlanmış olur.

groupdel

groupdel komutu, sistemde tanımlı olan bir grubun tamamen silinmesini sağlar. Bu işlem yalnızca grubu kaldırır; kullanıcı hesapları silinmez. Ancak silinen grup artık sistemde var olmadığı için, bu gruba bağlı tüm yetkilendirmeler anlamsız hâle gelir. Bu nedenle grup silme işlemi genellikle artık kullanılmayan projeler veya servisler sonlandırıldığında yapılır.

Örnek senaryo:

“muhasebe” grubu artık kullanılmıyor ve sistemden kaldırılmak isteniyor.

sudo groupdel muhasebe

Bu işlemden sonra:

Kullanıcılar muhasebe grubundan otomatik olarak çıkarılmış olur. Daha önce bu gruba ait olan dosyalar hâlâ sistemde durur, ancak grup adı yerine sayısal GID görünür. Bu durum yetki karmaşasına yol açabileceği için, silmeden önce grubun kullandığı dosyalar kontrol edilmelidir.

groups

groups komutu, bir kullanıcının hangi gruplara üye olduğunu gösterir. Linux’ta bir kullanıcının dosyalara erişimi büyük ölçüde grup üyelikleri üzerinden belirlendiği için bu komut yetki kontrolü açısından oldukça önemlidir.

Örnek senaryo:

Oturum açmış kullanıcının hangi gruplara dahil olduğunu görmek:

groups

Çıktı örneği:

john sudo developers

Bu çıktı, kullanıcının sudo grubunda olduğunu ve yönetici yetkilerine sahip olabileceğini gösterir. Eğer başka bir kullanıcının grupları kontrol edilmek istenirse:

groups rolvarn

Bu şekilde, bir kullanıcının beklenmeyen bir yetkisi olup olmadığı kolayca tespit edilir.

getent group

getent group komutu, sistemde tanımlı grupların ayrıntılı bilgilerini gösterir. Bu komut, yalnızca /etc/group dosyasını değil; sistemin kullandığı tüm kimlik kaynaklarını (örneğin LDAP gibi merkezi dizin servislerini) dikkate alır. Bu nedenle özellikle kurumsal veya çok kullanıcılı ortamlarda güvenilir bir bilgi kaynağıdır.

Örnek senaryo:

Tüm grupları ve üyelerini listelemek:

getent group

Belirli bir grubun detaylarını görmek:

getent group sudo

Örnek çıktı:

sudo:x:27:root,john

Bu çıktı şunu söyler:

Grup adı: sudo

GID: 27

Bu gruba üye kullanıcılar: root ve john

Bu sayede bir grubun kimleri kapsadığı net şekilde görülür ve yetkilendirme kontrolleri sağlıklı biçimde yapılır.

/etc/group Dosyası Nedir ve Ne İşe Yarar?

/etc/group dosyası, sistemde tanımlı olan tüm grupların kimlik bilgilerinin tutulduğu temel yapılandırma dosyasıdır. Bu dosya sayesinde Linux, hangi grubun var olduğunu, bu grubun sayısal kimliğini (GID) ve hangi kullanıcıların bu gruba üye olduğunu bilir. Dosya herkes tarafından okunabilir durumdadır; çünkü grup isimlerinin bilinmesi güvenlik açığı oluşturmaz. Ancak dosya yalnızca root tarafından değiştirilebilir.

```
┌──(root㉿kali)-[~]
└─# cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:kali
tty:x:5:
```

Bu dosyada her satır bir grubu temsil eder ve bilgiler iki nokta (:) karakteri ile ayrılmış dört alandan oluşur.

/etc/group Dosyasındaki Alanların Anlamı

Örnek satır:

dev:x:1002:john,root

İlk alan grup adıdır. Bu, insanların gördüğü ve kullandığı isimdir. Örnekte bu isim dev olarak tanımlanmıştır. İkinci alan grup parolası alanıdır. Günümüzde neredeyse hiç kullanılmaz ve genellikle x olarak bırakılır. Tarihsel olarak bazı sistemlerde grup parolası ile geçici erişim sağlanabiliyordu; modern Linux sistemlerinde bu yöntem terk edilmiştir. Üçüncü alan GID (Group ID) yani grup kimlik numarasıdır. Linux, grupları isimle değil bu numara ile tanır. Dosya ve dizin izinleri bu sayıya göre kontrol edilir. Örnekte 1002 sayısı grubun gerçek kimliğidir. Dördüncü alan gruba üye olan kullanıcıların listesidir. Virgül ile ayrılır. Burada yazan kullanıcılar, bu gruba ikincil grup olarak üyedir. Örnekte john ve rolvarn bu grubun üyeleridir.

Grup Oluşturma ve Sorgulama (groupadd ve getent)

Yeni bir grup oluşturmak için groupadd komutu kullanılır. Bu komut, sistemde yeni bir grup tanımlar ve ona benzersiz bir GID atar.

Örneğin:

sudo groupadd dev

Bu işlemden sonra grubun gerçekten oluşup oluşmadığını ve detaylarını görmek için getent group komutu kullanılır. Bu komut, sistemin grup veritabanına bakarak bilgiyi getirir.

getent group dev

Çıktı, grubun adı, GID'si ve üye listesi hakkında net bilgi verir. Eğer kullanıcı listesi boş görünüyorsa, bu gruba henüz kimse eklenmemiş demektir.

Kullanıcıyı Gruba Ekleme (usermod -aG)

Bir kullanıcıyı mevcut bir gruba eklemenin en güvenli ve doğru yolu usermod -aG komutudur. Buradaki parametrelerin anlamı kritik önemdedir.

-G parametresi, kullanıcının ekleneceği grubu belirtir.

-a (append) parametresi ise "mevcut grupları koru, buna ekle" anlamına gelir.

Örneğin john kullanıcısını dev grubuna eklemek için:

sudo usermod -aG dev john

Bu işlemden sonra kullanıcı gruba eklenmiş olur. Ancak değişikliğin etkili olması için kullanıcının oturumu kapatıp yeniden açması gerekir; çünkü grup bilgileri oturum başında yüklenir.

Kontrol için:

id john

Çıktıda groups= kısmında artık dev grubunun da yer aldığı görülür.

Kritik Uyarı: -a Parametresi Neden Hayati?

Eğer -a parametresi kullanılmazsa, Linux şunu varsayar:

"Bu kullanıcının tüm grup üyeliklerini sil, sadece bunu bırak."

Yani:

sudo usermod -G dev john

Bu komut, john kullanıcısını sudo dahil tüm diğer gruplardan çıkarır. Bu durum sistem erişimini tamamen bozabilir. Bu yüzden -aG birlikte kullanılmadıkça usermod komutu yarım kullanılmış sayılır.

Kullanıcıyı Gruptan Çıkarma (gpasswd -d)

Bir kullanıcıyı bir gruptan çıkarmak için gpasswd -d komutu kullanılır. Bu komut yalnızca belirtilen grubu etkiler; diğer grup üyeliklerine dokunmaz. Örneğin john kullanıcısını dev grubundan çıkarmak için:

sudo gpasswd -d john dev

Bu işlemden sonra kullanıcı artık o grubun yetkilerine sahip olmaz. Yine aynı şekilde, etkinin görülmesi için oturum yenilenmelidir.

Pratik Senaryo: Paylaşılan Proje Dizini Mantığı

Bu senaryo, grup kavramının neden var olduğunu net şekilde gösterir.

Önce proje için özel bir grup oluşturulur:

sudo groupadd project

Projede çalışacak kullanıcı bu gruba eklenir:

sudo usermod -aG project rolvarn

Ardından proje dizini oluşturulur:

sudo mkdir /opt/project

Bu dizinin grup sahipliği, proje grubuna verilir:

sudo chown :project /opt/project

Burada başına kullanıcı adı yazılmamasının sebebi, sadece grup bilgisinin değiştirilmek istenmesidir.

Son olarak izinler ayarlanır:

sudo chmod 770 /opt/project

Bu izin şu anlama gelir:

Sahip: tam yetki

Grup: tam yetki

Diğer herkes: hiçbir yetki yok

Sonuç olarak, sadece project grubuna üye olan kullanıcılar bu dizine girebilir, dosya oluşturabilir ve değiştirebilir. Sistemdeki diğer kullanıcılar dizinin varlığını görse bile içine giremez.

Permissions (İzinler)

Permissions, Linux'te dosya ve dizinlere kimlerin ne yapabileceğini belirleyen temel güvenlik mekanizmasıdır. Linux çok kullanıcılı bir sistem olduğu için, her dosyanın bir sahibi vardır ve bu dosya üzerinde okuma (read), yazma (write) ve çalıştırma (execute) gibi işlemleri kimin yapabileceği açık kurallarla sınırlandırılır. Aksi hâlde sistem kaosa sürüklenirdi. İzinler, bir dosyanın yalnızca sahibine mi, belirli bir gruba mı yoksa sistemdeki herkese mi açık olduğunu net biçimde tanımlar ve bu tanım çekirdeğin (kernel) seviyesinde uygulanır.

İzinlerin var olmasının ilk nedeni gizliliktir. Her kullanıcının e-postaları, SSH anahtarları, kişisel belgeleri gibi başkalarının görmemesi gereken dosyaları vardır. Dosya izinleri sayesinde başka bir kullanıcı bu dosyaları okuyamaz. İkinci neden bütünlüktür (integrity). Sistem dosyaları, yapılandırma dosyaları veya uygulama dosyaları herkes tarafından değiştirilebilseydi, tek bir yanlış komut tüm sistemi kullanılmaz hâle getirebilirdi. İzinler, "bu dosyayı kim değiştirebilir?" sorusuna kesin bir cevap verir. Üçüncü ve en kritik neden ise güvenliktir. Zararlı bir yazılım, çalıştırma izni olmayan bir dosyayı doğrudan çalıştıramaz; yetkisi olmayan bir dizine kendini kopyalayamaz veya sistem geneline yayılamaz.

Özetle, izinler Linux'te "iyi niyet" üzerine değil, zorlayıcı kurallar üzerine kurulu bir güvenlik katmanıdır. Eğer bu mekanizma olmasaydı, herhangi bir kullanıcı /bin, /etc veya /lib altındaki kritik bir dosyayı silebilir, sistemin açılmamasına veya anında çökmesine neden olabilirdi. Linux'un yıllardır sağlam ve güvenilir olmasının temel sebeplerinden biri, bu izin modelinin basit ama acımasız derecede katı olmasıdır.

ls –l Komutu ile Dosya İzinlerini ve Özelliklerini Kontrol Etme

```
┌──(root㉿kali)-[~/Desktop]
└─# ls -l
total 4
-rw-r--r-- 1 root root 8 Dec 22 11:14 test.txt
```

Linux dosya sisteminde bir dosya ya da dizinin izinlerini okumak için en temel araç ls -l komutudur. Bu komut, dosyaları yalnızca listelemekle kalmaz; aynı zamanda kime ait olduklarını, hangi gruba bağlı olduklarını ve kimlerin bu dosya üzerinde ne yapabileceğini açık biçimde gösterir. Güvenlik ve yetkilendirme mantığını gerçekten anlamak için bu çıktıyı doğru okumak zorunludur, ezberlenerek geçilecek bir konu değildir.

Örnek bir çıktı üzerinden ilerleyelim:

-rwxr--r-- 2 john development 4096 Jul 29 12:34 notes.txt

Bu satır soldan sağa doğru okunur ve her bölümün net bir anlamı vardır. En baştaki karakter dosya türünü belirtir. Eğer burada d görülseydi bunun bir dizin olduğu anlaşılırdı; - olması bunun normal bir dosya

olduğunu gösterir. Bu tek karakter bile sistem açısından kritiktir çünkü dizin izinleri ile dosya izinleri aynı şekilde yorumlanmaz.

Dosya türünden sonra gelen rwxr--r-- ifadesi asıl izin yapısını temsil eder ve toplam dokuz karakterden oluşur. Bu dokuz karakter üçlü gruplara ayrılır. İlk üçlü (rwx) dosyanın sahibinin izinlerini gösterir; burada r okuma, w yazma, x çalıştırma anlamına gelir. Yani dosya sahibi bu dosyayı okuyabilir, düzenleyebilir ve çalıştırabilir. İkinci üçlü (r--) dosyanın ait olduğu grubun izinlerini ifade eder; bu gruptaki kullanıcılar dosyayı yalnızca okuyabilir, değiştiremez veya çalıştıramaz. Üçüncü üçlü (r--) ise sistemdeki diğer tüm kullanıcılar için geçerli olan izinlerdir ve onlar da yalnızca okuma yetkisine sahiptir. Bu yapı Linux'te yetkilendirmenin bel kemiğidir.

İzinlerden sonra gelen 2 sayısı, dosyanın sahip olduğu hard link sayısını gösterir. Bu sayı genellikle yeni başlayanlar tarafından göz ardı edilir ama dosya sisteminin iç mantığı açısından önemlidir. Ardından gelen john, dosyanın sahibini yani hangi kullanıcıya ait olduğunu belirtir. Hemen sonrasındaki development ifadesi ise dosyanın bağlı olduğu grubu gösterir. Linux'te izinlerin yalnızca kullanıcı bazlı değil, grup bazlı da çalıştığını burada net biçimde görürsün.

Bir sonraki alan olan 4096, dosyanın boyutunu bayt cinsinden ifade eder. Dizinler için bu sayı genellikle 4096 olarak görülür çünkü bu değer dizinin içindeki dosyaları tutmak için ayrılan alanı temsil eder. Jul 29 12:34 kısmı dosyanın en son ne zaman değiştirildiğini gösterir; bu tarih oluşturulma tarihi değil, son değişiklik zamanıdır. En sonda yer alan notes.txt ise dosyanın adıdır.

r,w,x ve – Karakterleri

r (Read – Okuma Yetkisi)

Okuma yetkisi verildiğinde, bir dosyanın içeriğinin görüntülenmesine izin verilmiş olur. Yani dosya açılabilir, içeriği okunabilir fakat üzerinde değişiklik yapılamaz. Bu izin dizinler için geçerli olduğunda ise, dizin içindeki dosya ve klasörlerin listelenebilmesini sağlar. Başka bir ifadeyle, bir dizinde r izni yoksa ls komutu ile içeriği görülemez.

w (Write – Yazma Yetkisi)

Yazma yetkisi, bir dosyanın içeriğinin değiştirilmesine imkân tanır. Dosyanın düzenlenmesi, içeriğinin silinmesi ya da tamamen üzerine yazılması bu izinle mümkün olur. Dizinler söz konusu olduğunda ise anlamı biraz daha kritiktir: w izni, dizin içinde yeni dosya veya klasör oluşturma, mevcut olanları silme yetkisini ifade eder. Bu yüzden dizinlerde yazma izni doğrudan kontrol edilmezse ciddi güvenlik riskleri oluşabilir.

x (Execute – Çalıştırma Yetkisi)

Çalıştırma yetkisi, bir dosyanın program veya betik (script) olarak çalıştırılabilmesini sağlar. Bir dosya teknik olarak çalıştırılabilir bir program olsa bile x izni yoksa sistem bu dosyanın çalıştırılmasına izin vermez. Dizinler için x izni, o dizinin içine girilebilmesini ifade eder. Yani bir dizinde x izni yoksa cd komutu ile o dizine erişim sağlanamaz, bu da dizinin fiilen kapalı olduğu anlamına gelir.

- (Yetki Yok)

Bir izin konumunda r, w veya x yerine - karakteri görülüyorsa, o yetkinin verilmediği anlaşılır. Bu durum, ilgili kullanıcı, grup ya da diğer kullanıcılar için o işlemin açıkça engellendiğini gösterir. Linux'te güvenlik mantığı, izin verilmeyen her şeyin varsayılan olarak yasak kabul edilmesi üzerine kuruludur ve - karakteri bu kuralın açık bir göstergesidir.

User (Kullanıcı), Group (Grup), Others (Diğerleri)

User (u – Dosya Sahibi)

İzin dizisinin ilk üçlü kısmı dosyanın sahibine ait yetkileri gösterir. Bu bölüm, dosyayı veya dizini oluşturan kullanıcı için geçerli olan okuma, yazma ve çalıştırma izinlerini tanımlar. Örneğin rwx------ gibi bir yapıda, yalnızca dosya sahibi tüm yetkilere sahiptir; sistemdeki diğer kullanıcıların dosya üzerinde hiçbir hakkı yoktur. Linux'te sahiplik kavramı temel güvenlik katmanlarından biridir ve dosya üzerinde kimin tam kontrol sahibi olduğunu net biçimde belirler.

Group (g – Grup)

İkinci üçlü kısım, dosyanın ait olduğu grup için tanımlanan izinleri ifade eder. Bu grup, birden fazla kullanıcıyı kapsayabilir ve özellikle ekip çalışması yapılan ortamlarda kritik rol oynar. Örneğin aynı projede çalışan kullanıcıların aynı gruba eklenmesiyle, dosyalar üzerinde ortak okuma veya yazma yetkisi verilebilir. Böylece tek tek kullanıcı bazlı izin tanımlamak yerine, grup üzerinden merkezi bir yetkilendirme yapılmış olur.

Others (o – Diğerleri / World)

Son üçlü kısım ise sistemdeki geri kalan tüm kullanıcılar için geçerli olan izinleri gösterir. Dosyanın sahibi olmayan ve ilgili gruba dahil edilmeyen herkes bu kategoriye girer. Bu bölüm genellikle güvenlik açısından en dikkat edilmesi gereken alandır. Gereksiz yere r veya w yetkisi verilmesi, sistemdeki herhangi bir kullanıcının dosyaya erişebilmesi anlamına gelir. Bu nedenle “others” izinleri çoğu sistemde minimum seviyede tutulur ve yalnızca gerçekten gerekli durumlarda açılır.

Bir Örnek ile İzin Yapısı Nasıl Okunur?

Bu izin ifadesi -rwxr-xr--, bir dosyanın kimler tarafından hangi işlemlere tabi tutulabileceğini net biçimde gösterir. En baştaki - karakteri dosya türünü belirtir; burada bunun bir normal dosya olduğu anlaşılır (eğer d olsaydı bir dizin olduğu ifade edilirdi). Dosya türünden sonraki dokuz karakter ise üçlü gruplar hâlinde izinleri temsil eder ve sırasıyla dosya sahibi (owner), grup (group) ve diğer kullanıcılar (others) için geçerlidir.

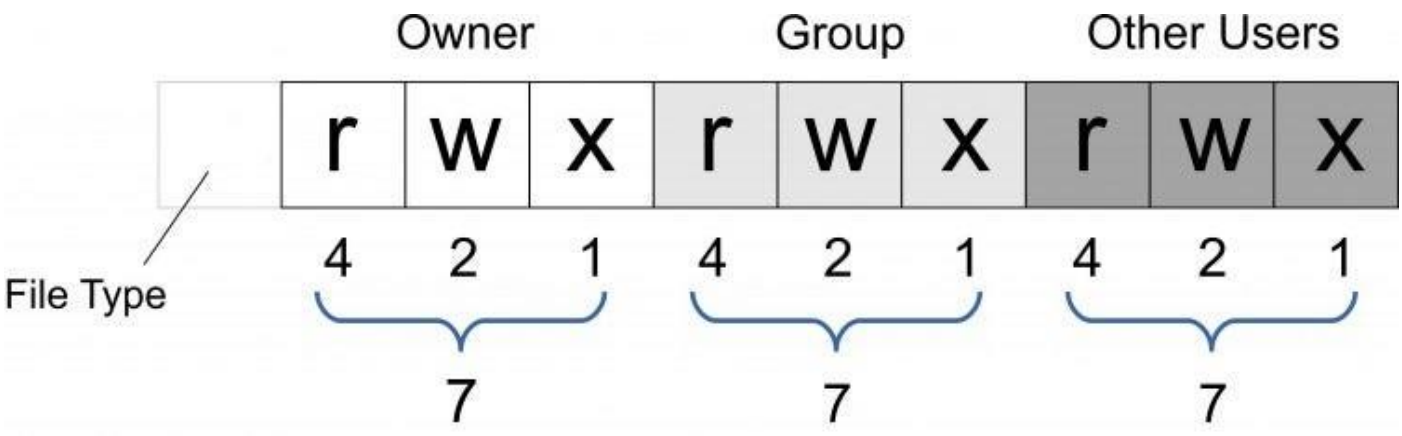
İlk üçlü olan rwx, dosya sahibinin yetkilerini gösterir. Bu durumda dosyanın sahibi olan kullanıcı dosyayı okuyabilir (read), içeriğini değiştirebilir (write) ve dosyayı bir program ya da betik olarak çalıştırabilir (execute). Yani dosya üzerinde tam yetkiye sahiptir. İkinci üçlü olan r-x, dosyaya atanmış grubun yetkilerini ifade eder. Grup üyeleri dosyanın içeriğini okuyabilir ve çalıştırabilir; ancak yazma izni olmadığı için dosya üzerinde değişiklik yapamaz. Üçüncü ve son üçlü olan r-- ise sistemdeki diğer tüm kullanıcıların yetkilerini gösterir. Bu kullanıcılar dosyayı yalnızca okuyabilir; ne değiştirebilir ne de çalıştırabilir.

Verilen örnekteki çıktı şu şekildedir:

-rwxr-xr-- 1 john dev 450 Aug 01 14:00 script.sh

Bu satıra bakıldığında script.sh adlı dosyanın sahibi john’dur ve dosya dev grubuna aittir. John dosyayı okuyabilir, yazabilir ve çalıştırabilir. dev grubuna dahil olan kullanıcılar dosyayı okuyup çalıştırabilir, fakat düzenleyemez. Sistem üzerindeki diğer tüm kullanıcılar ise dosyanın içeriğini yalnızca görüntüleyebilir. Bu yapı, dosyanın kim tarafından değiştirilebileceğini ve kimler için yalnızca okunabilir ya da çalıştırılabilir olacağını açık ve kontrollü bir şekilde tanımlar.

İzin Değerleri (Octal) ve Chmod



Sayısal (Octal) İzinler, Linux'ta dosya ve dizin yetkilerinin kısa, net ve hatasız biçimde ifade edilmesi için kullanılır. Temel prensip şudur: her izin türünün sabit bir sayısal değeri vardır ve bu değerler toplanarak sonuç elde edilir. Okuma izni (read) 4, yazma izni (write) 2, çalıştırma izni (execute) 1 değerine sahiptir. Bir kullanıcıya hangi izinler verilecekse, sadece o izinlerin sayıları toplanır. Örneğin bir kullanıcı hem okuyabilsin hem yazabilsin ama çalıştıramasın deniyorsa 4 + 2 = 6 elde edilir ve bu durum rw- olarak gösterilir.

Bu sistem her zaman üç ayrı kitle için ayrı ayrı uygulanır (Yani bu işlemde 777 girerseniz tüm kullanıcılar tüm izinlere sahip olur, spesifik değil sistem genelinde çalışır.): dosya sahibi (owner), dosyanın grubu (group) ve sistemdeki diğer herkes (others). Her biri için elde edilen sayı yan yana yazılarak üç basamaklı bir octal değer oluşturulur. Örneğin rwxr-xr-- izni ele alındığında, owner için rwx = 4+2+1 = 7, group için r-x = 4+1 = 5, others için r-- = 4 eder. Bu üç değer yan yana getirildiğinde 754 sonucu ortaya çıkar. Yani 754, "sahibi her şeyi yapabilsin, grup okuyup çalıştırsın ama yazamasın, diğerleri sadece okuyabilsin" anlamına gelir.

Bu yaklaşımın gücü, izinleri tek tek harflerle yazmak yerine tek bir sayı dizisiyle net biçimde ifade edebilmesidir. Örneğin chmod 754 script.sh komutu verildiğinde, yukarıda anlatılan izin yapısı doğrudan dosyaya uygulanır. Sayısal sistemin avantajı burada ortaya çıkar: Ne verdiğini bilerek verirsin, fazladan yetki açmazsın. Harfli gösterim görseldir, octal gösterim ise operatiftir. Linux'ta yetki yönetimini ciddiye alan herkesin bu sayısal mantığı içselleştirmesi gerekir; aksi hâlde ya gereksiz yere kısıtlarsın ya da farkında olmadan fazla yetki dağıtılır.

Örnekler ile chmod

1. chmod 755 script.sh
2. chmod 777 file.txt
3. chmod 600 private.txt
4. chmod o+w notes.txt
5. chmod ugo+rwx notes.txt

chmod, bir dosya ya da dizinin kim tarafından ne yapılabileceğini değiştiren komuttur. Bu değişiklik iki farklı mantıkla yapılabilir: sayısal (octal) ve sembolik. Yukarıdaki örneklerin hepsi bu iki mantığın farklı yüzleri.

İlk senaryoda script.sh dosyası başlangıçta -rw-r--r-- durumundadır. Yani dosya okunabilir, yazılabilir, ama çalıştırılamaz. Linux'ta bir dosyanın "script" olması hiçbir şey ifade etmez; x (execute) izni yoksa çalışmaz. chmod 755 script.sh dediğiniz anda dosya -rwxr-xr-x olur. Bunun anlamı şudur: sahibi her şeyi yapar, grup ve diğer kullanıcılar sadece okur ve çalıştırır, ama yazamaz. Script'ler için "doğru" ve güvenli varsayılan budur. Yani burada yapılan şey "herkese yetki vermek" değil, yetkiyi bilinçli dağıtmaktır.

İkinci örnekte chmod 777 file.txt ile dosya -rwxrwxrwx hâline gelir. Bu, dosyanın sistemdeki herkes tarafından okunabileceği, yazılabileceği ve çalıştırılabileceği anlamına gelir. Bu teknik olarak çalışır, evet; ama güvenlik açısından kontrolsüzlük demektir. Eğer bu dosya bir script, konfigürasyon ya da paylaşılan bir dizindeyse, burada artık "yetki" yoktur, kaos vardır. 777 bir çözüm değil, genellikle bir itiraftır: "İzinleri anlamadım."

Üçüncü örnekte chmod 600 private.txt görülüyor. Dosya -rw------- olur. Bu ne demek? Sadece dosyanın sahibi okuyabilir ve yazabilir, başka hiç kimse dosyanın varlığıyla bile ilgilenemez. Kişisel anahtarlar, özel konfigürasyonlar ve hassas bilgiler için doğru yaklaşım budur. Burada güvenlik bilinçli olarak yükseltilmiştir.

Sonraki örnekte sembolik kullanım devreye giriyor: chmod o+w notes.txt. Buradaki o, others (diğer herkes) anlamına gelir, +w ise yazma izni ekle demektir. Yani mevcut izinler bozulmadan, sadece "diğer kullanıcılar" için yazma izni eklenmiştir. Bu yöntem, "her şeyi sıfırdan ayarla" yerine mevcut durumu koruyarak küçük değişiklikler yapmak için kullanılır. İnce ayardır, refleks değil bilinç gerektirir.

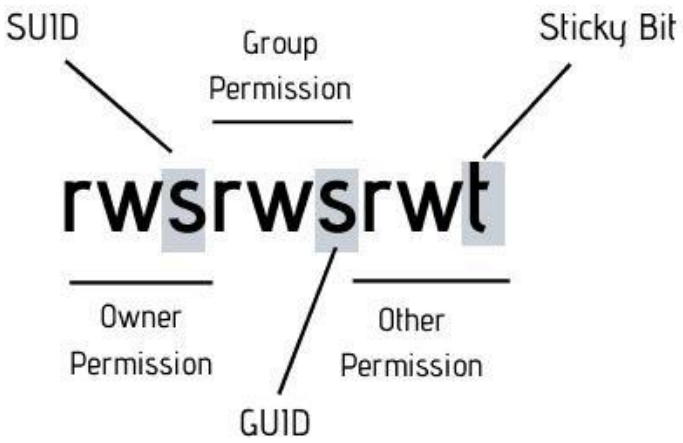
Özel İzinler (SUID, SGID, Sticky)

SUID (Set User ID) – Programın kimin yetkisiyle çalıştığını değiştiren mekanizma

Linux'ta normal davranış şudur: Bir programı kim çalıştırırsa, program o kullanıcının yetkileriyle çalışır. John bir program çalıştırıyorsa program John'dur; root çalıştırıyorsa program root'tur. Sistem varsayılan olarak böyle işler ve bu güvenlidir. Ancak bazı programlar vardır ki sıradan kullanıcılar tarafından çalıştırılması gerekir ama sistem seviyesinde iş yaparlar. En net örnek passwd komutudur. John kendi parolasını değiştirdiğinde /etc/shadow dosyası güncellenir. Bu dosyaya normal kullanıcıların yazma yetkisi yoktur. Eğer passwd komutu John'un yetkileriyle çalışsaydı, bu dosyaya erişemezdi ve parola değiştirilemezdi. Bu problemi çözmek için SUID kullanılır. SUID verilen bir dosya çalıştırıldığında, programı çalıştıran kullanıcının değil, dosyanın sahibinin yetkileriyle çalışır. Yani dosya root'a aitse ve SUID aktifse, kim çalıştırırsa çalıştırsın program geçici olarak root yetkisiyle çalışır. Bu özellik aktif olduğunda ls -l çıktısında dosya sahibine ait çalıştırma izni olan yerde x yerine s görünür. Bu s, "çalıştırılabilir ve SUID aktif" anlamına gelir. Sayısal karşılığı 4000'dir ve normal izinlerin başına eklenir (örneğin 4755). SUID çok güçlü ve aynı zamanda çok tehlikelidir; yanlış bir dosyada kullanılırsa doğrudan yetki yükseltme açığı oluşturur.

SGID (Set Group ID) – Grup bazlı çalışmayı düzenleyen mekanizma

SGID'nin amacı yetki yükseltmek değil, düzenli grup çalışması sağlamaktır. Normalde bir kullanıcı bir dosya oluşturduğunda, dosyanın grubu kullanıcının birincil grubu olur. Bu, ortak proje dizinlerinde ciddi bir problem yaratır çünkü aynı dizin içinde farklı gruplara ait dosyalar oluşur ve grup izinleri anlamsız hâle gelir. Bir dizine SGID verildiğinde bu davranış değişir. O dizin içinde oluşturulan tüm dosyalar, dosyayı kimin oluşturduğuna bakılmaksızın dizinin grubunu miras alır. Böylece bir proje dizininde herkes aynı grup üzerinden dosyalara erişebilir ve yazabilir. Bu, ekip çalışmasının temelidir. SGID aktif olduğunda ls -l çıktısında grup kısmındaki çalıştırma izni yerinde x yerine s görülür. Bu, "bu dizin SGID ile işliyor" demektir. Sayısal karşılığı 2000'dir ve normal izinlerin başına eklenir (örneğin 2770). SGID çoğunlukla dizinlerde kullanılır ve doğru yapılandırıldığında güvenliği artırır, azaltmaz.



Sticky Bit – Herkesin yazabildiği dizinlerde silme yetkisini sınırlayan mekanizma

Sticky Bit'in mantığını anlamak için önce normal durumu bilmek gerekir. Bir dizin üzerinde yazma (w) ve çalıştırma (x) izni olan herkes, o dizin içindeki her dosyayı silebilir. Dosyanın sahibi olması önemli değildir. Bu durum, herkese açık dizinlerde büyük bir güvenlik problemidir. Sticky Bit bu problemi çözmek için vardır. Bir dizine Sticky Bit verildiğinde, dizin yazılabilir olsa bile kullanıcılar yalnızca kendi oluşturdukları dosyaları silebilir. Başkasının dosyasını silemezler. /tmp dizini bunun klasik örneğidir: herkes dosya oluşturabilir ama kimse başkasının dosyasını silemez. Sticky Bit aktif olduğunda ls -l çıktısında others kısmındaki çalıştırma izni yerinde t harfi görülür. Bu t, silme yetkisinin kısıtlandığını gösterir. Sayısal karşılığı 1000'dir ve yine normal 9 izin bitinin başına eklenir (örneğin 1777). Buradaki kafa karışıklığı şundan kaynaklanır: SUID, SGID ve Sticky Bit rwx izinlerinin alternatifi değildir. Onlar ayrı bir katmandır. 9 izin biti (user/group/others) hâlâ vardır; bu üç özel izin onların üstüne eklenen davranış değiştiricilerdir. Bu yüzden sayısal olarak 4 haneli görünürler.

Umask

Umask, kullanıcı tarafından genellikle doğrudan bir şey yapmak için değil, sistemi nasıl davranacağını ayarlamak için kullanılır. Kullanımı umask komutu ile yapılır; komut tek başına çalıştırıldığında mevcut umask değeri görüntülenir, bir sayı ile birlikte kullanıldığında ise yeni varsayılan kural tanımlanmış olur. Örneğin umask 022 komutu, bundan sonra oluşturulacak tüm dosya ve dizinlerde grup ve diğer kullanıcıların yazma izninin otomatik olarak kaldırılacağını söyler. Hesaplama mantığı şudur: Linux, dosya oluştururken teorik olarak dosyalar için 666, dizinler için 777 izinlerini baz alır ve umask değerini bu sayıların üzerinden çıkarır. Yani umask bir "verilecek izin" değil, başlangıçta silinecek izinlerin listesi gibidir. Örneğin umask 022 ise, dosya için 666 − 022 = 644 (rw-r--r--), dizin için 777 − 022 = 755 (rwxr-xr-x) sonucu ortaya çıkar. Bu nedenle kullanıcı her dosya oluşturduğunda chmod yazmak zorunda kalmaz; umask, sistemin baştan güvenli davranmasını sağlayan otomatik bir filtre gibi çalışır.

Umask değeri geçici veya kalıcı olarak ayarlanabilir. Terminalde umask 027 gibi bir komut çalıştırıldığında bu ayar sadece mevcut oturum için geçerli olur ve oturum kapandığında sıfırlanır. Kalıcı bir ayar yapmak istendiğinde ise kullanıcının kabuk yapılandırma dosyalarına (örneğin ~/.bashrc, ~/.profile) veya sistem genelinde uygulanacaksa /etc/profile gibi dosyalara umask 022 benzeri bir satır eklenir. Bu şekilde sistem her oturum açıldığında aynı varsayılan izin kuralı otomatik olarak uygulanmış olur.

Dosya Özellikleri (chattr ve lsattr)

chattr (Change Attribute)

chattr komutu, bir dosya veya dizin üzerinde özel dosya öznitelikleri tanımlamak için kullanılır. Bu öznitelikler, klasik okuma-yazma-çalıştırma (rwx) izinlerinden bağımsızdır ve çok daha katı kurallar uygular. Yani bir dosya için tüm izinler açık olsa bile, chattr ile eklenen bir öznitelik bu izinlerin önüne geçebilir. En yaygın kullanılan öznitelik i (immutable – değiştirilemez) özelliğidir. Bir dosyaya i özniteliği verildiğinde bu dosya silinemez, içeriği değiştirilemez, üzerine yazılamaz ve yeniden adlandırılamaz. Bu durum root kullanıcı için bile geçerlidir. Örneğin sudo chattr +i critical_file.conf komutu çalıştırıldığında bu dosya tamamen kilitlenir. Ardından rm critical_file.conf veya dosya içeriğini düzenlemeye yönelik herhangi bir işlem denendiğinde sistem "Operation not permitted" hatası verir. Dosya üzerinde tekrar değişiklik yapılması gerekiyorsa önce kilidin kaldırılması gerekir; bu da sudo chattr -i critical_file.conf komutu ile yapılır. Bu mekanizma genellikle sistem yapılandırma dosyalarını veya yanlışlıkla silinmesi büyük sorunlara yol açacak dosyaları korumak için kullanılır.

lsattr (List Attributes)

lsattr komutu, bir dosya veya dizin üzerinde hangi özniteliklerin aktif olduğunu görmek için kullanılır. Bu komut, izinleri değil; chattr ile atanmış özel öznitelikleri listeler. Çıktıda dosya adının yanında bazı harfler görülür ve bu harflerin her biri aktif bir özniteliği temsil eder. En sık karşılaşılan harf i'dir ve bu harf dosyanın değiştirilemez olduğunu gösterir. Örneğin lsattr critical_file.conf komutu çalıştırıldığında çıktı şu şekilde olabilir:

----i-------- critical_file.conf

Bu çıktıda görülen i, dosyanın immutable olduğunu açıkça ifade eder. Eğer dosyada böyle bir harf yoksa, bu dosya üzerinde özel bir öznitelik tanımlı değildir ve klasik izin sistemi geçerlidir. lsattr özellikle "neden bu dosyayı silemiyorum veya düzenleyemiyorum?" sorusunun cevabını bulmak için kullanılır. Çünkü bazen dosya izinleri doğru görünmesine rağmen, chattr ile atanmış bir öznitelik işlemleri engelliyor olabilir. Bu nedenle sistem yönetiminde chattr ve lsattr komutları genellikle birlikte kullanılır.

Process Management – Süreç (Process) Nedir ve Nasıl Çalışır?

Linux işletim sisteminde çalışmakta olan her program veya komut bir process (süreç) olarak adlandırılır. Terminalde yazılan basit bir ls komutundan, arka planda çalışan bir web sunucusuna kadar her şey bir süreçtir. Bir süreç başlatıldığında işletim sistemi bu işlem için bellekte yer ayırır, CPU zamanı planlar ve süreci tanımlamak için benzersiz bir PID (Process ID) atar. PID, sistem içinde o süreci diğer tüm süreçlerden ayıran kimlik numarasıdır. Her sürecin aynı zamanda bir sahibi (owner) vardır; bu genellikle süreci başlatan kullanıcıdır ve sürecin yetkilerini belirler. Süreçler hiyerarşik bir yapı ile çalışır: Bir süreç başka bir süreci başlatabilir. Bu durumda başlatan sürece parent (ebeveyn), başlatılan sürece ise child (çocuk) süreç denir. Örneğin bir terminal açıldığında önce bir shell süreci çalışır; bu shell üzerinden Firefox başlatılırsa, shell parent, Firefox ise child süreç olur. Süreç yönetimi; sistemi yavaşlatan işlemleri tespit etmek, kilitlenmiş programları sonlandırmak ve sistem kaynaklarının dengeli kullanılmasını sağlamak açısından kritik öneme sahiptir.

Foreground Processes

Linux'ta bir komut çalıştırıldığında süreç varsayılan olarak foreground (ön plan) modunda başlatılır. Foreground process, klavyeden girdi alır ve çıktısını doğrudan terminale yazdırır. Bu tür süreçler çalıştıkları süre boyunca terminali meşgul eder; yani işlem tamamlanmadan kullanıcı başka bir komut giremez. Örneğin pwd komutu çalıştırıldığında, işlem hemen yürütülür, bulunduğun dizini ekrana yazar ve görevini bitirdiği anda sonlanır. Kısa süren komutlarda bu durum fark edilmezken, uzun süren bir işlem foreground

olarak çalıştırıldığında terminal o süreç bitene kadar kilitli kalır. Bu davranış, kullanıcının süreci doğrudan takip etmesi ve gerektiğinde anında müdahale edebilmesi için tercih edilir.

Background Processes

Background (arka plan) process'ler ise terminali kilitlemeden çalıştırılan süreçlerdir. Bir komutun sonuna & karakteri eklendiğinde, işlem arka planda başlatılır ve terminal hemen tekrar kullanılabilir hâle gelir. Bu sayede aynı anda birden fazla komut paralel olarak çalıştırılabilir. Örneğin ping 127.0.0.1 & komutu çalıştırıldığında, ping işlemi arka planda sürekli devam ederken kullanıcı terminalde başka komutlar yazmaya devam edebilir. Terminalde görünen [1] ifadesi kabuğun verdiği job number'ı (iş numarasını), yanındaki sayı ise sürecin sistem genelindeki PID (Process ID) değerini ifade eder. Bu yapı, özellikle uzun süren veya sürekli çalışan işlemlerin kullanıcıyı bekletmeden yönetilmesini sağlar.

Aktif Terminal Üzerinde İşlemleri (Process) Yönetme

Aynı terminal oturumu içinde başlatılan foreground ve background süreçlerin yönetilmesi için job control mekanizması kullanılır. Arka planda çalışan işlemleri görmek için jobs komutu kullanılır; bu komut, yalnızca mevcut terminal oturumunda başlatılmış süreçleri listeler. Çıktıda görülen [1] gibi ifadeler job number (iş numarası) olarak adlandırılır ve bu numara, kabuk tarafından süreci takip etmek için atanır. Örneğin [1] running ping 127.0.0.1 çıktısı, ping komutunun hâlen arka planda çalıştığını gösterir.

Arka planda çalışan bir sürecin tekrar ön plana alınması gerektiğinde fg komutu kullanılır. fg %1 ifadesindeki %1, birinci job numarasını temsil eder ve ilgili süreç foreground'a taşınır. Bu işlemden sonra süreç tekrar terminali kilitler, klavyeden girdi alır ve çıktısını doğrudan ekrana yazdırır. Bu mekanizma, yanlışlıkla arka plana alınan bir işlemi kontrol altına almak veya etkileşimli hâle getirmek için kullanılır.

System Genelindeki Süreçlerin Yönetimi (ps)

Mevcut terminal oturumuyla sınırlı kalmadan, sistem üzerinde çalışan tüm süreçleri görmek ve analiz etmek için ps komutu kullanılır. ps aux ifadesi, sistemde çalışan bütün süreçlerin anlık bir fotoğrafını (snapshot) alır. Buradaki a parametresi tüm kullanıcıların süreçlerini, u kullanıcı odaklı detaylı çıktıyı, x ise terminale bağlı olmayan (arka planda, servis olarak çalışan) süreçleri de listelemeyi sağlar. Çıktıda her satır bir süreci temsil eder; USER süreci başlatan kullanıcıyı, PID sürecin benzersiz kimlik numarasını, %CPU ve %MEM sırasıyla işlemcinin ve belleğin ne kadarının kullanıldığını gösterir. STAT sütunu sürecin durumunu ifade eder: R çalışıyor, S bekliyor (sleeping), Z ise sonlanmış ama sistemden tamamen temizlenmemiş (zombie) süreç anlamına gelir. COMMAND kısmı ise sürecin hangi komutla başlatıldığını gösterir. ps çıktısı canlı değildir; komut çalıştırıldığı andaki durumu yansıtır ve performans analizi ya da belirli bir süreci tespit etmek için kullanılır.

Canlı Süreç İzleme (top)

Süreçlerin sistem üzerindeki etkisini anlık ve dinamik olarak izlemek gerektiğinde top komutu devreye girer. Bu araç, Windows'taki Görev Yöneticisi'ne benzer şekilde çalışır ve sistem durumu sürekli güncellenen bir ekranda gösterilir. Üst bölümde sistemin ne kadar süredir açık olduğu, kullanıcı sayısı ve load average (sistemin son 1, 5 ve 15 dakikadaki işlem yükü ortalaması) bilgileri yer alır. Hemen altında toplam süreç sayısı ve bunların kaçının çalışır, bekler veya zombie durumda olduğu gösterilir. CPU satırı, işlemcinin hangi işlere ne kadar zaman harcadığını (kullanıcı işlemleri, sistem işlemleri, boşta kalma oranı gibi) detaylı biçimde sunar. Bellek bölümünde ise toplam RAM, kullanılan, boşta olan ve önbelleğe ayrılan bellek miktarları yer alır. Alt kısımda listelenen süreçler, CPU ve bellek kullanımına göre sıralanır; bu sayede sistemi yoran süreçler hızlıca tespit edilebilir. top etkileşimlidir ve q tuşuna basılarak çıkılır.

Çalışan İşlemleri Kapatma (kill)

Linux sistemlerde bir program çalıştırıldığında, işletim sistemi tarafından bir process (işlem) oluşturulmakta ve bu işlem PID (Process ID) adı verilen benzersiz bir numara ile tanımlanmaktadır. Çalışan bir işlemin durdurulması veya kontrol edilmesi gerektiğinde kill komutu kullanılmaktadır. kill komutunun temel işlevi, bir işlemi doğrudan "öldürmek" değil, ilgili işleme bir sinyal göndermek olarak tanımlanmaktadır. Varsayılan olarak kill PID komutu çalıştırıldığında, hedef process'e SIGTERM (15) sinyali gönderilmektedir. Bu sinyal, işlemin nazik bir şekilde kapatılmasını talep eder; açık dosyaların kaydedilmesine, bağlantıların düzgün biçimde sonlandırılmasına ve programın kendi temizliğini yapmasına izin verilmektedir.

Bazı durumlarda gönderilen SIGTERM sinyaline rağmen işlemin sonlanmadığı gözlemlenebilmektedir. Bunun nedeni, işlemin kilitlenmiş olması, sinyali bilinçli olarak yok sayması veya yanıt veremez durumda bulunması olabilmektedir. Bu gibi durumlarda kill -9 PID komutu kullanılarak SIGKILL (9) sinyali gönderilmektedir. SIGKILL sinyali, diğer sinyallerden farklı olarak işlem tarafından yakalanamaz veya engellenemez; işlem kernel tarafından anında sonlandırılmaktadır. Bu yöntemle işlem kapatıldığında dosya kaydı veya temizlik işlemleri yapılmadığından, veri kaybı veya sistem tutarsızlığı oluşabileceği unutulmamalıdır. Bu nedenle SIGKILL, yalnızca son çare olarak tercih edilmelidir.

Sinyaller yalnızca işlemleri kapatmak amacıyla değil, farklı kontrol mekanizmaları için de kullanılmaktadır. SIGHUP (1) sinyali genellikle servislerin yeniden başlatılması veya yapılandırma dosyalarının tekrar okunması amacıyla gönderilmektedir. SIGINT (2) sinyali, terminal üzerinden CTRL+C tuş kombinasyonu kullanıldığında iletilmekte ve işlemin kesilmesi talep edilmektedir. SIGSTOP (19) sinyali ile bir işlem duraklatılmakta, CPU kullanımı durdurulmakta ancak işlem bellekten silinmemektedir. Daha sonra SIGCONT (18) sinyali gönderilerek işlem kaldığı yerden devam ettirilebilmektedir.

pkill Komutu

Sistemde birden fazla Python tabanlı işlem çalıştığı varsayıldığında, bu işlemlerin isimlerinin python, python3 ve python-script olduğu kabul edilmektedir. Bu durumda aşağıdaki komut çalıştırıldığında:

pkill python

Bu komut ile birlikte, process listesi taranmakta ve adı içerisinde “python” ifadesi geçen tüm işlemler hedef alınmaktadır. Yani yalnızca python değil, python3 ve python-script gibi isimler de eşleşme kapsamına alınmaktadır. Eşleşen tüm işlemlere varsayılan olarak SIGTERM (15) sinyali gönderilmekte ve işlemlerin düzgün bir şekilde sonlandırılması talep edilmektedir. Bu örnekten de anlaşılacağı üzere, pkill komutunun kısmi eşleşme yapması nedeniyle aynı anda birden fazla işlemin etkilenmesi mümkün olmaktadır.

Farklı bir senaryoda, yalnızca belirli bir kullanıcıya ait işlemlerin kapatılması istendiğinde aşağıdaki komut kullanılabilmektedir:

pkill -u user1 python

Bu kullanımda, yalnızca user1 kullanıcısına ait ve adı “python” ifadesini içeren işlemler hedef alınmakta, diğer kullanıcıların işlemleri kapsam dışı bırakılmaktadır. Bu örnek, pkill komutunun filtreleme yetenekleri sayesinde daha kontrollü bir şekilde kullanılabileceğini göstermektedir.

Zorlayıcı bir kapatma işlemi gerektiğinde ise aşağıdaki komut tercih edilmektedir:

pkill -9 python

Bu komut ile eşleşen tüm işlemlere SIGKILL (9) sinyali gönderilmekte ve işlemler kernel tarafından anında sonlandırılmaktadır. Bu yöntemin veri kaybı riski taşıdığı göz önünde bulundurulmalıdır.

killall Komutu

Sistemde aynı programın birden fazla kopyasının çalıştığı ve bu programın adının firefox olduğu varsayıldığında, aşağıdaki komut çalıştırılabilmektedir:

killall firefox

Bu komut kullanıldığında, process listesi taranmakta ve adı tam olarak “firefox” olan tüm işlemler hedef alınmaktadır. Eşleşme yalnızca birebir isim üzerinden yapıldığından, örneğin firefox-bin veya farklı bir isimle çalışan işlemler kapsam dışı bırakılmaktadır. Hedeflenen işlemlere varsayılan olarak SIGTERM (15) sinyali gönderilmekte ve işlemlerin düzgün şekilde kapanması talep edilmektedir.

İşlemlerin normal yollarla sonlanmadığı durumlarda, aşağıdaki komut kullanılabilmektedir:

killall -9 firefox

Bu komut ile adı tam olarak “firefox” olan tüm işlemler SIGKILL (9) sinyali ile zorla kapatılmaktadır. Bu durumda da dosya kaydı veya temizlik işlemleri yapılmadan kapatma gerçekleştiği için risklerin bulunduğu kabul edilmektedir.

Başka bir örnekte, sistemde python ve python3 isimli iki farklı işlem çalıştığı varsayıldığında:

killall python

komutu çalıştırıldığında yalnızca adı tam olarak python olan işlemler sonlandırılmakta, python3 işlemleri etkilenmemektedir. Bu örnek, killall komutunun daha sınırlı ve öngörülebilir bir davranış sergilediğini açıkça ortaya koymaktadır.

Süreç Önceliği Ayarlama (nice, renice)

Linux’ta aynı anda çalışan birçok süreç vardır ve hepsi CPU’yu eşit şekilde kullanmaz; işletim sistemi hangisine daha fazla kaynak verileceğine öncelik (priority) değerine bakarak karar verir. Bu öncelik değeri -20 ile 19 arasındadır: sayı küçüldükçe öncelik artar, sayı büyüdükçe öncelik düşer. Varsayılan değer 0’dır. nice komutu, bir programı başlatırken hangi öncelikle çalışacağını belirlemek için kullanılır. Örneğin nice -n 10 tar -czf backup.tar.gz /home dediğinde, yedekleme işlemini bilerek düşük öncelikle başlatmış olursun; bu sayede sistemde aktif çalışan başka uygulamalar (tarayıcı, editör vb.) daha akıcı çalışır ve CPU’yu bu işlem “boğmaz”. renice ise zaten çalışmakta olan bir sürecin önceliğini sonradan değiştirmek için kullanılır. Mesela renice -n -5 -p 5432 komutu, PID’si 5432 olan sürecin önceliğini yükseltir ve sistem bu sürece daha fazla CPU zamanı ayırır. Kritik nokta şu: normal kullanıcılar önceliği genelde düşürebilir, öncelik yükseltmek (negatif değerlere gitmek) çoğu zaman root yetkisi gerektirir; çünkü bu doğrudan sistem performansını etkiler.

Servis Yönetimi (systemd, systemctl)

Modern Linux sistemlerinde arka planda çalışan servislerin (web sunucuları, veritabanları, ağ servisleri gibi) yönetimi systemd adlı sistem tarafından yapılır ve kullanıcı bununla systemctl komutu üzerinden iletişim kurar. Bir servis, terminale bağlı olmadan çalışan ve sistem açıldığında otomatik başlayabilen bir programdır; örneğin nginx, ssh, docker gibi. systemctl status nginx komutu, nginx servisinin çalışıp çalışmadığını, ne zamandır aktif olduğunu ve varsa hatalarını gösterir; bu, servisle ilgili ilk bakılması gereken yerdir. systemctl start nginx servisi başlatır, systemctl stop nginx tamamen durdurur. Eğer bir servisin her sistem açılışında otomatik olarak çalışmasını istiyorsan systemctl enable nginx komutunu kullanırsın; bu komut servisi hemen başlatmaz, sadece boot sırasında otomatik başlatılacaklar listesine ekler. Tersi için disable kullanılır. Özetle systemctl, sistemde sessizce çalışan ama sistemin bel kemiğini oluşturan servisleri kontrol ettiğin merkezdir; süreç yönetimi anlık programlarla ilgilenirken, servis yönetimi sistemin sürekli çalışan altyapısını kontrol eder.

Ağ Yönetimi (Network Management)

Bir Linux sisteminde ağ kavramı, makinelerin birbirini tanıyıp veri alışverişi yapabilmesini sağlayan kimlikler ve yollar bütünü olarak düşünülmelidir. IP adresi, bir cihazın ağ üzerindeki mantıksal kimliğidir ve iletişim bu adres üzerinden kurulur; değiştirilebilir ve ağa göre farklılık gösterebilir. MAC adresi ise ağ kartına üretim aşamasında verilen, donanıma özgü fiziksel kimliktir ve normal şartlarda değişmez; aynı ağ içindeki cihazların birbirini ayırt etmesinde temel rol oynar. Network interface, sistemin ağa bağlandığı somut noktadır; kablolu bağlantılar genelde eth0, kablosuz bağlantılar wlan0 gibi isimlerle temsil edilir ve her IP adresi bu arayüzlere atanır. Gateway ise yerel ağın dış dünyaya açılan kapısıdır; sistem, kendi ağının dışındaki bir IP'ye paket göndereceği zaman bu trafiği doğrudan hedefe değil, önce gateway'e yollar. Kısacası IP "kime gideceğini", MAC "hangi donanıma gittiğini", interface "hangi kapıdan çıkacağını", gateway ise "yerel ağdan nasıl çıkılacağını" belirler.

Kullanılabilir Cihazları Listelemek

Linux'ta bu aygıtların listelenmesi ve durumlarının incelenmesi için geleneksel olarak ifconfig, modern sistemlerde ise ip komutu kullanılmaktadır. Amaç, sistemin hangi ağ arayüzlerine sahip olduğunun, bu arayüzlerin aktif olup olmadığının ve hangi adreslerle yapılandırıldığının anlaşılmasını sağlamaktır.

ifconfig komutu parametresiz olarak çalıştırıldığında, yalnızca aktif (UP durumunda olan) ağ arayüzleri listelenmektedir. Her bir arayüz için çeşitli teknik bilgiler sunulmaktadır. Arayüz adı, ilgili ağ kartını tanımlamak için kullanılmaktadır. inet ifadesi arayüze atanmış IPv4 adresini, inet6 IPv6 adresini göstermektedir. netmask, ağ maskesini ifade ederek hangi IP aralığının aynı ağda bulunduğunu belirtmektedir. broadcast, ağ üzerindeki tüm cihazlara gönderilen yayın adresini temsil etmektedir. ether alanında görülen MAC adresi, ağ kartına özgü fiziksel adres olarak tanımlanmaktadır. RX ve TX alanları ise sırasıyla alınan ve gönderilen veri paketlerini ve bu paketlere ait hata istatistiklerini göstermektedir. Bu bilgiler, ağ bağlantısının çalışıp çalışmadığının ve veri iletiminde sorun olup olmadığının anlaşılması için kullanılmaktadır.

Sistemde tanımlı olmasına rağmen aktif olmayan ağ arayüzlerinin de görüntülenmesi gerektiğinde ifconfig -a komutu tercih edilmmektedir. Bu kullanımda, DOWN durumda olan, yani henüz IP adresi atanmamış veya devre dışı bırakılmış arayüzler de listeye dâhil edilmektedir. Bu özellik, özellikle bir sistemde birden fazla ağ kartı bulunması durumunda, mevcut donanımın eksiksiz şekilde tespit edilmesini sağlamaktadır. Böylece kullanılmayan veya yapılandırılmamış ağ arayüzleri kolaylıkla fark edilebilmektedir.

Güncel Linux sistemlerinde ağ yapılandırması ve izleme işlemleri için ip komutu önerilmektedir. ip addr komutu, ağ arayüzlerini ve bunlara atanmış adresleri daha modern ve tutarlı bir biçimde göstermektedir. Çıktıda yer alan durum bilgileri, arayüzün yazılımsal ve fiziksel olarak aktif olup olmadığını ifade etmektedir. Ayrıca IPv4 ve IPv6 adreslerinin birlikte görüntülenmesi, modern ağ yapılarının daha iyi anlaşılmasını

sağlamaktadır. Bu yaklaşım, ağ yönetimini daha esnek ve güçlü hâle getirdiği için günümüzde standart yöntem olarak kabul edilmektedir.

Bir Ağ Arayüzünü Açma ve Kapatma

Ağ arayüzünü kapatma veya açma işlemi, donanımı fiziksel olarak sökmek anlamına gelmemekte; yalnızca işletim sistemi seviyesinde ağ trafiğinin durdurulması veya yeniden başlatılması olarak değerlendirilmektedir. Bu yöntem genellikle ağ sorunlarının giderilmesi, yapılandırma değişikliklerinin test edilmesi veya güvenlik amaçlı geçici bağlantı kesilmesi durumlarında kullanılmaktadır.

Geleneksel yöntem olarak kullanılan ifconfig aracı ile bir ağ arayüzü down duruma getirildiğinde, ilgili arayüz pasif hâle getirilmekte ve üzerinden veri alışverişi durdurulmaktadır. Aynı arayüz up duruma getirildiğinde ise tekrar aktif edilmekte ve ağ trafiği yeniden başlatılmaktadır. Bu işlem sırasında arayüzün yapılandırması silinmemekte, yalnızca geçici olarak devre dışı bırakılmaktadır. Ancak ifconfig komutunun güncel Linux sistemlerinde artık varsayılan olarak bulunmadığı ve kullanımının önerilmediği bilinmelidir.

Modern Linux dağıtımlarında bu işlemler ip komutu aracılığıyla gerçekleştirilmektedir. ip link set ifadesi kullanılarak bir ağ arayüzü yazılımsal olarak kapatılmakta (down) veya yeniden açılmaktadır (up). Bu yöntem, çekirdek ile daha doğrudan çalıştığı için daha güvenilir ve tutarlı kabul edilmektedir. Ayrıca ip komutu, yalnızca ağ arayüzlerinin durumunu değiştirmekle kalmayıp, adresleme, yönlendirme ve bağlantı yönetimi gibi daha gelişmiş ağ işlemlerinin de yapılmasına olanak sağlamaktadır.

Sonuç olarak, bir ağ arayüzünün açılıp kapatılması işlemi, sistem yöneticilerine ağ bağlantısını kontrol etme ve sorun giderme imkânı sunmaktadır. Güncel sistemlerde bu işlemlerin ip komutu kullanılarak yapılması önerilmekte, ifconfig yöntemi ise daha çok eski sistemler veya geriye dönük uyumluluk kapsamında değerlendirilmektedir.

Örnek kullanım: ip link set eth0 up/ ip link set eth0 down

Ifconfig eth0 up/ifconfig eth0 down

Geçici IP Adresi Atama (Assigning IP Address Temporarily)

Bir Linux sistemin bir ağ üzerinde iletişim kurabilmesi için, ağ arayüzüne bir IP adresi atanması gerekmektedir. Komut satırı üzerinden yapılan IP atamaları geçici olarak kabul edilmektedir. Bu, yapılan yapılandırmanın sistem yeniden başlatıldığında otomatik olarak silineceği anlamına gelmektedir. Bu yöntem genellikle test, eğitim veya geçici ağ bağlantıları için kullanılmaktadır.

Eski Yöntem (ifconfig)

Aşağıdaki komut ile eth0 adlı ağ arayüzüne IP adresi ve ağ maskesi atanmaktadır:

root@kali:~$ ifconfig eth0 172.20.1.110 netmask 255.255.255.0 up

Bu kullanımda:

eth0 → IP adresi atanacak ağ arayüzü

172.20.1.110 → Atanan IPv4 adresi

netmask 255.255.255.0 → Ağ maskesi

up → Arayüzün aktif hâle getirilmesi

Netmask (ağ maskesi), IP adresinin hangi kısmının ağ adresi, hangi kısmının cihaz adresi olduğunu belirlemek için kullanılmaktadır.

255.255.255.0 değeri, ilk 24 bitin ağ adresine, son 8 bitin ise cihazlara ayrıldığını ifade etmektedir.

Modern Yöntem (ip)

Güncel Linux sistemlerinde IP atama işlemi ip komutu ile yapılmaktadır:

sudo ip addr add 172.20.1.110/24 dev eth0

Bu komutta:

ip addr add → Arayüze yeni IP adresi eklenmesi

172.20.1.110/24 → IP adresi ve ağ maskesi

dev eth0 → Hedef ağ arayüzü

Burada kullanılan CIDR (Classless Inter-Domain Routing) gösterimi /24 şeklindedir.

/24 değeri, klasik ağ maskesi olan 255.255.255.0 karşılığıdır.

CIDR – Netmask eşleşmesi:

/8 → 255.0.0.0

/16 → 255.255.0.0

/24 → 255.255.255.0

Bu gösterim, modern ağ yapılandırmalarında standart kabul edilmektedir.

Atamanın doğrulanması için şu komut kullanılabilmektedir:

ip addr show eth0

Promiscuous Mod (Trafik İzleme / Sniffing Modu)

Normal şartlarda bir ağ arayüzü, yalnızca kendi MAC adresine veya broadcast adresine gönderilen paketleri kabul etmektedir. Promiscuous mod etkinleştirildiğinde, bu filtreleme kaldırılmakta ve ağ üzerinden geçen tüm paketler ağ kartı tarafından alınmaktadır.

Bu mod, genellikle: Paket yakalama (sniffing), Ağ trafiği analizi, Güvenlik testleri, Hata ayıklama, amaçlarıyla kullanılmaktadır.

Eski Yöntem (ifconfig)

root@hackerbox:~$ ifconfig eth0 promisc

Bu komut ile eth0 arayüzü promiscuous moda alınmaktadır.

Promiscuous modun kapatılması için:

root@hackerbox:~$ ifconfig eth0 -promisc

Modern Yöntem (ip)

sudo ip link set eth0 promisc on

Promiscuous modun devre dışı bırakılması için:

sudo ip link set eth0 promisc off

Bu işlem geçicidir ve sistem yeniden başlatıldığında otomatik olarak iptal edilmektedir.

MAC Adresi Değiştirme (Changing MAC Address)

MAC adresi (Media Access Control Address), bir ağ arayüzüne üretici tarafından atanan ve ağ üzerinde cihazın fiziksel kimliğini temsil eden benzersiz bir adrestir. Bu adres, Ethernet seviyesinde iletişim için kullanılmakta ve IP adresinden farklı olarak donanıma özgü kabul edilmektedir. Linux sistemlerde MAC adresinin yazılımsal olarak geçici biçimde değiştirilmesi mümkündür. Yapılan değişiklik, sistem yeniden başlatıldığında veya ağ arayüzü yeniden yapılandırıldığında varsayılan hâline geri dönmektedir.

MAC adresi değiştirilirken, arayüzün aktif olmaması önerilmektedir. Bu nedenle işlem üç adımda gerçekleştirilmektedir: arayüzün kapatılması, MAC adresinin değiştirilmesi ve arayüzün tekrar açılması.

MAC Adresi Değiştirme (ifconfig ile)

root@kali:~$ ifconfig eth0 down

root@kali:~$ ifconfig eth0 hw ether AA:BB:CC:DD:EE:FF

root@kali:~$ ifconfig eth0 up

Bu komutlarda yer alan ifadelerin anlamları aşağıda açıklanmaktadır:

Ifconfig: Ağ arayüzlerini yapılandırmak ve yönetmek için kullanılan geleneksel Linux ağ aracıdır.

Eth0: MAC adresi değiştirilecek olan ağ arayüzünü ifade etmektedir.

Down: Belirtilen ağ arayüzünün geçici olarak devre dışı bırakılmasını sağlamaktadır. Bu adım, MAC adresi değişikliği sırasında çakışma veya iletişim hatası yaşanmaması için uygulanmaktadır.

Hw: “hardware” kelimesinin kısaltmasıdır ve donanım seviyesinde bir ayarın yapılacağını belirtmektedir. Bu ifade, yapılacak işlemin IP gibi mantıksal değil, fiziksel adresle ilgili olduğunu göstermektedir.

Ether: Ethernet türündeki ağ arayüzleri için kullanılan donanım adresi tipini ifade etmektedir. Yani bu anahtar kelime, atanacak adresin bir Ethernet MAC adresi olduğunu belirtmektedir.

AA:BB:CC:DD:EE:FF: Yeni atanacak MAC adresini temsil etmektedir. Bu adres, 6 bayttan (48 bit) oluşmakta ve her bayt iki haneli onaltılık (hexadecimal) sayı ile ifade edilmektedir. Bu değer örnek olarak verilmiştir ve gerçek kullanımda geçerli bir MAC adresi formatında olmalıdır.

Up: MAC adresi değiştirildikten sonra ağ arayüzünün yeniden aktif hâle getirilmesini sağlamaktadır.

Bu işlem sonucunda ağ arayüzü, yeni MAC adresi ile çalışmaya başlamaktadır. Ancak bu değişiklik kalıcı değildir ve sistem yeniden başlatıldığında üretici tarafından atanmış olan orijinal MAC adresine geri dönülmektedir.

MAC Adresinin Doğrulanması

MAC adresi değişikliğinin başarılı olup olmadığı aşağıdaki komutlarla kontrol edilebilmektedir:

```
ifconfig eth0
```

veya

```
ip link show eth0
```

Çıktıda görülen ether alanı, arayüzün aktif olarak kullandığı MAC adresini göstermektedir.

Önemli Not

MAC adresinin değiştirilmesi, ağ üzerindeki diğer cihazların ARP tablolarında geçici tutarsızlıklara yol açabilmektedir. ARP tabloları, IP adresleri ile MAC adreslerini eşleştiren yapılardır. Bu nedenle MAC değişikliği işleminin kontrollü ve bilinçli şekilde yapılması gerektiği kabul edilmektedir.

Port ve Bağlantı Kontrolü (netstat ve ss)

Bir Linux sisteminde hangi servislerin çalıştığının, hangi portların dış dünyaya açık olduğunun ve bu portları hangi işlemlerin dinlediğinin bilinmesi, hem sistem yönetimi hem de güvenlik açısından kritik kabul edilmektedir. Açık durumda bulunan gereksiz servisler, saldırı yüzeyini genişletmekte ve sistem için ciddi riskler oluşturmaktadır. Bu nedenle, sistem üzerinde aktif olarak dinleme yapan (LISTEN durumunda olan) portların düzenli olarak kontrol edilmesi gerekmektedir.

Bu kontrol işlemi geçmişte netstat komutu ile yapılırken, modern Linux sistemlerinde aynı işlev daha hızlı ve detaylı şekilde ss komutu ile sağlanmaktadır.

netstat ile Açık Portların Görüntülenmesi (Eski Yöntem)

Aşağıdaki komut kullanılarak sistemde TCP ve UDP üzerinden dinleme yapan tüm servisler görüntülenebilmektedir:

```
┌──(root㉿kali)-[~/Desktop]
└─# netstat -tulpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      1587/sshd: /usr/sbi
tcp6       0      0 :::22                   :::*                    LISTEN      1587/sshd: /usr/sbi
```

Bu komutun çıktısı, yalnızca sunucu (LISTEN) durumundaki bağlantıları göstermektedir. Çıktıda yer alan sütunların anlamları aşağıda açıklanmaktadır:

Proto: Kullanılan protokolü göstermektedir. tcp TCP protokolünü, udp ise UDP protokolünü ifade etmektedir.

Recv-Q (Receive Queue): Uygulama tarafından henüz okunmamış gelen veri miktarını göstermektedir. Normal şartlarda 0 olması beklenmektedir.

Send-Q (Send Queue): Henüz gönderilememiş veri miktarını göstermektedir. Sürekli dolu olması, ağ veya uygulama sorununa işaret edebilmektedir.

Local Address: Servisin dinleme yaptığı IP adresi ve portu göstermektedir.

0.0.0.0:22 ifadesi, servisin tüm ağ arayüzlerinden gelen bağlantıları kabul ettiğini belirtmektedir.

127.0.0.1:5432 ifadesi ise servisin yalnızca yerel makineden erişilebilir olduğunu göstermektedir.

Foreign Address: Bağlantının karşı uç adresini göstermektedir. Dinleme durumunda genellikle * olarak görülmektedir.

State: Bağlantının durumunu göstermektedir.

LISTEN, ilgili servisin bağlantı beklediğini ifade etmektedir.

PID/Program name: Portu kullanan işlemin Process ID (PID) numarasını ve program adını göstermektedir.

Örnek Çıktının Yorumlanması

tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN 950/sshd

Bu satırdan şu bilgiler çıkarılmaktadır:

SSH servisi (sshd) çalışmaktadır.

TCP protokolü kullanılmaktadır.

22 numaralı port dinlenmektedir.

Servis tüm ağ arayüzlerinden erişime açıktır.

İşlemin PID değeri 950'dir.

Bu durum, sistemin uzaktan SSH bağlantılarına açık olduğunu göstermektedir.

ss ile Açık Portların Görüntülenmesi (Modern Yöntem)

Güncel Linux dağıtımlarında netstat artık önerilmemekte, yerine ss komutu kullanılmaktadır. Aynı işlemin modern karşılığı aşağıdaki komutla gerçekleştirilmektedir:

```
┌──(root㉿kali)-[~/Desktop]
└─# ss -tulpn
Netid   State      Recv-Q    Send-Q       Local Address:Port       Peer Address:Port   Process
tcp     LISTEN     0         128                0.0.0.0:22              0.0.0.0:*       users:(("sshd",pid=1587,fd=6))
tcp     LISTEN     0         128                   [::]:22                 [::]:*       users:(("sshd",pid=1587,fd=7))
```

Bu komut, kernel seviyesinde çalıştığı için daha hızlı ve daha detaylı çıktı sağlamaktadır.

Çıktıda yer alan alanların anlamları şu şekildedir:

Netid= Kullanılan protokol türünü göstermektedir (tcp, udp).

State= Bağlantının durumunu göstermektedir. LISTEN, ilgili portun bağlantı beklediğini ifade etmektedir.

Recv-Q / Send-Q= netstat çıktısındaki kuyruk alanları ile aynı anlamı taşımaktadır.

Local Address:Port= Dinleme yapılan IP adresi ve port numarasını göstermektedir.

Peer Address:Port= Karşı uç adresini göstermektedir. Dinleme durumunda genellikle * olarak görülmektedir.

Process= Portu kullanan programın adı, PID bilgisi ve dosya tanımlayıcısı (fd) burada listelenmektedir.

Komutta Kullanılan Parametrelerin Anlamları

-t : TCP protokolünü gösterir

-u : UDP protokolünü gösterir

-l : Dinleme (LISTEN) durumundaki portları listeler

-p : Portu kullanan işlem (program) bilgisini gösterir

-n : Servis ve port adlarını çözümlemeden, sayısal olarak gösterir

Örneğin, -n parametresi kullanılmadığında 22 yerine ssh, 80 yerine http gibi isimler gösterilebilmektedir. Sayısal gösterim, analiz sırasında daha net ve hızlı yorum yapılmasını sağlamaktadır.

Neden Bu Kontrol Önemlidir?

Bu komutlar sayesinde:

Sistemde gereksiz çalışan servisler tespit edilebilmektedir.

Dış dünyaya açık portlar belirlenebilmektedir.

Hangi uygulamanın hangi portu kullandığı net şekilde görülebilmektedir.

Güvenlik duvarı kuralları oluşturulmadan önce mevcut durum analiz edilebilmektedir.

Routing Table (Yönlendirme Tablosu)

Bir Linux sisteminin ağ üzerindeki paketleri hangi yoldan göndereceği, yönlendirme tablosu (routing table) üzerinden belirlenmektedir. İnternete çıkışın mümkün olabilmesi için, sistemin varsayılan ağ geçidini (default gateway) bilmesi gerekmektedir. Bu bilgi, yönlendirme tablosunda tutulmaktadır.

Eski Yöntem: route

Yönlendirme tablosu aşağıdaki komut ile görüntülenebilmektedir:

```
┌──(root㉿kali)-[~/Desktop]
└─# route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         192.168.64.1    0.0.0.0         UG    100    0        0 eth0
192.168.64.0    0.0.0.0         255.255.255.0   U     100    0        0 eth0
```

Bu komutun çıktısında yer alan alanların anlamları şu şekildedir:

Destination Hedef ağı göstermektedir. 0.0.0.0 değeri, tüm bilinmeyen ağları ifade etmekte ve varsayılan yönlendirmeyi temsil etmektedir.

Gateway Paketlerin gönderileceği ağ geçidini göstermektedir. İnternete çıkış için genellikle modem veya router IP’si burada yer almaktadır.

Genmask Hedef ağın ağ maskesini belirtmektedir.

Flags U harfi rotanın aktif olduğunu, G harfi ise bir gateway kullanıldığını göstermektedir.

Iface Hangi ağ arayüzü (örneğin eth0) üzerinden çıkış yapılacağını belirtmektedir.

Örneğin:

0.0.0.0 192.168.1.1 0.0.0.0 UG eth0

Bu satırdan, tüm internet trafiğinin 192.168.1.1 adresine, yani varsayılan ağ geçidine eth0 arayüzü üzerinden gönderildiği anlaşılmaktadır.

Modern Yöntem: ip route

Güncel Linux sistemlerinde yönlendirme bilgileri aşağıdaki komut ile görüntülenmektedir:

ip route

```
┌──(root㉿kali)-[~/Desktop]
└─# ip route
default via 192.168.64.1 dev eth0 proto dhcp src 192.168.64.4 metric 100
192.168.64.0/24 dev eth0 proto kernel scope link src 192.168.64.4 metric 100
```

Örnek bir çıktı şu şekildedir:

default via 192.168.1.1 dev eth0 src 192.168.1.50 metric 100

Bu satır şu şekilde yorumlanmaktadır:

default via 192.168.1.1 → Varsayılan ağ geçidi 192.168.1.1 olarak belirlenmiştir

dev eth0 → Çıkışın eth0 arayüzü üzerinden yapılacağı belirtilmiştir

src 192.168.1.50 → Kaynak IP adresi gösterilmektedir

metric → Birden fazla rota varsa öncelik değerini ifade etmektedir

Bu komut, route komutuna göre daha okunabilir ve sistematik kabul edilmektedir.

Bağlantı Testleri (ping, nc, tracepath)

Bir ağ yapılandırmasının doğru çalışıp çalışmadığı, çeşitli bağlantı testleri ile kontrol edilmektedir.

ping – Erişilebilirlik Testi

ping komutu, bir hedef sistemin erişilebilir olup olmadığını test etmek için kullanılmaktadır. ICMP paketleri gönderilerek karşı taraftan yanıt alınıp alınmadığı kontrol edilmektedir.

ping -c 3 google.com

Bu komutta:

-c 3 → Sadece 3 adet paket gönderilmesini sağlamaktadır

Dönen time değeri → Gecikme süresini (latency) göstermektedir

Yanıt alınması → Ağ bağlantısının çalıştığını göstermektedir

Eğer yanıt alınamazsa, ağ bağlantısında, DNS çözümlemesinde veya hedef sunucuda sorun olduğu düşünülmektedir.

nc (Netcat) – Port Kontrolü

nc (netcat) komutu, belirli bir IP adresindeki belirli bir portun açık olup olmadığını test etmek için kullanılmaktadır. Telnet'in modern ve daha esnek bir alternatifi olarak kabul edilmektedir.

nc -zv 192.168.1.50 22

Bu komutta:

-z → Veri göndermeden sadece port taraması yapılmasını sağlar

-v → Ayrıntılı çıktı verir

22 → SSH portu test edilmektedir

Bağlantı başarılı olursa, ilgili servisin çalıştığı anlaşılmaktadır.

tracepath – Paket Güzergâhı Analizi

tracepath komutu, bir paketin hedefe ulaşana kadar geçtiği router'ları adım adım göstermektedir. Ağ gecikmeleri ve kopmalar bu sayede analiz edilebilmektedir.

tracepath google.com

Bu çıktıda her satır:

Paketin geçtiği bir ağ cihazını (router)

O noktadaki gecikme süresini göstermektedir.

Bu araç, özellikle bağlantının nerede yavaşladığını veya koptuğunu tespit etmek için kullanılmaktadır.

DNS Sorgulama (dig, nslookup)

DNS (Domain Name System), alan adlarını IP adreslerine çeviren sistem olarak tanımlanmaktadır.

dig – Detaylı DNS Sorgusu

dig google.com +short

Bu komut, yalnızca alan adının çözümlendiği IP adresini göstermektedir. Test ve otomasyon işlemlerinde sıkça tercih edilmektedir.

nslookup – Basit DNS Sorgusu nslookup google.com

Bu komut:

Hangi DNS sunucusunun kullanıldığını

Alan adının hangi IP adresine çözümlendiğini göstermektedir.

dig daha detaylı kabul edilirken, nslookup daha basit kullanım sunmaktadır.

DNS Ayarları (/etc/resolv.conf)

Linux sistemlerinde DNS sunucuları /etc/resolv.conf dosyası içinde tanımlanmaktadır. Sistem, alan adı çözümleme işlemlerinde bu dosyada belirtilen DNS sunucularını kullanmaktadır.

Dosya bir metin düzenleyici ile açılabilmektedir:

nano /etc/resolv.conf

Dosya içeriğinde her satır bir DNS sunucusunu temsil etmektedir:

nameserver 1.1.1.1 nameserver 1.0.0.1

Bu örnekte:

Cloudflare DNS sunucuları sistem genelinde tanımlanmıştır

Sorgular sırayla bu sunuculara gönderilmektedir

Bu dosyada yapılan değişiklikler bazı sistemlerde ağ yöneticisi (NetworkManager) tarafından üzerine yazılabilmektedir. Ancak temel DNS mantığının anlaşılması açısından bu dosya kritik öneme sahiptir.

Dosya Transferi (wget, curl, scp)

wget – Dosya İndirme Aracı

wget komutu, internet üzerindeki bir dosyanın komut satırı üzerinden indirilmesi amacıyla kullanılmaktadır. Bu araç, HTTP, HTTPS ve FTP protokollerini desteklemekte ve özellikle grafik arayüz olmayan sistemlerde dosya indirme işlemlerinde tercih edilmektedir. Örneğin, bir web sunucusunda bulunan file.zip adlı dosya indirilmek istendiğinde aşağıdaki komut kullanılmaktadır:

```
wget https://example.com/file.zip
```

Bu komut çalıştırıldığında, belirtilen dosya mevcut dizine indirilmektedir. İndirme işlemi sırasında bağlantı koparsa, wget kaldığı yerden devam edebilme özelliğine sahiptir. Bu yönüyle büyük dosyaların indirilmesinde güvenilir bir araç olarak kabul edilmektedir.

curl – Web İstekleri ve Başlık Analizi

curl komutu, web sunucularına istek göndermek ve dönen yanıtları incelemek amacıyla kullanılmaktadır. Bu araç yalnızca dosya indirmek için değil, aynı zamanda HTTP başlıklarını görüntülemek, API testleri yapmak ve servislerin ayakta olup olmadığını kontrol etmek için de kullanılmaktadır. Örneğin, bir web sitesinin HTTP başlıklarını görmek için aşağıdaki komut çalıştırılmaktadır:

```
curl -I https://google.com
```

Bu kullanımda -I parametresi sayesinde yalnızca sunucunun yanıt başlıkları gösterilmektedir. Dönen HTTP/1.1 200 OK ifadesi, sunucunun isteğe başarıyla yanıt verdiğini göstermektedir. Bu özellik, ağ ve web servislerinin durumunu hızlıca test etmek için sıkça kullanılmaktadır.

scp – SSH Üzerinden Güvenli Dosya Kopyalama

scp (Secure Copy) komutu, dosyaların bir sistemden başka bir sisteme şifreli biçimde aktarılmasını sağlamaktadır. Bu işlem SSH protokolü üzerinden gerçekleştirildiği için veri güvenliği sağlanmaktadır.

Örneğin, yerel sistemde bulunan report.pdf adlı dosyanın uzak bir sunucudaki kullanıcı dizinine kopyalanması için aşağıdaki komut kullanılmaktadır:

scp report.pdf user@192.168.1.50:/home/user/Documents

Bu komut çalıştırıldığında, uzak sistemin parolası sorulmakta ve dosya belirtilen dizine aktarılmaktadır. scp, özellikle yapılandırma dosyaları, raporlar ve yedekler gibi hassas verilerin aktarımında tercih edilmektedir.

SSH (Secure Shell) – Güvenli Uzak Bağlantı

SSH, bir bilgisayardan başka bir bilgisayara ağ üzerinden güvenli şekilde bağlanmayı ve uzaktaki sistemde komut çalıştırmayı sağlayan bir protokol olarak tanımlanmaktadır. SSH bağlantıları şifreli olduğu için, kullanıcı bilgileri ve komutlar ağ üzerinde açık şekilde iletilmemektedir. Bir uzak sisteme bağlanmak için aşağıdaki temel kullanım gerçekleştirilmektedir:

ssh user@ip_address

```
bilgisayar@bilgisayars-MacBook-Air ~ % ssh root@192.168.64.4
root@192.168.64.4's password:
Linux kali 6.12.38+kali-arm64 #1 SMP Kali 6.12.38-1kali1 (2025-08-12) aarch64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Dec 22 09:15:16 2025 from 192.168.64.1
┌──(root㉿kali)-[~]
└─#
```

Örneğin, 192.168.1.100 adresindeki bir sunucuya root kullanıcısı ile bağlanmak için şu komut kullanılmaktadır:

ssh root@192.168.1.100

Bağlantı sırasında uzak sistemin parolası girilmekte ve başarılı olursa komut satırı erişimi sağlanmaktadır.

SSH Servisinin Kurulması ve Başlatılması

SSH bağlantılarının kabul edilebilmesi için, hedef sistemde SSH servisinin kurulu ve çalışır durumda olması gerekmektedir. Debian tabanlı sistemlerde bu işlem aşağıdaki komutlarla gerçekleştirilmektedir:

sudo apt-get update

sudo apt-get install openssh-server

Kurulum tamamlandıktan sonra servis başlatılmaktadır:

sudo systemctl start ssh

Sistemin her açılışında SSH servisinin otomatik olarak çalışması için ise şu komut kullanılmaktadır:

sudo systemctl enable ssh

Bu işlemler tamamlandığında, sistem uzaktan SSH bağlantılarını kabul edebilir hâle gelmektedir.

SSH Anahtar Tabanlı Kimlik Doğrulama (Passwordless Login)

Parola tabanlı SSH bağlantıları yerine, daha güvenli kabul edilen anahtar tabanlı kimlik doğrulama yöntemi kullanılabilmektedir. Bu yöntemde bir anahtar çifti oluşturulmaktadır. Anahtar oluşturma işlemi aşağıdaki komut ile yapılmaktadır:

ssh-keygen

Bu komut çalıştırıldığında bir özel anahtar (private key) ve buna karşılık gelen bir açık anahtar (public key) oluşturulmaktadır. Açık anahtar, uzak sunucuya aşağıdaki komut ile kopyalanmaktadır:

ssh-copy-id user@ip_address

Örneğin:

ssh-copy-id root@192.168.1.100

Bu işlem tamamlandıktan sonra, ilgili kullanıcı uzak sisteme parola girmeden bağlanabilmektedir. Bu yöntem hem güvenliği artırmakta hem de otomasyon işlemlerini kolaylaştırmaktadır.

SSH Yapılandırma Dosyası (sshd_config)

SSH servisinin davranışı /etc/ssh/sshd_config dosyası üzerinden kontrol edilmektedir. Bu dosya içinde SSH portu, root kullanıcısının bağlanıp bağlanamayacağı ve diğer güvenlik ayarları tanımlanmaktadır. Dosya aşağıdaki komutla düzenlenmektedir:

sudo nano /etc/ssh/sshd_config

Örneğin, varsayılan SSH portu olan 22’nin değiştirilmesi için aşağıdaki satır düzenlenmektedir:

Port 2222

Yapılan değişikliklerin geçerli olabilmesi için SSH servisinin yeniden başlatılması gerekmektedir:

sudo systemctl restart ssh

Bu ayarlar sayesinde SSH erişimi daha güvenli hâle getirilebilmektedir.

SON

Okuduğunuz için teşekkürler. Umarım faydalı bir süreç olmuştur. Aşağıdaki formdan isteklerinizi ve önerilerinizi belirtebilirsiniz. Daha fazlası için takipte kalmayı unutmayın!

Öneri formu: https://forms.gle/A7qcfwjwEcKmwhYG6